République Algérienne Démocratique et Populaire
الجمهورية الجزائرية الديمقراطية الشعبية
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
وزارة التعليم العالي والبحث العلمي

المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام الآلي سابقا)
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Final Year Dissertation

For the obtainment of the State Engineer Diploma in Computer Science

Option : Data and Intelligent Systems (SID)

Code Optimization in Quantum Compilation Using Large Language Models

Authors :
M. Arab Hamza
M. Djeghri Lotfi

Supervised by :
Dr. Riyadh Baghdadi (NYUAD)

Defended on June 24, 2026, before the jury composed of :

Mr. Zegour Djamal Eddine	: **ESI - President**
Mrs. Guelfout Nasreddine	: **ESI - Examiner**
Mrs. Sadeg Souhila	: **ESI - Reporter**

Promotion : 2025/2026

Acknowledgements

First and foremost, all praise and gratitude are due to Allah, the Almighty, for granting us the strength, patience, and guidance to complete this work.

We would like to express our appreciation to our supervisor, **Dr. Riyadh Baghdadi**. His invaluable guidance, continuous support, and profound insights have been instrumental throughout this research journey. We are truly grateful for his mentorship, patience, and the time he dedicated to helping us grow.

We would also like to thank our predecessors, **Marouane Benbetka** and **Merwan Bekkar**, for their foundational work and the support they provided, which greatly facilitated the progress of our research.

We also extend our sincere thanks to the faculty and staff of **l'École nationale Supérieure d'Informatique (ESI)** for providing a stimulating academic environment and for their unwavering dedication to our education.

A special and heartfelt thank you goes to our families, and especially our parents. Your unconditional love, endless encouragement, and countless sacrifices have been our driving force. Words cannot express our gratitude for everything you have done to help us reach this milestone. we dedicate this achievement to you.

Finally, we thank everyone who contributed, from near or far, to the completion of this work. Whether through academic assistance, moral support, or a simple word of encouragement, your presence in our journey is deeply appreciated.

Abstract

Running a quantum algorithm on today's quantum computers requires first placing the algorithm's logical qubits onto the physical qubits of the chip, a step known as the qubit mapping problem. This placement matters because two physical qubits can only interact if they sit next to each other on the chip. Whenever the algorithm needs two distant qubits to interact, the compiler must insert extra "SWAP" operations that shuffle qubits into position. These extra operations make the circuit longer and noisier, which lowers the reliability of the final result, so a good qubit mapping is essential. Traditionally, this mapping has been decided either by exact solvers, which only work on very small circuits, or by hand-written rules (heuristics) that are slow to design and hard to adapt as quantum chips grow larger and more varied. To move beyond this manual effort, this work introduces **QPilot**, a framework that uses a Large Language Model (LLM) to automatically synthesize and optimize these mapping heuristics rather than relying on a human to write them. Instead of generating code blindly, QPilot runs a closed feedback loop in which it proposes candidate heuristics, measures how well they perform, and improves them over successive rounds. Guiding this loop is a Hypothesis-Driven Knowledge Graph (HD-KG), which behaves like an automated analyst: it identifies the recurring algorithmic traits of successful heuristics, turns them into clear and testable claims about what makes a placement good, and then checks each claim against real performance data. By steering its evolutionary operators with this accumulated evidence, QPilot keeps refining its strategies while avoiding getting stuck on weak solutions. We evaluate the heuristics generated by framework on both synthetic (QUEKO) and real-world (QASMBench) circuits, targeting the IBM Sherbrooke and Rigetti Ankaa-3 quantum processors. The heuristics it discovers reduce the costly downstream SWAP overhead, and match or rival industry-standard tools such as Qiskit's SABRE, MQT QMAP, and Pytket, offering a flexible and extensible new approach to quantum software optimization.

Keywords : Quantum Circuit Mapping, Automatic Heuristic Design, Large Language Models, Quantum Compilation, NISQ, Hypothesis-Driven Search, Evolutionary Algorithms.

Résumé

L'exécution d'un algorithme quantique sur les ordinateurs quantiques actuels nécessite d'abord de placer les qubits logiques de l'algorithme sur les qubits physiques de la puce, une étape connue sous le nom de problème du mappage des qubits (*qubit mapping problem*). Ce placement est crucial car deux qubits physiques ne peuvent interagir que s'ils sont adjacents sur la puce, chaque fois que l'algorithme exige l'interaction de deux qubits éloignés, le compilateur doit insérer des opérations "SWAP" supplémentaires qui déplacent les qubits pour les mettre en position. Ces opérations superflues allongent le circuit et génèrent du bruit, ce qui réduit la fiabilité du résultat final, un bon mappage des qubits est donc essentiel. Traditionnellement, ce mappage est déterminé soit par des solveurs exacts, qui ne fonctionnent que sur de très petits circuits, soit par des règles empiriques écrites à la main (heuristiques), qui sont lentes à concevoir et difficiles à adapter à mesure que les puces quantiques deviennent plus grandes et plus variées. Pour dépasser cet effort manuel, ce travail introduit **QPilot**, un framework qui s'appuie sur un grand modèle de langage (LLM) pour inventer et optimiser automatiquement ces heuristiques de mappage, plutôt que de s'en remettre à un humain pour les écrire. Au lieu de générer du code à l'aveugle, QPilot exécute une boucle de rétroaction fermée dans laquelle il propose des heuristiques candidates, mesure leurs performances et les améliore au fil des cycles successifs. Cette boucle est guidée par un graphe de connaissances axé sur des hypothèses (HD-KG, pour *Hypothesis-Driven Knowledge Graph*), qui se comporte comme un analyste automatisé : il identifie les composants récurrents des heuristiques performantes, les transforme en affirmations claires et testables sur ce qui rend un placement efficace, puis vérifie chaque affirmation par rapport à des données de performance réelles. En orientant ses opérateurs évolutionnaires grâce à ces preuves accumulées, QPilot affine continuellement ses stratégies tout en évitant de s'enfermer dans des solutions médiocres. Nous évaluons ce framework sur des circuits synthétiques (QUEKO) et réels (QASMBench), en ciblant les processeurs quantiques IBM Sherbrooke et Rigetti Ankaa-3. Les heuristiques qu'il découvre réduisent le surcoût coûteux des opérations SWAP en aval, et égalent ou rivalisent avec les outils standards de l'industrie tels que SABRE de Qiskit, MQT QMAP et Pytket, offrant ainsi une nouvelle approche flexible et évolutive pour l'optimisation des logiciels quantiques.

Mots-clés : Mappage des circuits quantiques, Conception automatique d'heuristiques, grands modèles de langage, Compilation quantique, NISQ (calcul quantique bruyant à échelle intermédiaire), Recherche guidée par des hypothèses, Algorithmes évolutionnaires.

ملخص

يتطلّب تشغيل أيّ خوارزمية كمومية على الحواسيب الكمومية الحالية أن نضع أوّلًا كيوبتات الخوارزمية على الكيوبتات الفيزيائية للرقاقة، وهي خطوةٌ تُعرف بمشكلة تموضع الكيوبتات. ولهذا التموضع أهمّيةٌ بالغة، لأنّ كيوبتَين فيزيائيَّين لا يمكنهما التفاعل إلّا إذا كانا متجاورَين على الرقاقة؛ فكلّما احتاجت الخوارزمية إلى تفاعل كيوبتَين متباعدَين، وجب على المُترجِم إدراج عمليات إضافية تُسمّى SWAP لنقل الكيوبتات إلى مواضع متجاورة. وهذه العمليات الإضافية تجعل الدارة أطول وأكثر عرضةً للضوضاء، ممّا يُقلّل من موثوقية النتيجة النهائية، ولذلك فإنّ التموضع الابتدائيّ الجيّد أمرٌ أساسيّ. وقد جرت العادة على تحديد هذا التموضع إمّا عبر حلّالات دقيقة لا تصلح إلّا للدارات الصغيرة جدًّا، أو عبر قواعد مكتوبة يدويًّا (خوارزميات تقريبية) يصعب تصميمها وتبطؤ مواكبتها لتزايد حجم الرقائق الكمومية وتنوّعها. ولتجاوز هذا الجهد اليدويّ، يُقدّم هذا العمل إطار العمل **QPilot**، وهو إطارٌ يستعين بنموذج لغويّ كبير (LLM) لابتكار خوارزميات التموضع هذه آليًّا بدلًا من الاعتماد على إنسانٍ يكتبها. وبدلًا من توليد الشيفرة بشكلٍ أعمى، يُشغّل QPilot حلقة تغذية راجعة مغلقة يقترح فيها خوارزميات مرشّحة، ويقيس مدى جودة أدائها، ثمّ يُحسّنها عبر جولاتٍ متتالية. ويقود هذه الحلقة بيانٌ معرفيّ قائمٌ على الفرضيّات (HD-KG) يعمل كمُحلِّلٍ آليّ: فهو يحدّد المكوّنات المتكرّرة في الخوارزميات الناجحة، ويحوّلها إلى ادّعاءاتٍ واضحة وقابلة للاختبار حول ما يجعل التموضع جيّدًا، ثمّ يتحقّق من كلّ ادّعاءٍ بمقارنته ببيانات الأداء الفعليّة. ومن خلال توجيه عمليّاته التطوّرية بهذه الأدلّة المتراكمة، يواصل QPilot صقل استراتيجيّاته مع تفادي الوقوع في حلولٍ ضعيفة. وقد قيّمنا الإطار على دارات اصطناعية (QUEKO) وأخرى واقعية (QASMBench)، مستهدفين معالجَي IBM Sherbrooke وRigetti Ankaa-3 الكموميَّين. وتتميّز الخوارزميات التي يكتشفها بأنّها شفّافة وسهلة الفحص، وتُقلّل عبء عمليات SWAP اللاحقة المُكلِفة، وتُضاهي أو تُنافس الأدوات القياسية الرائدة مثل SABRE من Qiskit وMQT QMAP وPytket، مقدّمةً نهجًا جديدًا مرنًا وقابلًا للتوسّع لتحسين البرمجيّات الكمومية.

الكلمات المفتاحية:

تموضع الدارات الكمومية، التصميم الآلي للخوارزميات التقريبية، نماذج اللغات الكبيرة، الترجمة الكمومية، أجهزة NISQ، البحث القائم على الفرضيات، الخوارزميات التطورية.

Contents

General Introduction **1**
Context 1
Problem 1
Objectives 2
Contributions 2
Organization of the Dissertation 3

1 Quantum Computing Fundamentals **4**
1.1 Introduction 4
1.2 Quantum Information Theory 4
1.3 Quantum Circuit Model 7
1.3.1 Single-Qubit Gate Examples 8
1.3.2 Multi-Qubit Gate Examples 8
1.3.3 Gate Construction and Universality 9
1.4 Physical Realizations of Qubits 9
1.5 The Noisy Intermediate-Scale Quantum (NISQ) Era 10
1.6 Physical Constraints of NISQ Devices 10
1.6.1 Decoherence and the Coherence Bottleneck 10
1.6.2 Error Rates and the Challenge of Correction 10
1.6.3 Hardware Connectivity Constraints 11
1.7 Quantum Compilation 11
1.8 Quantum Mapping 13
1.8.1 Complexity 13
1.8.2 Formal Problem Statement 14
1.9 Fidelity and Metric Evaluation 14
1.9.1 Gate and Circuit Fidelity 14
1.9.2 Quantum Volume (QV) 15
1.10 Conclusion 15

2 Qubit Mapping Methods **16**
2.1 Introduction 16
2.2 Taxonomy of Qubit Mapping Methods 17
2.3 Early Exact Approaches 18
2.3.1 Integer Linear Programming (ILP) 18
2.3.2 Boolean Satisfiability (SAT) 18
2.3.3 Satisfiability Modulo Theories (SMT) 19
2.3.4 Comparative Analysis 20

2.4 Learning Based Approaches . . . 21
2.4.1 Reinforcement Learning (RL) and Deep RL (DRL) . . . 21
2.4.2 Graph Neural Networks (GNN) Aided Search . . . 21
2.4.3 Artificial Neural Networks (ANN) . . . 22
2.4.4 Comparative Analysis . . . 22
2.5 Heuristic Approaches . . . 23
2.5.1 A* Search-Based Methods . . . 23
2.5.2 Graph-Based Heuristics . . . 24
2.5.3 Greedy Heuristics . . . 25
2.5.4 Comparative Analysis . . . 28
2.6 Synthesis of Qubit Mapping Methods . . . 29
2.7 Conclusion . . . 31

3 Large Language Models . . . 33
3.1 Introduction . . . 33
3.2 Large Language Models . . . 34
3.2.1 From NLP Models to Foundation Models . . . 34
3.2.2 Transformer Architecture . . . 34
3.2.3 Training Paradigms of Large Language Models . . . 36
3.2.4 Emergent Capabilities of Large Language Models . . . 36
3.3 Intelligent Agents . . . 37
3.3.1 Fundamentals of Agency . . . 37
3.3.2 The Archetypal Properties of Intelligent Agents . . . 37
3.3.3 Taxonomy of Agent Architectures . . . 38
3.3.4 Core Components of an Agentic System . . . 38
3.4 LLM-Based Agents . . . 39
3.4.1 LLMs as Cognitive Engines . . . 39
3.4.2 Agent Architectures . . . 40
3.4.3 Tool-Augmented LLM Systems . . . 40
3.4.4 Reasoning and Planning Strategies . . . 40
3.5 Limitations and Challenges of LLM-Based Agents . . . 42
3.6 LLM Agents for Scientific Discovery and Optimization . . . 42
3.6.1 Key Examples of LLM Usage in Scientific Discovery and Optimization 42
3.6.2 LLMs in Compilation: Taxonomy and Roles . . . 43
3.6.3 The LLM-as-Generator: Prioritizing Development Speed . . . 43
3.7 Conclusion . . . 44

4 LLMs as Heuristic Designers . . . 45
4.1 Introduction . . . 45
4.2 Theoretical Foundations of Hyper-Heuristics . . . 45
4.2.1 The Heuristic Concept . . . 45
4.2.2 Definition and Scope of Hyper-Heuristics . . . 46
4.3 Taxonomy of Hyper-Heuristics . . . 46
4.4 Limitations of Traditional Hyper-Heuristics . . . 48
4.5 Theoretical Foundations of Large Language Models as a Hyper-Heuristic . . . 49
4.5.1 Large Language Models in Algorithm Design . . . 49

4.5.2 Taxonomy of LLM Roles in Algorithm Design . . . 49
4.6 State-of-the-Art LLM-as-a-Heuristic-Designer (LLM-AHD) Frameworks . . . 51
4.6.1 FunSearch: Evolutionary Program Search . . . 51
4.6.2 EoH: Evolution of Heuristics . . . 52
4.6.3 ReEvo: Reflective Evolution . . . 52
4.6.4 MCTS-AHD: Monte Carlo Tree Search for Heuristic Design . . . 53
4.6.5 PartEvo: Niching-Enhanced Evolution . . . 54
4.7 Synthesis . . . 55
4.7.1 Taxonomy: Three Generations of LLM-AHD . . . 55
4.7.2 Evaluation . . . 55
4.7.3 Limitations of the Surveyed LLM-AHD Frameworks . . . 56
4.8 Conclusion . . . 56

5 Design and Implementation . . . 57
5.1 Introduction . . . 57
5.2 Problem Presentation and Motivation for LLM-based AHD . . . 57
5.2.1 The Manual-Design Bottleneck in Qubit Mapping . . . 58
5.2.2 Limitations of Existing LLM-AHD Frameworks . . . 58
5.2.3 From Limitations to QPilot Design Choices . . . 59
5.3 QPilot Framework Architecture and Components . . . 60
5.4 QPilot Automatic Heuristic Search Pipeline . . . 61
5.5 QPilot Automatic Heuristic Design Pipeline in Details . . . 63
5.5.1 Knowledge-Grounded Ideation Stage . . . 63
5.5.2 Implementation and Empirical Evaluation Stage . . . 65
5.5.3 Knowledge Extraction and Hypothesis Generation Stage . . . 67
5.5.4 Evolutionary Refinement and Re-Ideation Stage . . . 70
5.6 Implementation of the QPilot Framework . . . 72
5.6.1 Problem Scope: Initial Qubit Mapping . . . 72
5.6.2 Libraries and Tools Used . . . 72
5.6.3 Large Language Model Selection . . . 73
5.6.4 Anthropic API Integration . . . 74
5.6.5 LLM4AD Framework Adaptation . . . 75
5.6.6 QPilot Configuration Parameters . . . 75
5.7 Conclusion . . . 76

6 Evaluation . . . 78
6.1 Introduction . . . 78
6.2 Experimental Setup . . . 78
6.2.1 Evaluation Metrics . . . 79
6.2.2 Baselines for Comparison . . . 79
6.2.3 Benchmark Circuits . . . 81
6.2.4 Target Quantum Hardware Architectures . . . 82
6.2.5 Experimental Environment . . . 82
6.2.6 Experimental Protocol . . . 83
6.3 Results and Analysis . . . 83
6.3.1 Search Progress and Convergence . . . 83

6.3.2 Evaluation on IBM Sherbrooke backend 84
6.3.3 Evaluation on Rigetti Ankaa-3 backend 89
6.3.4 Mapping-Time Comparison. 95
6.3.5 Token Consumption and Generation Efficiency 97
6.3.6 Ablation Study: Hypothesis-Driven Knowledge Graph and Evolutionary Refinement 98
6.4 Conclusion 99

Conclusion **101**

List of Figures

1.1 *The Bloch Sphere representation of a single qubit state $|\psi\rangle$.* . . . 6
1.2 *Matrix representation of a two-qubit quantum gate.* . . . 7
1.3 *Example of a quantum circuit diagram.* . . . 8
1.4 *Equivalence between a SWAP gate and three CNOT gates.* . . . 9
1.5 *IBM Hanoi backend architecture, generated using qiskit (IBM 2024)* . . . 11
1.6 *The Quantum Compilation Pipeline* . . . 13

2.1 *Taxonomy of qubit mapping methods.* . . . 17

3.1 *The Transformer Architecture (Vaswani et al. 2017)* . . . 35
3.2 *Scaled Dot-Product Attention (Vaswani et al. 2017)* . . . 35
3.3 *Multi-Head Attention (Vaswani et al. 2017)* . . . 35
3.4 *The perception-action loop in intelligent agents (Wooldridge 1999)* . . . 37
3.5 *Overview of LLM-based agent architecture (Dong et al. 2024)* . . . 39
3.6 *Visual comparison of CoT and ToT reasoning strategies (Yao et al. 2023a).* . . . 41
3.7 *The iterative self-reflection loop for error correction (Shinn et al. 2023).* . . . 41
3.8 *Overview of LLM-as-Generator methodology (S. Zhang et al. 2026)* . . . 43

4.1 *A classification of hyper-heuristic approaches. (E. K. Burke et al. 2010)* . . . 47
4.2 *LLM as Optimizer (LLMaO) (F. Liu et al. 2026)* . . . 50
4.3 *LLM as Predictor (LLMaP) (F. Liu et al. 2026)* . . . 50
4.4 *LLM as Extractor (LLMaE) (F. Liu et al. 2026)* . . . 50
4.5 *LLM as Designer (LLMaD) (F. Liu et al. 2026)* . . . 51
4.6 *Overview of FunSearch (Romera-Paredes et al. 2024)* . . . 52
4.7 *Overview of EoH (Fei Liu et al. 2024a)* . . . 52
4.8 *Overview of ReEvo (H. Ye et al. 2024)* . . . 53
4.9 *Overview of MCTS-AHD (Z. Zheng et al. 2025)* . . . 54
4.10 *Overview of PartEvo (Hu and Q. Zhang 2026)* . . . 54

5.1 *Architecture of the QPilot framework.* . . . 60
5.2 *Global pipeline of the QPilot framework for Automatic Heuristic Design.* . . . 62
5.3 *Internal structure of the ideas generation stage.* . . . 65
5.4 *Internal structure of the implementation stage.* . . . 66
5.5 *An illustrative Hypothesis-Driven Knowledge Graph.* . . . 68
5.6 *Internal workflow of the Trait Extraction step* . . . 69
5.7 *Internal workflow of the Hypothesis Generation phase* . . . 69
5.8 *Internal workflow of the Evolutionary Refinement stage.* . . . 71
5.9 *Performance of LLMs on the SWE-bench and Humanity's Last Exam benchmarks.* 74

6.1 *Coupling graphs of the target hardware architectures used in the evaluation.* . . 82
6.2 *Search trajectory of a complete QPilot run on the IBM Sherbrooke backend.* . . . 84
6.3 *SWAP gate counts on the IBM Sherbrooke backend across QUEKO benchmarks.* . 84
6.4 *Final circuit depths on the IBM Sherbrooke backend across QUEKO benchmarks.* . 85
6.5 *SWAP gate counts on the IBM Sherbrooke backend across QUEKO benchmarks, QPilot against LLM-based AHD frameworks.* 87
6.6 *Final circuit depths on the IBM Sherbrooke backend across QUEKO benchmarks.* . 87
6.7 *SWAP gate counts on the Rigetti Ankaa-3 backend across QUEKO benchmarks.* . 89
6.8 *Final circuit depths on the Rigetti Ankaa-3 backend across QUEKO benchmarks.* . 90
6.9 *SWAP gate counts on the Rigetti Ankaa-3 backend across QUEKO benchmarks, comparing QPilot against alternative LLM-based AHD frameworks.* 92
6.10 *Final circuit depths on the Rigetti Ankaa-3 backend across QUEKO benchmarks.* . 92
6.11 *Average initial mapping time per production compiler on the QASMBench* Medium *and* Large *benchmark sets.* 96
6.12 *Average initial mapping time of the LLM-based AHD heuristics on the QASMBench* Medium *and* Large *benchmark sets.* 97
6.13 *LLM token consumption and heuristic generation efficiency of the AHD frameworks.* 98
6.14 *SWAP counts and final circuit depths on the Rigetti Ankaa-3 QUEKO suites, comparing the full QPilot heuristic against its two ablated variants:* without refinement *(ideation-only) and* without HD-KG. 99

List of Tables

2.1 Synthesis and Comparison of Qubit Mapping Methods. 30

4.1 Taxonomy of the five surveyed LLM-AHD frameworks. 55

5.1 Mapping between the limitations of existing LLM-AHD frameworks and the corresponding QPilot design choices. 59
5.2 Description of experimental settings 76

6.1 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the IBM Sherbrooke backend (QPilot vs Production Compilers). 85
6.2 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the IBM Sherbrooke backend (QPilot vs Production Compilers). 86
6.3 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the IBM Sherbrooke backend (QPilot vs AHD Frameworks). 88
6.4 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the IBM Sherbrooke backend (QPilot vs AHD Frameworks). 89
6.5 Average SWAP count and final circuit depth on the 16 qubit QUEKO benchmark targeting the Rigetti Ankaa-3 backend. 90
6.6 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the Rigetti Ankaa-3 backend (QPilot vs Production Compilers). 91
6.7 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the Rigetti Ankaa-3 backend (QPilot vs Production Compilers). 92
6.8 Average SWAP count and final circuit depth on the 16 qubit QUEKO targeting the Rigetti Ankaa-3 backend. 93
6.9 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the Rigetti Ankaa-3 backend. . . . 94
6.10 Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the Rigetti Ankaa-3 backend. . 94
6.11 Average initial mapping time per compiler on the QASMBench *Medium* and *Large* benchmark sets, for the IBM Sherbrooke and Rigetti Ankaa-3 backends. 95
6.12 Average initial mapping time of the LLM-based AHD heuristics on the QASMBench *Medium* and *Large* benchmark sets, for the IBM Sherbrooke and Rigetti Ankaa-3 backends. 96

List of Acronyms

AD	Algorithm Design
AHD	Automatic Heuristic Design
ANN	Artificial Neural Network
API	Application Programming Interface
AST	Abstract Syntax Tree
BDI	Belief-Desire-Intention
BERT	Bidirectional Encoder Representations from Transformers
BPE	Byte Pair Encoding
BPP	Bin Packing Problem
CN	Crossover within Niches
CNOT	Controlled-NOT
CO	Combinatorial Optimization
CoT	Chain-of-Thought
CSV	Code Similarity Vector
CVRP	Capacitated Vehicle Routing
DAG	Directed Acyclic Graph
DAI	Distributed Artificial Intelligence
DPO	Direct Preference Optimization
DRL	Deep Reinforcement Learning
EOH	Evolution of Heuristics
F	Fidelity
F2	Two-qubit gate fidelities

GNN	Graph Neural Network
GP	Genetic Programming
H	Hadamard
HD-KG	Hypothesis-Driven Knowledge Graph
HH	Hyper-Heuristic
HPC	High-Performance Computing
ICL	In-Context Learning
ILP	Integer Linear Programming
LGE	Local Genetic Evolution
LLH	Low-Level Heuristic
LLM	Large Language Model
LLM-AHD	Large Language Models for Automated Heuristic Design
LLMaD	LLM as Designer
LLMaE	LLM as Extractor
LLMaO	LLM as Optimizer
LLMaP	LLM as Predictor
MAS	Multi-Agent Systems
MCTS	Monte Carlo Tree Search
MDP	Markov Decision Process
MQT	Munich Quantum Toolkit
NFL	No Free Lunch
NISQ	Noisy Intermediate-Scale Quantum
NLP	Natural Language Processing
OPRO	Optimization by PROmpting
PST	Probability of Success Threshold
QAOA	Quantum Approximate Optimization Algorithm
QASM	Quantum Assembly Language
QEC	Quantum Error Correction

QPU	Quantum Processing Unit
QUEKO	QUantum Examples with Known Optimal
QV	Quantum Volume
RAG	Retrieval-Augmented Generation
RE	Random Evolution
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
SABRE	SWAP-based BidiRectional heuristic search
SAT	Boolean Satisfiability
SE	Structured Evolution
SMT	Satisfiability Modulo Theories
ToT	Tree-of-Thoughts
TSP	Traveling Salesman Problem
VQA	Variational Quantum Algorithms
VQE	Variational Quantum Eigensolver

General Introduction

Context

Quantum computing promises to solve certain problems that lie far beyond the reach of classical machines, from simulating molecules to factoring large numbers. The machines available today, however, belong to the Noisy Intermediate-Scale Quantum (NISQ) era: they hold only a limited number of qubits, their operations are imperfect, and the quantum information they store decays quickly. Before a quantum algorithm can run on such a machine, it must pass through a *compilation* pipeline that rewrites the idealized algorithm into a form the hardware can actually execute. A central step of this pipeline deals with a simple physical fact: on a real chip, a qubit can only interact directly with its immediate neighbours. Two qubits that the algorithm wants to entangle may sit far apart on the chip, so the compiler must first decide where to place each qubit and then move qubits around during execution. The quality of these decisions has a direct effect on how reliably the final result comes out, which makes compilation a decisive factor in turning today's noisy hardware into a useful computational tool.

Problem

This work focuses on the *qubit mapping* problem, and more precisely on the *initial mapping*: choosing the starting placement of the algorithm's qubits on the chip's physical qubits. Because non-adjacent qubits cannot interact directly, a poor initial placement forces the compiler to insert many extra position-swapping, state-exchanging SWAP operations later on to bring distant qubits together. These extra operations lengthen the circuit and expose it to more noise, which lowers the reliability of the computation. A good initial mapping is therefore essential to keep this overhead small. Finding such a mapping is hard. Exact solvers can find optimal placements but only for very small circuits, so in practice, the task is handled by hand-written rules, known as heuristics. Designing these heuristics is a slow, intuition-driven craft. Each new optimization objective or hardware topology typically requires an expert to re-tune the rules by hand, and the result rarely transfers across the growing variety of quantum chips. In parallel, recent Large Language Model (LLM) based methods have shown that the design of algorithms can itself be automated, yet this idea has not been brought to quantum compilation, and the existing automatic-design frameworks suffer from recurring weaknesses such as unstructured feedback and exploration that ignores past failures. Together, these gaps leave the design of qubit mapping heuristics largely manual and difficult to scale.

Objectives

The overall goal of this dissertation is to replace the manual design of initial qubit mapping heuristics with an automatic, evidence-driven process. By leveraging the generative and optimization capabilities of Large Language Models (LLMs), this work aims to show that such automated processes can produce heuristics competitive with those crafted by experts. This goal is pursued through the following general objectives:

- Reduce the reliance on slow, intuition-driven manual engineering when designing qubit mapping heuristics.
- Leverage the recent progress of Large Language Models for the automatic design of algorithms, and bring this capability to the quantum compilation domain for the first time.
- Obtain mapping heuristics that lower the downstream routing overhead while remaining transparent, fast to apply, and applicable across different hardware topologies.
- Assess the resulting heuristics rigorously against both industry-standard compilers and existing automatic-design methods on a range of benchmarks.

Contributions

The primary contribution of this dissertation is **QPilot**, an automated, LLM-driven framework that synthesizes and optimizes initial qubit mapping heuristics. Using a closed-feedback loop, **QPilot** autonomously designs transparent, hardware-aware strategies, bypassing the need for manual engineering or data-heavy machine learning models. The specific contributions are summarized below:

- **QPilot Framework:** We introduce a novel ”LLM as a Heuristic Designer” architecture that automates the design of quantum mapping heuristics. This marks the first time an LLM is used in a generative capacity to improve compiler performance.
- **Hypothesis-Driven Knowledge Graph (HD-KG):** We propose a structured HD-KG that tracks and validates successful performance hypotheses to guide the LLM’s evolutionary search.
- **Mitigation of the Mapping Bottleneck:** We apply our framework to the initial mapping problem, enabling it to autonomously discover human-readable heuristics that minimize mapping overhead.
- **Rigorous Empirical Validation:** We validate our framework across synthetic and real-world benchmarks, demonstrating that our approach matches or outperforms industry-standard tools.

Organization of the Dissertation

The remainder of this dissertation is organized as follows:

- **Chapter 1 – Quantum Computing Fundamentals.** This chapter establishes the background needed to follow the rest of the dissertation. It introduces the basic building blocks of quantum computation, namely qubits, quantum gates, and quantum circuits, and explains the physical constraints of current quantum machines, in particular their limited qubit connectivity, that make a compilation step unavoidable.
- **Chapter 2 – Qubit Mapping Methods.** This chapter focuses on the problem at the heart of the dissertation. It formally defines the qubit mapping task, explains why finding a good mapping is computationally hard, and reviews the existing solution methods, ranging from exact solvers that are accurate but do not scale, to the hand-crafted heuristics that are used in practice.
- **Chapter 3 – Large Language Models.** This chapter provides the background on Large Language Models required to understand the proposed approach. It explains, at a high level, how these models are built and trained, and highlights the reasoning and code-generation capabilities that make them suitable for designing algorithms.
- **Chapter 4 – LLMs as a Heuristic Designer.** This chapter reviews the emerging field in which Large Language Models are used to design algorithms automatically. It analyzes the most representative frameworks, compares their search strategies, and identifies the recurring limitations that the present work sets out to address.
- **Chapter 5 – Design and Implementation.** This chapter presents the main contribution of the dissertation, the QPilot framework. It describes the overall architecture and the search pipeline through which qubit mapping heuristics are automatically generated, evaluated, and progressively refined, together with the concrete implementation choices.
- **Chapter 6 – Evaluation.** This chapter assesses the proposed framework experimentally. It compares the automatically discovered heuristics against both industry-standard compilers and competing automatic-design methods, using synthetic and real-world benchmarks on several hardware backends, and analyzes the results in terms of mapping quality, runtime cost, and generalization.

Finally, a general conclusion summarizes the contributions of this dissertation and outlines promising directions for future work.

Chapter 1

Quantum Computing Fundamentals

1.1 Introduction

Quantum computing represents a profound paradigm shift in the computational domain, leveraging the counterintuitive principles of quantum mechanics to process information in ways that fundamentally differ from classical systems (Nielsen and Chuang 2010). This enables unprecedented computational power and exponential speedups for specific classes of complex problems (Feynman 1986).

However, realizing the full potential of quantum computation presents immense engineering and theoretical hurdles. Bridging the gap between idealized algorithmic theories and physical hardware execution introduces formidable constraints. These challenges are particularly pronounced today, in what is known as the Noisy Intermediate-Scale Quantum (NISQ) era (Preskill 2018). During this stage, available quantum processors have a limited number of qubits, suffer from high operational error rates, exhibit rapid decoherence due to environmental noise, and impose severe topological constraints on how qubits can physically interact. Consequently, achieving reliable execution of quantum circuits demands a rigorous understanding of both the abstract quantum theory and the strict physical limitations of real-world devices.

This chapter establishes the foundational concepts required to understand this work. It briefly explores Quantum Information Theory, the Quantum Circuit Model, and practical hardware limitations. Finally, it outlines Quantum Compilation (Siraichi et al. 2018), the multi-stage process of translating abstract algorithms into optimized hardware instructions, paving the way for the advanced routing methodologies explored subsequently.

1.2 Quantum Information Theory

In classical computing, information is represented using bits, which can take one of two possible values: 0 or 1. These values are typically encoded physically using two distinct and stable states of an electronic system, such as low and high voltage levels, with logical operations governed by Boolean algebra and executed via digital circuits (Nielsen and Chuang 2010).

Quantum computing generalizes this framework through the qubit, the fundamental unit

of quantum information. Unlike its classical counterpart, a qubit is described mathematically using vectors within a complex vector space known as a **Hilbert space** (Nielsen and Chuang 2010).

A defining feature of quantum mechanics is **superposition** (Nielsen and Chuang 2010), which allows a qubit to exist in a linear combination of basis states simultaneously, rather than being restricted to a strict 0 or 1. However, a qubit is not permanently locked in this blended state it can also occupy a pure basis state.

When a qubit in superposition is measured, it probabilistically collapses into one of the computational basis states ($|0\rangle$ or $|1\rangle$). This outcome is fundamentally probabilistic, meaning the qubit does not harbor a hidden, predetermined classical value before measurement. Furthermore, the **no-cloning theorem** dictates that it is physically impossible to create an identical copy of an arbitrary, unknown quantum state (Born 1955).

In quantum mechanics, a value can be represented with the Dirac notation (Dirac 1981). The classical states 0 and 1 turn into $|0\rangle$ and $|1\rangle$. A qubit state $|\psi\rangle$ would be:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

α and β are complex numbers that satisfy $|\alpha|^2 + |\beta|^2 = 1$. α and β represent the probability amplitudes. $|\alpha|^2$ and $|\beta|^2$ are the probability of collapsing into $|0\rangle$ or $|1\rangle$ (Born 1955).

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

To simplify the representation of qubit states, the Bloch sphere was introduced. A geometric way to represent the state of a single qubit as a point on the surface of a unit sphere in 3D space. A qubit state $|\psi\rangle$ would be:

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle$$

where θ and φ are angles $0 \leq \theta \leq \pi$ and $0 \leq \varphi < 2\pi$. θ measures the "latitude" (from the north pole $|0\rangle$ to the south pole $|1\rangle$) and φ measures the "longitude" around the equator (Nielsen and Chuang 2010).

Another fundamental phenomenon in quantum computation is **entanglement** (Horodecki et al. 2009), describing a state where two or more quantum systems become so deeply correlated that their individual identities merge in such a way that their states cannot be described independently, even if the particles are far apart.

Mathematically, we represent the state of multiple qubits as a vector in a joint Hilbert space, constructed via the tensor product of individual qubit spaces (Nielsen and Chuang 2010). For a two-qubit system, the state $|\psi\rangle$ is a linear combination of all possible basis states:

$$|\psi\rangle = \alpha\,|00\rangle + \beta\,|01\rangle + \gamma\,|10\rangle + \delta\,|11\rangle$$

where $\alpha, \beta, \gamma, \delta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1$.

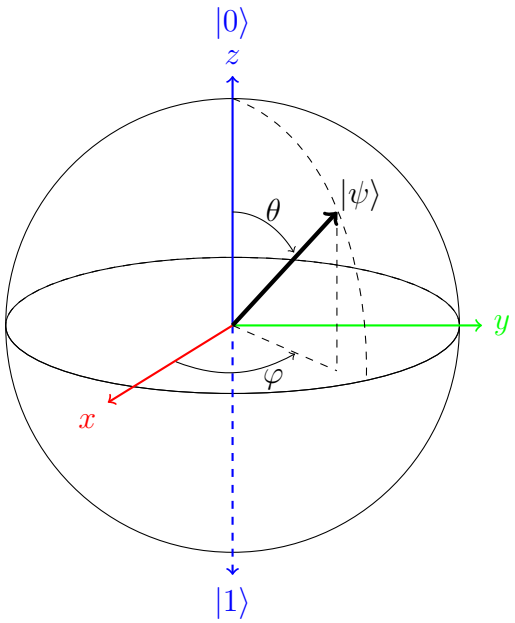


Figure 1.1: The Bloch Sphere representation of a single qubit state $|\psi\rangle$.

The state is entangled if it cannot be factorized into a tensor product of individual states, i.e., it cannot be written in the form:

$$|\psi\rangle = |\psi_A\rangle \otimes |\psi_B\rangle$$

Beyond its theoretical definition, entanglement serves as the engine for quantum speedup. When combined with other quantum principles, it transforms how information is processed and structured.

Through **superposition**, a quantum computer can express a global state as a linear combination of 2^n computational basis states simultaneously (Nielsen and Chuang 2010), creating an exponential mathematical workspace that scale-free dwarfs any classical memory layout. Rather than evaluating these possibilities individually, a common misconception known as the **quantum parallelism fallacy**, the true power emerges when these states are coupled.

Entanglement then bridges these individual states, ensuring that logical operations manipulate a single, non-separable global wavefunction rather than isolated physical qubits (Nielsen and Chuang 2010). This global correlation allows a quantum algorithm to exploit **interference** across the entire state space (Grover 1996).

During an algorithm's execution, complex-valued probability amplitudes are strategically manipulated so that computational paths leading to incorrect solutions cancel each other out via **destructive interference** (Grover 1996). Concurrently, the probability amplitudes of the correct solutions are amplified through **constructive interference** (Grover 1996). Consequently, when a final measurement is performed, the system collapses onto the correct answer with high probability, navigating vast problem spaces with an efficiency that classical architectures cannot natively replicate.

1.3 Quantum Circuit Model

Quantum gates are the fundamental building blocks of quantum circuits (Feynman 1986), analogous to how logic gates (like AND, OR, and NOT) work in classical computing. While classical gates manipulate binary bits (0s and 1s), quantum gates manipulate qubits.

A quantum gate is a reversible transformation represented by a unitary matrix. In quantum mechanics, a matrix U is unitary if $U^\dagger U = UU^\dagger = I$, where $U^\dagger$ is the conjugate transpose (Dirac 1981). This condition ensures that the gate preserves the norm of the quantum state, meaning the total probability remains equal to one after the operation. This reversibility is a defining characteristic: any quantum operation can be "undone" by applying its inverse (the Hermitian conjugate).

When a gate acts on a qubit (or a set of qubits), it rotates the state vector of that qubit within a complex vector space. In stark contrast, a classical logic gate is typically an irreversible, non-linear operation.

Quantum computation is conceptualized through two primary frameworks that bridge the gap between abstract math and physical execution:

- **Mathematical Representation:** Every quantum operation is a unitary transformation. A gate acting on n qubits is represented by a $2^n \times 2^n$ unitary matrix (Dirac 1981). By multiplying this matrix with the system's state vector, the operation transforms the complex amplitudes associated with each basis state. Consequently, a two-qubit gate requires a 4×4 matrix to describe these probability-shifting interactions, as shown in Figure 1.2.

	$\|00\rangle_{in}$	$\|01\rangle_{in}$	$\|10\rangle_{in}$	$\|11\rangle_{in}$
$\langle 00\|_{out}$	$M_{00,00}$	$M_{00,01}$	$M_{00,10}$	$M_{00,11}$
$\langle 01\|_{out}$	$M_{01,00}$	$M_{01,01}$	$M_{01,10}$	$M_{01,11}$
$\langle 10\|_{out}$	$M_{10,00}$	$M_{10,01}$	$M_{10,10}$	$M_{10,11}$
$\langle 11\|_{out}$	$M_{11,00}$	$M_{11,01}$	$M_{11,10}$	$M_{11,11}$

Figure 1.2: Matrix representation of a two-qubit quantum gate.

  Each entry $M_{out,in}$ dictates how the probability amplitude of an initial state contributes to the probability of observing a final state after the quantum operation.

- **Circuit Diagrams:** This is the visual language of quantum algorithms. Each horizontal line (or "wire") represents the timeline of a single qubit. Gates are depicted as boxes or symbols placed on these wires. Operations are read from left to right, mirroring the sequential evolution of the quantum state over time (Barenco et al. 1995).

The circuit depth represents the maximum number of sequential gates along the longest path from input to output, effectively determining the minimum time required to execute the algorithm if parallel operations are performed simultaneously (Nielsen and Chuang 2010).

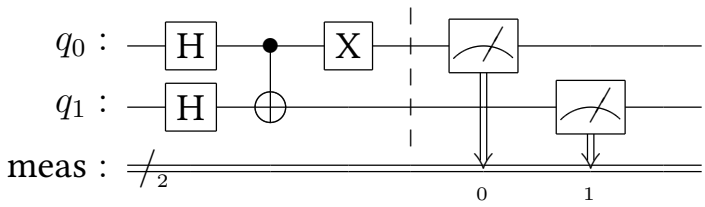


Figure 1.3: Example of a quantum circuit diagram.

1.3.1 Single-Qubit Gate Examples

Hadamard (H) Gate: The Hadamard gate is essential for generating superposition. It transforms the computational basis states $|0\rangle$ and $|1\rangle$ into equal linear combinations of both states. Geometrically, this corresponds to moving a state from the poles of the Bloch sphere to points on its equator, creating a balanced superposition. Its matrix representation is:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Pauli Gates (X, Y, and Z): The Pauli gates are fundamental single-qubit operations, each corresponding to a rotation of π radians (180 degrees) around the x, y, and z axes of the Bloch sphere, respectively.

- **Pauli-X:** Acts as a bit-flip by swapping $|0\rangle$ and $|1\rangle$.
- **Pauli-Z:** Performs a phase-flip, leaving $|0\rangle$ unchanged but mapping $|1\rangle$ to $-|1\rangle$.
- **Pauli-Y:** Combines both bit and phase flips.

Their matrix representations are:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

1.3.2 Multi-Qubit Gate Examples

Controlled-NOT (CNOT): The CNOT gate operates on two qubits: a control and a target. When the control qubit is in the state $|1\rangle$, the gate applies a bit-flip (Pauli-X) operation to the target qubit otherwise, the target qubit remains unchanged. This gate is fundamental for creating entanglement between qubits. Explicitly, it transforms the computational basis states as follows:

$$\text{CNOT}\,|00\rangle = |00\rangle\,, \quad \text{CNOT}\,|01\rangle = |01\rangle\,, \quad \text{CNOT}\,|10\rangle = |11\rangle\,, \quad \text{CNOT}\,|11\rangle = |10\rangle$$

Its matrix representation is:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

SWAP Gate: The SWAP gate exchanges the quantum states of two qubits. Although conceptually simple, it plays a crucial role in practical implementations by enabling qubit movement across hardware architectures with limited connectivity. Its matrix representation is:

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

1.3.3 Gate Construction and Universality

A key principle in quantum computing is universality: any quantum operation can be decomposed into a sequence of gates drawn from a finite universal set. In practice, combinations of single-qubit gates and a discrete, finite set of two-qubit gate types (such as CNOT) are sufficient to approximate any unitary transformation to arbitrary precision (Dawson and Nielsen 2006).



Figure 1.4: Equivalence between a SWAP gate and three CNOT gates.

The equivalence in Figure 1.4 decomposes a SWAP gate into three alternating CNOT gates. Mathematically, the SWAP matrix is the product of these CNOT matrices:

$$\text{SWAP} = \text{CNOT}_{0,1} \cdot \text{CNOT}_{1,0} \cdot \text{CNOT}_{0,1}$$

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

1.4 Physical Realizations of Qubits

While a qubit is mathematically defined in a Hilbert space, its physical implementation varies across different hardware technologies. Each modality offers different trade-offs:

- **Superconducting Qubits:** Leveraging Josephson junctions at cryogenic temperatures, these qubits (used by IBM and Google) offer fast gate execution but can only execute circuits with a low depth (Clarke and Wilhelm 2008).
- **Trapped Ions:** Qubits are encoded in the internal electronic states of individual ions confined by electromagnetic fields. They can maintain quantum states intact for a long time and have high, often all-to-all connectivity but feature slower gate speeds (Leibfried et al. 2003).

- **Photonic Qubits:** Information is encoded in the properties of photons (e.g., polarization). These systems can operate at room temperature and are ideal for quantum communication, though implementing multi-qubit gates is statistically challenging (O'brien, Furusawa, and Vučković 2009).

1.5 The Noisy Intermediate-Scale Quantum (NISQ) Era

The term NISQ, coined by Preskill (2018), characterizes the current state of quantum computing hardware. This era is defined by processors that have enough qubits to perform tasks beyond the reach of classical supercomputers but are significantly hampered by physical noise and a lack of error correction.

The "intermediate-scale" refers to the qubit count of modern processors, typically ranging from 50 to a few hundred qubits. While this scale is sufficient for exploring quantum advantage, it is far below the millions of qubits required for full-scale fault-tolerant computation. The "noisy" designation highlights that these qubits are not perfectly isolated, leading to a loss of quantum information.

1.6 Physical Constraints of NISQ Devices

1.6.1 Decoherence and the Coherence Bottleneck

One of the biggest challenges in quantum computing comes from **decoherence**: the tendency of qubits to lose their quantum behavior when they interact with their surroundings (Jazaeri et al. 2019). In practice, no qubit is perfectly isolated, so its state gradually degrades over time (Preskill 2018).

This effect is usually described using two timescales: **the relaxation time** T_1, which measures how quickly a qubit loses its energy, and **the dephasing time** T_2, which tells us how long it can maintain the delicate phase relationships that make superposition possible (Jazaeri et al. 2019). For today's superconducting qubits, these times are often only tens to hundreds of microseconds.

Since every quantum gate takes time to execute, this creates a practical limit: only a finite number of operations can be performed before the information becomes too noisy to be useful (Herbert 2020). This limitation is often referred to as the **coherence bottleneck**, and it directly restricts how deep or complex a quantum circuit can be in the **NISQ era** (Preskill 2018).

1.6.2 Error Rates and the Challenge of Correction

Even before decoherence fully takes over, quantum computations are already affected by errors. These can come from imperfect gate operations, noisy measurements (Nielsen and Chuang 2010), or small environmental disturbances. As a circuit runs, these errors build up and reduce the reliability of the final result.

To deal with this, researchers use **quantum error correction (QEC)**, which spreads the

information of a single logical qubit across many physical qubits (Nielsen and Chuang 2010). This makes it possible to detect and fix errors without directly measuring (and disturbing) the quantum state itself.

The downside is that this approach is extremely resource-intensive. It requires many extra qubits, frequent checks for errors, and very precise, fast control systems. Because of this overhead, building a fully fault-tolerant quantum computer remains one of the biggest open challenges in the field.

1.6.3 Hardware Connectivity Constraints

Another practical limitation comes from the way quantum hardware is built. In theory, any qubit could interact with any other, but in real devices, interactions are usually limited to nearby qubits. Most current architectures, especially superconducting ones, are arranged in grid-like layouts. If a computation requires two distant qubits to interact, they first need to be brought next to each other using a sequence of **SWAP operations** (Zulehner, Paler, and Wille 2018).

While this solves the connectivity problem, it comes at a cost: more gates, longer circuits, and therefore more exposure to noise and decoherence (Herbert 2020; Zulehner, Paler, and Wille 2018). In this way, hardware constraints indirectly make all the other challenges even harder.

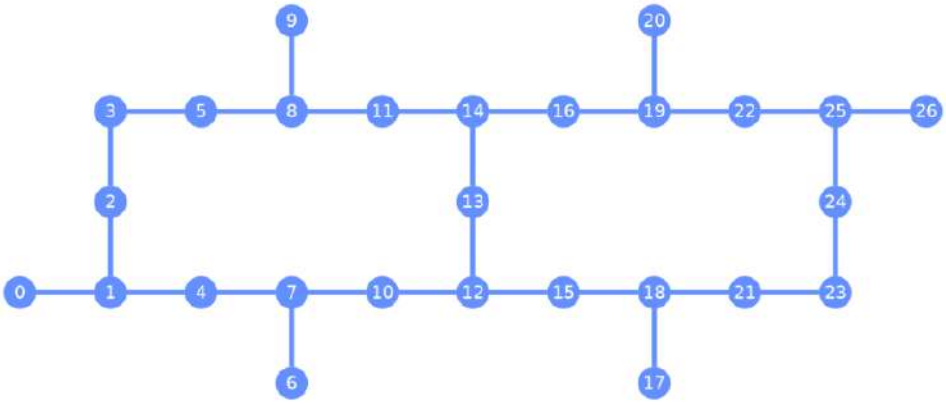


Figure 1.5: IBM Hanoi backend architecture, generated using qiskit (IBM 2024)

1.7 Quantum Compilation

Quantum circuit compilation is the process of transforming a high-level quantum algorithm into a sequence of hardware-level operations that can be executed on a specific quantum processor while considering hardware constraints (Siraichi et al. 2018). This process generally consists of several stages:

1. **High-Level Optimization** The compilation process begins with optimizations applied directly to the logical circuit, independent of the target hardware. Common techniques

include simplifying gate sequences using algebraic identities, canceling adjacent inverse gates (for example, $(H \cdot H = I)$), and merging or commuting compatible operations when possible (Amy et al. 2013). These optimizations aim to reduce circuit depth, execution time, and error accumulation.

2. **Gate Decomposition** Logical gates are then translated into the native gate set supported by the target quantum device (Barenco et al. 1995). Since many quantum devices can only execute a limited number of basic operations, more complex gates must be rewritten as combinations of simpler native gates. For example, a Hadamard gate may be implemented using a sequence of rotation gates available on the processor. Results such as the Solovay–Kitaev theorem guarantee that efficient approximations are possible (Dawson and Nielsen 2006).

3. **Initial Mapping (Qubit Allocation)** Once the circuit is expressed in the native gate set, logical qubits are assigned to physical qubits on the quantum processor. This step, known as qubit allocation or initial mapping, strongly influences the routing cost required later (Siraichi et al. 2018).

4. **Routing (SWAP Insertion)** Because most quantum devices support interactions only between neighboring qubits, the compiler must handle cases where two-qubit gates involve non-adjacent qubits. This is achieved by inserting SWAP gates that move quantum states across the hardware coupling graph until the required qubits become adjacent. Since SWAP operations significantly increase circuit depth and error probability, routing algorithms aim to minimize the number of inserted SWAPs (Herbert 2020).

5. **Scheduling** After routing, the resulting operations are scheduled over time. The scheduler determines when each gate should execute while respecting data dependencies and hardware constraints such as gate durations and limits on parallel operations. The objective is usually to minimize the overall execution time, or makespan, to reduce the effects of decoherence (Siraichi et al. 2018).

6. **Pulse Characterization (Lowering)** In the final stage of compilation, the scheduled quantum gates are translated into low-level control signals, or pulses, that directly manipulate the physical qubits of the quantum processor. This process, often called lowering, converts abstract gate operations into precise microwave or laser pulses whose amplitude, frequency, phase, and duration are carefully calibrated according to the hardware technology. At the very bottom of the stack, quantum gates aren't actually "gates" they are carefully calibrated microwave or laser pulses (Alexander et al. 2020).

In practice, modern quantum compilers do not always execute these stages independently. Instead, many compilation frameworks optimize several stages jointly and iteratively to improve overall circuit performance. Recent approaches also integrate noise-aware strategies by selecting physical qubits and connections with higher reliability and lower error rates, even if this slightly increases the routing cost (Murali et al. 2019).

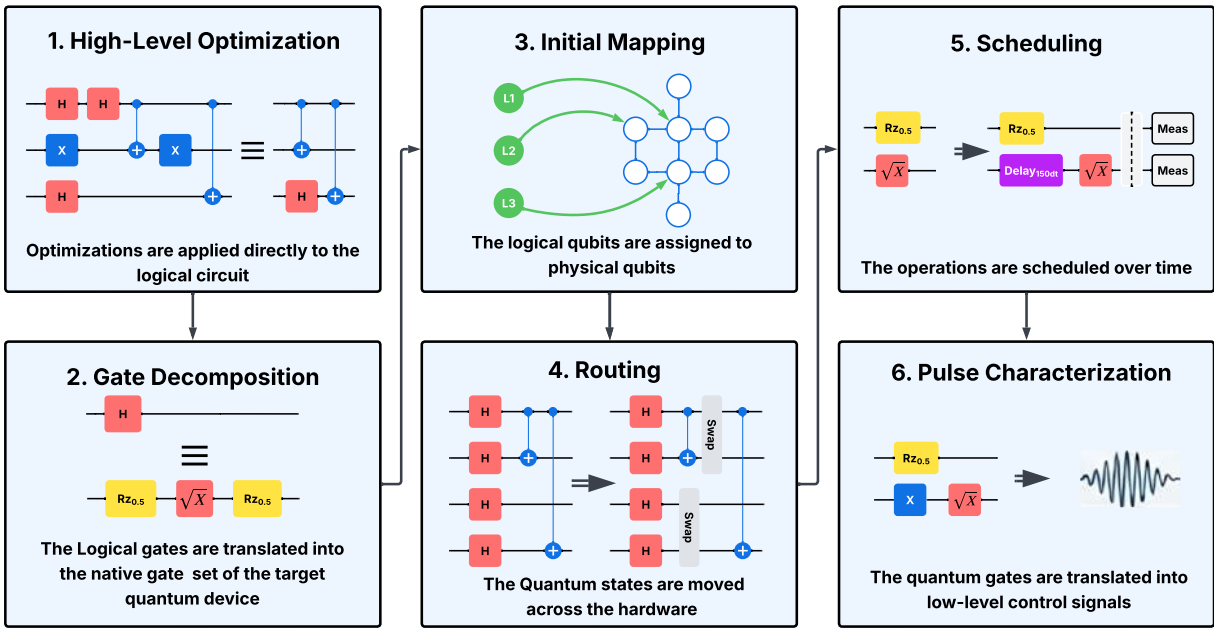


Figure 1.6: The Quantum Compilation Pipeline

1.8 Quantum Mapping

Among the compilation stages introduced in the previous section, qubit mapping is the step that directly confronts the hardware connectivity constraints. It is responsible for establishing a valid correspondence between the logical structure of the algorithm and the physical layout of the device (Siraichi et al. 2018). Because of the severe impact that additional routing overhead has on the fidelity of NISQ circuits, finding an optimal mapping is arguably the most critical and challenging phase of quantum compilation. The underlying optimization problem is known to be NP-hard (Siraichi et al. 2018), meaning that exact solutions are computationally intractable for all but the smallest circuits, necessitating the use of heuristic and approximation methods.

1.8.1 Complexity

The challenge of quantum mapping is not merely an engineering hurdle but a fundamental problem in computational complexity theory. Finding an optimal qubit mapping defined as the assignment and sequence of SWAP gates that results in the minimum circuit depth or gate count has been proven to be **NP-hard** (Siraichi et al. 2018).

The complexity arises from the massive search space of possible mappings and the temporal dependencies within the quantum circuit. For a processor with n physical qubits and a circuit with n logical qubits, there are $n!$ possible initial placements. As the circuit progresses, each layer of gates may require a new permutation of the mapping.

1.8.2 Formal Problem Statement

The physical layout of a quantum processor is captured by an undirected graph $G = (V, E)$, called the **coupling graph**, where each vertex $v \in V$ represents a physical qubit and each edge $(u, v) \in E$ indicates that a native two-qubit gate can be executed directly between qubits u and v (Siraichi et al. 2018). This graph is fixed by the hardware manufacturer an example is shown in Figure 1.5.

A logical quantum circuit C operates on a set of logical qubits $Q = \{q_1, q_2, \ldots, q_n\}$. The **quantum mapping problem** requires finding a bijective mapping function

$$\pi : Q \to V$$

that assigns each logical qubit to a distinct physical qubit, together with a sequence of SWAP operations inserted into C. This ensures that every two-qubit gate (q_i, q_j) in the circuit is executed only when $\pi(q_i)$ and $\pi(q_j)$ are adjacent in G (Zulehner, Paler, and Wille 2018).

The overall mapping task invariably decomposes into two coupled sub-problems:

- **Initial Mapping (Placement):** Choosing the initial bijection π before circuit execution begins. A high-quality initial placement anticipates the communication needs of the circuit, aiming to place frequently interacting qubits close to one another. This initial decision directly shapes the routing workload for the remainder of the circuit (Siraichi et al. 2018).
- **Routing (SWAP Insertion):** Dynamically updating π over time during execution by inserting SWAP gates whenever a required two-qubit gate acts on non-adjacent physical qubits. Each inserted SWAP, as shown in Figure 1.4, decomposes into three CNOT gates, compounding both gate count and circuit depth, and drastically accelerating decoherence and error accumulation (Zulehner, Paler, and Wille 2018).

The objective of an effective quantum mapping algorithm is therefore to minimize the total number of inserted SWAP gates (or equivalently, the increase in circuit depth) by intelligently co-optimizing both the initial placement and the dynamic routing strategy.

1.9 Fidelity and Metric Evaluation

The primary objective of optimizing quantum mapping and routing is to maximize the probability that the hardware produces the correct result. In the NISQ era, where gates are imperfect and qubits are short-lived, we utilize several metrics to evaluate the quality of a compiled circuit.

1.9.1 Gate and Circuit Fidelity

The performance of individual quantum operations is quantified by **fidelity** (F), a measure of how close a physical operation is to its ideal mathematical transformation. For a single gate i, the fidelity F_i is limited by control errors and decoherence. However, the total **Circuit**

Fidelity ($\mathcal{F}_{circ}$) is the most critical metric for end-users. It can be modeled as the cumulative product of the fidelities of all constituent operations:

$$\mathcal{F}_{circ} \approx \left(\prod_{i=1}^{N_1} F_{1,i}\right) \cdot \left(\prod_{j=1}^{N_2} F_{2,j}\right) \cdot \left(\prod_{k=1}^{N_m} F_{m,k}\right)$$

where N_1, N_2, and N_m represent the number of single-qubit gates, two-qubit gates, and measurements, respectively. Because two-qubit gate fidelities (F_2) are typically one to two orders of magnitude lower than single-qubit fidelities, the **SWAP overhead**, which introduces three CNOT gates per operation, is the dominant factor in fidelity degradation.

1.9.2 Quantum Volume (QV)

Introduced by IBM, **Quantum Volume** is a hardware-agnostic metric that quantifies the capability of a quantum computer (Cross et al. 2019). It ties together qubit count, gate fidelity, connectivity, and compiler efficiency. A high-performance routing algorithm can effectively increase the effective Quantum Volume of a device by allowing deeper circuits to run successfully on the same underlying hardware.

1.10 Conclusion

This chapter introduced the theoretical foundations of quantum computing, progressing from the fundamental concepts of quantum information to the formulation of quantum circuits. While quantum algorithms promise exponential speedups, executing them on contemporary NISQ hardware introduces severe physical constraints. Specifically, restricted qubit connectivity, high error rates, and rapid decoherence threaten the fidelity of any non-trivial computation.

To successfully bridge the gap between idealized algorithmic models and these restricted physical backends, a rigorous quantum compilation process is essential. As highlighted, qubit mapping and routing stand out as the most critical bottlenecks within this compilation stack, directly dictating the SWAP overhead and resulting circuit depth required to execute algorithms on constrained hardware topologies. Addressing this complex mapping problem is a primary focus of current quantum software research.

Chapter 2

Qubit Mapping Methods

2.1 Introduction

Recent advances in quantum hardware necessitate efficient quantum compilation to bridge abstract algorithm design and physical device constraints. A critical step in this process is qubit mapping (also known as allocation, layout, or routing), which assigns logical circuit qubits to specific physical hardware qubits (C. Zhu et al. 2025). Because physical quantum processors have limited connectivity, non-adjacent qubits cannot interact directly. Resolving this mismatch requires inserting additional operations, primarily SWAP gates to route quantum states across the device topology (Siraichi et al. 2018; Rattacaso et al. 2025).

Efficient mapping is especially vital in the NISQ era (Preskill 2018). Since NISQ devices suffer from limited resources and high susceptibility to noise (Ladd et al. 2010), the increased circuit depth and gate count introduced by SWAP operations severely degrade overall execution fidelity. Therefore, modern mapping strategies focus heavily on minimizing this overhead (Murali et al. 2019; Chi Zhang et al. 2020).

However, solving the qubit mapping problem is NP-Hard (Zulehner and Wille 2019), posing significant computational challenges as circuit sizes grow. Exact methods (e.g., integer linear programming and SAT solvers) guarantee optimal solutions but are restricted to small-scale instances due to their exponential cost. Consequently, scalable heuristic methods have become the standard in practice, offering high performance without guaranteed optimality. More recently, learning-based techniques have emerged as a promising alternative for learning automated mapping strategies, though generalizing these models to larger architectures remains an ongoing challenge.

This chapter provides a comprehensive survey of these qubit mapping methodologies. Section 2.2 introduces a taxonomy for these methods, establishing a clear classification framework for the approaches. Section 2.3 reviews the foundational exact methods, discussing their underlying principles and inherent limitations. Section 2.4 examines the emerging learning-based techniques, outlining their methodologies and current status. Section 2.5 delves into the widely used heuristic approaches, synthesizing representative techniques implemented in prominent quantum software tools and analyzing their performance characteristics. Finally, Section 2.6 presents a synthesis of all approaches, offering a comparative analysis, summarizing key insights, and discussing the overall state-of-the-art in quantum circuit mapping.

2.2 Taxonomy of Qubit Mapping Methods

To provide a structured overview of the diverse strategies developed to solve the qubit mapping problem, the existing literature can be organized into a clear taxonomy. As illustrated in Figure 2.1, these approaches are broadly classified into three primary categories based on their underlying paradigms:

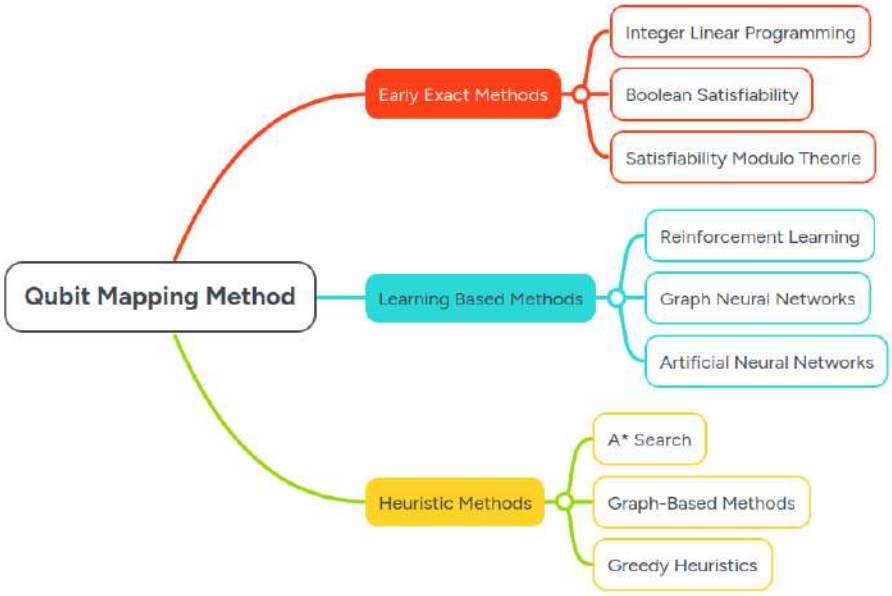


Figure 2.1: Taxonomy of qubit mapping methods.

- **Early Exact Methods:** This branch relies on rigorous mathematical and logical formulations, including Integer Linear Programming (ILP), Boolean Satisfiability (SAT), and Satisfiability Modulo Theories (SMT). These techniques systematically explore the solution space to guarantee absolute optimality, though they are heavily constrained by exponential runtime scaling.
- **Learning Based Methods:** Representing the latest shift towards artificial intelligence, this category includes Reinforcement Learning, Graph Neural Networks, and Artificial Neural Networks. These techniques focus on training models to autonomously discover or predict highly efficient mapping or routing policies, trading initial training overhead for rapid or adaptive execution.
- **Heuristic Methods:** The most widely utilized category in practical quantum compilation, this branch encompasses A* Search, Graph-Based Methods, and Greedy Heuristics. By relying on informed approximations and state-space pruning, these methods deliver scalable, ”good enough” solutions within polynomial time for complex NISQ-era circuits.

2.3 Early Exact Approaches

In the initial stages of quantum compiler development, exact methods were heavily used to guarantee optimal solutions for the qubit mapping problem. These methods translate mapping requirements into strict mathematical optimization problems, exhaustively exploring the search space to find the most efficient circuit configuration based on metrics like minimum depth or gate count. The literature primarily highlights three foundational families: Integer Linear Programming (ILP), Boolean Satisfiability (SAT), and Satisfiability Modulo Theories (SMT).

2.3.1 Integer Linear Programming (ILP)

ILP uses strictly linear objective functions and constraints alongside integer-restricted variables. In quantum mapping, it typically minimizes the overhead of SWAP gates across discrete time steps. Representative methods include the depth-optimal formulations for arbitrary topologies (Bhattacharjee and Chattopadhyay 2017) and the multi-constraint MUQUT framework (Bhattacharjee et al. 2019).

Mathematical Formulation
Let L represent the set of logical qubits and P denote physical qubits on the hardware topology $G = (P, E)$. The decision variable $x^t_{q,p} \in \{0, 1\}$ equals 1 if logical qubit $q \in L$ is mapped to physical qubit $p \in P$ at time t. Another variable $y^t_{p_1,p_2} \in \{0, 1\}$ indicates if a SWAP is inserted between adjacent physical qubits $(p_1, p_2) \in E$ at time t.

To ensure every logical qubit is uniquely assigned at any given time:

$$\sum_{p \in P} x^t_{q,p} = 1 \quad \forall q \in L, \forall t$$

The objective minimizes the total added SWAP operations over circuit duration T:

$$\min \sum_{t=1}^{T} \sum_{(p_1,p_2) \in E} y^t_{p_1,p_2}$$

Contributions, Strengths, and Limitations
Early ILP frameworks mathematically proved that arbitrary topologies could be modeled to guarantee depth-optimal compilation (Bhattacharjee and Chattopadhyay 2017), while later iterations like MUQUT concurrently optimized gate count and execution runtime (Bhattacharjee et al. 2019). ILP's greatest strength is its absolute mathematical guarantee of optimality. However, as an NP-hard problem, its execution time scales exponentially with qubit count and circuit depth, strictly limiting its use to small-scale circuits or heuristic benchmarking.

2.3.2 Boolean Satisfiability (SAT)

SAT solvers treat qubit mapping as an exact optimization problem, searching for a valid sequence of mappings that minimizes the total cost of additional SWAP and Hadamard (H) op-

erations. By passing an objective function to a reasoning engine (such as a SAT solver), the method dynamically determines the absolute minimum overhead required to comply with the hardware's coupling map. A primary representative of this exact methodology is the layout formulation for IBM QX architectures proposed by Wille et al. (Wille, Burgholzer, and Zulehner 2019).

Mathematical Formulation
Rather than using generalized time steps, the system state is defined relative to the sequence of logical CNOT gates $g_k \in G$ in the circuit. The exact formulation relies on boolean, permutation, and switching variables:

- $x_{ij}^k \in \{0, 1\}$: Equals 1 if logical qubit q_j is mapped to physical qubit p_i right before gate g_k.
- $y_\pi^k \in \{0, 1\}$: Represents the choice of a physical qubit permutation π (realized via SWAP operations) applied prior to g_k.
- $z^k \in \{0, 1\}$: Indicates whether the direction of the CNOT gate g_k must be reversed using H operations.

The solver identifies a valid mapping that respects the physical coupling map while minimizing the total cost function F. In the IBM QX architecture, a SWAP operation costs 7 elementary gates, and reversing a CNOT direction costs 4. Therefore, the objective function is:

$$\text{Minimize} \quad F = \sum_{k=2}^{|G|} \sum_{\pi \in \Pi} \left(7 \cdot \text{swaps}(\pi) \cdot y_\pi^k \right) + \sum_{k=1}^{|G|} \left(4 \cdot z^k \right)$$

This minimization is subject to constraints ensuring valid one-to-one logical-to-physical mappings (e.g., $\sum_i x_{ij}^k = 1$), valid transitions bounded by y_π^k, and strict coupling-map adjacency for all executed CNOT gates.

Contributions, Strengths, and Limitations
Wille et al. experimentally demonstrated through this exact formulation that default heuristics often inserted excess SWAP and Hadamard operations, exceeding optimal bounds by over 100% on average (Wille, Burgholzer, and Zulehner 2019). The exact solver effectively traverses massive search spaces to find the absolute minimum cost without falling into local optima. Unfortunately, the search space scales exponentially to $\mathcal{O}(2^{n \cdot m \cdot |G|})$ for n logical qubits, m physical qubits, and $|G|$ gates. This combinatorial explosion severely limits exact SAT approaches to very shallow quantum circuits with a small number of qubits.

2.3.3 Satisfiability Modulo Theories (SMT)

SMT extends SAT by incorporating mathematical theories for real numbers and continuous variables, enabling the solver to account for real-world hardware characteristics like error rates and coherence times. A foundational work is the noise-adaptive compiler mapping introduced by Murali et al. (Murali et al. 2019).

Mathematical Formulation
Let $\text{Fidelity}(g, \text{map}(g))$ represent the probability of successfully executing a gate g on its mapped physical qubits. The objective maximizes the product of success probabilities:

$$\max \prod_{g \in G} \text{Fidelity}(g, \text{map}(g))$$

This is typically linearized for the solver using logarithms:

$$\min \sum_{g \in G} -\log \Big(\text{Fidelity}(g, \text{map}(g)) \Big)$$

Contributions, Strengths, and Limitations
Murali et al. shifted the exact paradigm from pure gate-count minimization to noise-adaptive mapping (Murali et al. 2019). By ingesting daily hardware calibration data (gate errors, T_1/T_2 times), their SMT approach routed circuits across the most reliable chip areas, maximizing actual execution fidelity. While highly expressive, SMT suffers from the same exponential runtime constraints as ILP and SAT, and its success is highly vulnerable to calibration data drift between compilation and execution.

2.3.4 Comparative Analysis

While ILP, SAT, and SMT all guarantee a mathematically optimal qubit mapping, they exhibit distinct strategic differences in their formulations:

- **Optimization Objectives:** ILP and SAT focus on abstract topological metrics, such as minimizing circuit depth (Bhattacharjee and Chattopadhyay 2017) or gate count (Wille, Burgholzer, and Zulehner 2019). In contrast, SMT leverages continuous variables to optimize for physical execution fidelity based on real-world noise profiles (Murali et al. 2019).
- **State Space Modeling:** ILP generally tracks circuit progression across global, discrete time steps to ensure depth optimality. SAT, however, typically anchors its state space directly to the sequence of logical operations (e.g., CNOT gates) to dynamically determine exact permutation overheads.
- **Hardware Robustness:** Formulations based on static topology (ILP, SAT) are inherently robust to daily device fluctuations. SMT trades this stability for physical expressiveness, making its solutions highly susceptible to hardware calibration drift between compilation and execution.
- **Shared Scalability Bottleneck:** Regardless of their specific modeling strategies, all three paradigms are constrained by the NP-hard nature of the mapping problem. Their exponential runtime and severe combinatorial explosions strictly limit their applicability to small-scale benchmarks rather than broad NISQ-era compilation.

2.4 Learning Based Approaches

As quantum circuits grow in complexity, the limitations of exact methods (exponential runtimes) and heuristic algorithms (susceptibility to local optima) have become increasingly apparent. Artificial Intelligence (AI) techniques provide a powerful alternative, automatically learning efficient mapping and routing policies from data or environmental interaction. The literature primarily groups these into Reinforcement Learning (RL), Graph Neural Networks (GNN), and Artificial Neural Networks (ANN).

2.4.1 Reinforcement Learning (RL) and Deep RL (DRL)

RL frames qubit mapping and routing as a sequential decision-making problem. An agent interacts with the quantum hardware and circuit, receiving feedback (rewards) to learn a policy that maximizes a long-term objective. Representative frameworks have successfully applied this to autonomous SWAP insertion for minimizing gate count (Huang, Lien, and Mak 2022) and Deep RL strategies for noise-adaptive routing to maximize fidelity (Pascoal, Fernandes, and Abreu 2024).

Mathematical Formulation The mapping problem is modeled as a Markov Decision Process (MDP) tuple (S, A, P, R, γ). At time step t, state $s_t \in S$ represents the logical-to-physical mapping and unexecuted gates. Action $a_t \in A$ corresponds to inserting SWAPs or executing ready gates. The agent learns a policy $\pi(a|s)$ to maximize the expected cumulative discounted reward:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

where γ is the discount factor. Rewards $R(s_t, a_t)$ apply negative penalties for SWAP insertions or incorporate hardware fidelity metrics for noise-adaptive routing.

Contributions, Strengths, and Limitations RL methods significantly reduce reliance on human-engineered heuristics. The DEAR framework demonstrated autonomous discovery of efficient SWAP insertion strategies (Huang, Lien, and Mak 2022), while DRL approaches act as noise-aware compilers mapping around error-prone qubits (Pascoal, Fernandes, and Abreu 2024). The primary strength of RL is fast inference and high adaptability to complex constraints. However, training is computationally intensive, and RL agents often struggle to generalize to drastically larger circuits or different hardware topologies without retraining.

2.4.2 Graph Neural Networks (GNN) Aided Search

Because quantum circuits (Directed Acyclic Graphs) and hardware topologies (undirected coupling graphs) are inherently graph-structured, GNNs efficiently extract their topological features. When paired with algorithms like Monte Carlo Tree Search (MCTS), GNNs provide intelligent heuristics to guide the search process for highly optimized qubit routing (Sinha, Azad, and Singh 2022).

Mathematical Formulation A GNN maps nodes (qubits) to a continuous vector space, generating embeddings $h_v^{(l)}$ at layer l:

$$h_v^{(l)} = \text{UPDATE}\left(h_v^{(l-1)}, \text{AGGREGATE}\left(\{h_u^{(l-1)} : u \in \mathcal{N}(v)\}\right)\right)$$

These embeddings evaluate the state s quality to guide MCTS. During selection, the Upper Confidence Bound applied to Trees (UCT) determines the next action:

$$U(s,a) = Q(s,a) + c \cdot P(s,a)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

where $Q(s,a)$ is the GNN-derived action value, $P(s,a)$ the prior probability, N the visit count, and c the exploration constant.

Contributions, Strengths, and Limitations By leveraging GNNs to predict the most promising search branches, hybrid methods capture both spatial hardware constraints and temporal circuit dependencies, drastically narrowing the search space (Sinha, Azad, and Singh 2022). While this leads to highly optimized routing pathways and avoids local optima, the iterative nature of MCTS means compilation runtime remains slower than pure feed-forward AI models.

2.4.3 Artificial Neural Networks (ANN)

Supervised ML techniques bypass runtime algorithmic search by using pre-compiled datasets to train ANNs to directly predict optimal or near-optimal initial circuit layouts (Paler et al. 2023).

Mathematical Formulation Mapping is framed as a regression or classification problem. A feature vector $\mathbf{x} \in \mathbb{R}^d$ represents circuit and hardware properties. The ANN, parameterized by weights θ, outputs a predicted layout $y = f_\theta(\mathbf{x})$. Training minimizes a loss function, typically Mean Squared Error (MSE), over N optimal examples:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(f_\theta(\mathbf{x}_i) - y_i\right)^2$$

Contributions, Strengths, and Limitations ANN models front-load computational effort to the training phase, rapidly outputting high-quality placements in milliseconds and drastically reducing compilation bottlenecks (Paler et al. 2023). This ultra-fast compilation speed is its greatest asset. However, generating optimal training datasets is NP-hard, and the network is strictly limited by data quality, often exhibiting poor out-of-distribution generalization for larger, unseen circuits.

2.4.4 Comparative Analysis

While RL, GNN-aided search, and supervised ANNs all leverage machine learning to overcome the scalability bottlenecks of exact methods, a comparative analysis reveals distinct trade-offs in how they acquire knowledge and execute mapping:

- **Learning Paradigms and Data Dependency:** The methods diverge fundamentally in how they learn. Supervised ANNs are entirely bottlenecked by offline data generation they require pre-compiled, NP-hard optimal datasets to learn layout predictions (Paler et al. 2023). RL bypasses this data dependency by learning autonomously through active interaction and reward feedback (e.g., heavily penalizing SWAP overhead) (Huang, Lien, and Mak 2022). GNNs, meanwhile, focus explicitly on structural feature extraction, learning the continuous vector representations of the hardware and circuit topologies to guide algorithmic search (Sinha, Azad, and Singh 2022).
- **Compilation Speed vs. Optimization (The Inference Trade-off):** At compilation time, supervised ANNs offer the ultimate speed, utilizing single feed-forward passes to output layouts in milliseconds. GNN-aided MCTS sits at the opposite end of the spectrum while it achieves highly optimized routing pathways by avoiding local optima, its reliance on iterative tree search makes compilation significantly slower. RL models generally sit between the two, offering fast heuristic inference once an optimal policy is trained.
- **Generalization and Hardware Adaptability:** Both RL and ANN models notoriously suffer from out-of-distribution brittleness. An RL policy or ANN trained on a specific hardware topology or circuit depth usually fails on drastically different architectures without computationally expensive retraining. In contrast, GNN-aided MCTS is inherently more adaptable to unseen structural variations, as the neural network guides a dynamic, real-time search rather than attempting to output a rigid, final mapping policy.
- **Handling of Hardware Noise:** While ANNs and GNNs typically optimize for structural metrics (like depth or gate count), Deep RL natively accommodates real-time hardware characteristics. By integrating calibration data into its reward function, DRL can dynamically adapt its routing policy to avoid noisy qubits, offering a level of physical adaptiveness that purely structural ML models lack (Pascoal, Fernandes, and Abreu 2024).

2.5 Heuristic Approaches

While exact methods guarantee optimality and learning-based models offer autonomous adaptation, heuristic algorithms remain the most practical and widely adopted approach for quantum circuit mapping. By relying on greedy choices, graph theory, or guided state-space searches, heuristics scale efficiently to NISQ-era devices with dozens to hundreds of qubits. Rather than guaranteeing a mathematically perfect layout, they aim to find ”good enough” mappings in polynomial time. The literature typically divides these into A* search-based methods, graph-based methods, and greedy layer-by-layer methods.

2.5.1 A* Search-Based Methods

Search-based heuristics attempt to navigate the combinatorial explosion of the mapping problem by intelligently exploring a state-space tree. The A* search algorithm is heavily utilized to

guarantee optimal shortest paths for sub-circuits, mitigating the overhead of full integer linear programming. A prominent example of this approach is the MQT QMAP framework (Wille and Burgholzer 2023).

Mathematical Formulation
The mapping process is modeled as a state-space search where each state s represents a specific logical-to-physical qubit mapping. The algorithm evaluates the cost of moving between states using the standard A* function:

$$f(s) = g(s) + h(s)$$

Here, $g(s)$ is the exact cost incurred so far (typically the number of SWAP gates already inserted to reach state s), and $h(s)$ is an admissible heuristic function estimating the remaining cost to execute the next layer of gates. The heuristic $h(s)$ is usually derived from the shortest path distance $D(p_i, p_j)$ between the physical qubits p_i and p_j on the hardware topology.

Contributions, Strengths, and Limitations
MQT QMAP demonstrated that using an admissible distance-based heuristic allows the search space to be pruned efficiently without sacrificing optimal depth for local sub-circuits (Wille and Burgholzer 2023). The major strength of A* methods is their ability to bridge the gap between exact optimal routing and heuristic speed. However, they remain bound by memory constraints. For very large circuits or highly constrained topologies, the number of explored states can still explode exponentially, leading to memory-outs.

2.5.2 Graph-Based Heuristics

Graph-based methods approach the mapping and routing task by evaluating both the hardware architecture and the quantum circuit as mathematical graphs. The t|ket⟩ compiler is a leading framework employing this strategy, adapting structural analysis with temporal heuristics to abstract hardware constraints (Sivarajah et al. 2020).

Mathematical Formulation
The quantum hardware is represented as an architecture graph $AG = (P, E_P)$, and the logical circuit interactions as an interaction graph $IG = (L, E_L)$. While the ideal initial placement can be framed globally as a subgraph isomorphism search to maximize edge overlap, the t|ket⟩ compiler relies on a temporal heuristic. The circuit is sliced into sequential timesteps, and early timesteps are used to construct a simplified interaction graph where every logical qubit has a maximum degree of two, naturally forming line or ring topologies. This simplified graph is then mapped onto a dense subgraph of the AG chosen for its high average degree and low diameter.

For the subsequent routing phase, the task is formally equivalent to a token-swapping problem on the AG. The algorithm computes a distance vector to approximate the number of SWAP gates required, sequentially evaluating and selecting SWAP operations that minimize the distance between interacting qubits across current and upcoming timesteps.

Contributions, Strengths, and Limitations
The t|ket⟩ framework pioneered highly retargetable compilation, demonstrating that graph heuristics can produce optimized mapping for diverse hardware topologies (Sivarajah et al. 2020). A core strength of this approach is its ability to capture temporal dynamics rather than relying on a static interaction graph, it evaluates candidate mappings across multiple sequential timesteps (lookahead) to prioritize immediate dependencies. While highly scalable and effective in practice, the primary limitation of such approaches is their heuristic nature. Because finding an optimal routing configuration is theoretically NP-hard, the sequential approximations can sometimes yield suboptimal SWAP overheads compared to exhaustive search methods, though at a significantly reduced computational cost.

2.5.3 Greedy Heuristics

Greedy heuristics represent the most widely adopted paradigm in modern quantum compilers due to their exceptional scalability and speed. Rather than attempting to find a globally optimal layout, which requires exponential runtime, these methods iteratively resolve mapping conflicts by making locally optimal decisions. By moving sequentially through the circuit, greedy algorithms insert SWAP gates to satisfy the connectivity requirements of the immediate upcoming operations. Because of their polynomial time complexity, greedy heuristics are the default choice for compiling large-scale quantum circuits in the NISQ era. Representative frameworks include the highly influential SABRE algorithm, its optimized variant LightSABRE, and the more recent dependence-driven Qlosure framework.

General Algorithm for Greedy Heuristics
While specific implementations vary, the fundamental control flow of a greedy heuristic mapper follows a standard layer-by-layer resolution process, as outlined in Algorithm 1. The procedure can be summarized in the following key steps:

1. **Initialization:** Begin with an initial logical-to-physical qubit mapping and extract the first set of executable gates from the circuit (the front layer).
2. **Execution Check:** Evaluate whether the logical qubits involved in the front layer's two-qubit gates are mapped to adjacent physical qubits on the hardware topology.
3. **Gate Scheduling:** If the connectivity constraints are satisfied, schedule the gate for execution, remove it from the dependency graph, and update the set of executable gates by pulling the next available operations.
4. **SWAP Candidate Generation:** If a gate cannot be executed due to a lack of connectivity, the algorithm generates a list of candidate SWAP operations (usually involving the physical qubits mapped to the blocked logical qubits and their immediate neighbors).
5. **Heuristic Scoring:** Evaluate each candidate SWAP using a predefined cost function to estimate which move best resolves the current routing bottleneck.
6. **Mapping Update:** Select the SWAP with the lowest heuristic cost, update the logical-to-physical mapping, and return to step 2. This loop continues until the entire circuit is mapped.

Algorithm 1 General Greedy Heuristic for Qubit Mapping

Input: Quantum circuit C, Hardware graph G
Output: Mapped circuit C_{mapped}
Initialize logical-to-physical mapping φ and front layer F
while F is not empty **do**
    $G_{exec} \leftarrow$ gates in F satisfying connectivity under φ ▷ Execution Check
    if $G_{exec} \neq \emptyset$ **then** ▷ Gate Scheduling
        Schedule G_{exec}, add to C_{mapped}, and update F from C
    else ▷ SWAP Generation & Scoring
        $S_{cand} \leftarrow$ Generate candidate SWAPs for blocked gates in F
        $best_swap \leftarrow \arg\min_{s \in S_{cand}} \text{Cost}(s)$ ▷ Evaluate heuristic
        Add $best_swap$ to C_{mapped} and update mapping φ ▷ Mapping Update
    end if
end while
return C_{mapped}

The SABRE Family: SABRE and LightSABRE

The SABRE (SWAP-based BidiRectional heuristic search) algorithm (Gushu, Ding, and Xie 2019) and its enhanced derivative, LightSABRE (Zou et al. 2024), are cornerstones of heuristic quantum mapping. These methods tackle the qubit mapping problem by relying on a dynamic lookahead window and a bidirectional traversal technique to optimize both the initial layout and the subsequent routing phase.

Heuristic SWAP Selection

When gates in the front layer F cannot be executed due to connectivity issues, candidate SWAPs are considered. Each candidate is scored using a heuristic cost function H:

$$H = \frac{1}{|F|} \sum_{(i,j) \in F} \text{dist}(\varphi(i), \varphi(j)) + \frac{k}{|E|} \sum_{(i,j) \in E} \text{dist}(\varphi(i), \varphi(j))$$

where:

- φ is the current logical-to-physical qubit mapping.
- F is the front layer, and E is the extended set of upcoming gates.
- $\text{dist}(p, q)$ is the shortest path between physical qubits p and q.
- k is a tunable weight for the lookahead component.

The SWAP with the minimum H value is selected.

Contributions, Strengths, and Limitations

The SABRE family's primary contribution is a fast, bidirectional compilation pass that iteratively refines initial mappings using a lookahead window, offering the core strength of highly scalable, polynomial-time efficiency that successfully prevents trivial routing bottlenecks. However, its main limitation is its shallow scope because the lookahead only captures immediate dependency neighbors rather than the global circuit structure, SABRE is prone to falling into local optima, potentially generating excessive SWAP counts and deeper delays when processing highly entangled algorithms.

Qlosure: Dependence-Driven Mapping

To overcome the short-sighted nature of traditional geometric heuristics, the Qlosure algorithm introduces a mapping strategy grounded in affine abstractions and the polyhedral model.

The polyhedral model (Paul Feautrier and Lengauer 2011) is a mathematical model for representing code and code optimizations and is used in state-of-the-art compilers to apply complex code transformations and reason about their correctness (Irigoin and Triolet 1988; P. Feautrier 1988; Wolf and Lam 1991; Lefebvre and Paul Feautrier 1998; Bondhugula et al. 2008; Baghdadi 2015; Baghdadi et al. 2018; Grosser, Groslinger, and Lengauer 2012; Grosser et al. 2014; Vasilache et al. 2018; Baghdadi et al. 2011; Pouchet et al. 2011; Baghdadi et al. 2013; Merouani et al. 2024; Baghdadi et al. 2015; Lamouri et al. 2025).

Designed to capture a global view of the circuit's evolution, Qlosure fundamentally shifts the heuristic focus from immediate gate proximity to transitive dependence resolution (Benbetka et al. 2026).

Heuristic Cost Function and Affine Abstractions

Unlike SABRE, which relies on a simple, flat lookahead window, Qlosure takes advantage of transitive dependence information to inform its SWAP insertions. By computing the transitive closure of the dependence graph, each gate is annotated with a measure of its downstream impact, providing a proxy for its criticality (Benbetka et al. 2026).

The heuristic cost function $M(s)$ evaluates each candidate SWAP s using a composite score (Benbetka et al. 2026):

$$M(s) = \max\big(\delta_{q_1}, \delta_{q_2}\big) \times \left(\sum_{G_\ell \in \mathcal{G}} \frac{\Gamma_\ell}{|G_\ell|} \right)$$

$$\text{where } \Gamma_\ell = \sum_{g \in G_\ell} \frac{\omega_g \cdot D_{\text{phys}}\big[\varphi_s(g_{q_a}), \varphi_s(g_{q_b})\big]}{\ell}$$

This function balances multiple structural constraints:

- **Decay Factor (δ):** Penalizes SWAPs that involve recently moved logical qubits, preventing the algorithm from undoing recent moves and encouraging the exploration of fresher qubits.

- **Physical Distance (D_{phys}):** Prioritizes SWAPs that reduce the physical separation of upcoming gate operands on the QPU.
- **Dependence Weight (ω_g):** Assigns a lower cost to gates with fewer future-dependent operations, favoring the execution of gates that safely free up qubits without heavily impacting the remaining circuit.
- **Layer Discount ($1/\ell$) & Normalization ($|G_\ell|$):** Gates are weighted by their topological depth ℓ to prioritize immediate operations, while normalization prevents wider circuit layers from dominating the heuristic score.

Contributions, Strengths, and Limitations
Qlosure contributes a dependence-driven approach utilizing polyhedral representations to precisely enumerate transitive dependencies (Benbetka et al. 2026), with its primary strength being a far-sighted heuristic that significantly minimizes both circuit depth and total SWAP operations by evaluating the downstream impacts of routing decisions. The main limitation of this approach is the substantial computational overhead incurred during the precomputation phase, such as extracting affine representations and calculating all-pairs shortest paths $O(N^3)$, which demands considerably more upfront processing time compared to the lightweight distance calculations of localized greedy methods.

2.5.4 Comparative Analysis

Heuristic methods provide a spectrum of trade-offs between mapping quality and execution speed, distinguished by their search depth and dependency modeling:

- **Search Depth vs. Scalability:** A* search (Wille and Burgholzer 2023) guarantees optimal routing for local sub-circuits but is strictly memory-bound. Conversely, greedy methods (SABRE) sacrifice global optimality for polynomial-time scalability, making them the industry standard for large-scale NISQ devices where memory-heavy search trees fail.
- **Temporal Lookahead Scopes:** The methods diverge in how they "see" the circuit's future. Standard greedy mappers (SABRE) utilize a flat, immediate lookahead window. In contrast, Qlosure leverages transitive dependencies and polyhedral abstractions to assess long-term downstream impacts (Benbetka et al. 2026), significantly reducing total SWAP overhead by identifying critical bottlenecks before they occur.
- **Structural vs. Metric-Driven Routing:** Graph-based heuristics (t|ket⟩ prioritize the structural alignment of hardware and circuit topologies through temporal slicing and subgraph analysis (Sivarajah et al. 2020). Metric-driven methods (SABRE, LightSABRE) focus instead on minimizing instantaneous distance vectors while faster, they are more susceptible to local optima in highly constrained topologies.
- **Computational Precomputation:** There is a direct trade-off between initialization latency and routing efficiency. Lightweight distance-based heuristics allow for rapid,

iterative compilation passes. Dependency-driven methods (Qlosure) require substantial O(N^3) precomputation for transitive closure, trading upfront processing time for a reduction in final circuit depth and gate count.

2.6 Synthesis of Qubit Mapping Methods

Table 2.1 provides a high-level comparison of the specific methods discussed in this chapter, highlighting their key features, strengths, and inherent limitations.

Table 2.1: Synthesis and Comparison of Qubit Mapping Methods.

Method	Key Features	Strengths	Limitations
		Early Exact Approaches	
ILP (Bhattacharjee and Chattopadhyay 2017)	Linear constraints for min SWAP/depth.	Mathematical guarantee of optimal depth/gates.	Exponential runtime and strictly for tiny circuits.
SAT (Wille, Burgholzer, and Zulehner 2019)	Exact physical SWAP/H overhead tied to logical ops.	Absolute minimum routing overhead, no local optima.	Extreme combinatorial explosion $\mathcal{O}(2^{n \cdot m \cdot \lvert G \rvert})$.
SMT (Murali et al. 2019)	Continuous variables and daily calibration data for fidelity.	Noise-adaptive, natively routes around hardware errors.	Exponential runtime and vulnerable to calibration drift.
		Learning Based Approaches	
RL (Huang, Lien, and Mak 2022)	RL agent learns policies for SWAP reduction/noise avoidance.	Fast inference and handles real-time metrics.	Expensive training and poor generalization to new topologies.
GNN (Sinha, Azad, and Singh 2022)	Graph structural features guide Monte Carlo Tree Search.	Captures complex dependencies	Iterative tree search causes slow compilation runtime.
ANN (Paler et al. 2023)	Predicts mapping via regression on optimal pre-compiled data.	Fast compilation (millisecond mapping generation).	Needs NP-hard offline data, poor out-of-distribution scope.
		Heuristic Approaches	
A* Search (Wille and Burgholzer 2023)	Prunes state-space trees via physical distance heuristics.	Optimal for local sub-circuits and faster than exact methods.	Memory-bound and fails on highly constrained topologies.
t|ket⟩ (Sivarajah et al. 2020)	Aligns logic/hardware graphs via temporal slicing.	Captures immediate temporal dynamics.	Sequential approximations yield suboptimal SWAPs.
SABRE (Gushu, Ding, and Xie 2019)	Layer-by-layer bidirectional passes with flat lookahead.	High polynomial scalability, industry standard for NISQ.	Prone to local optima and excessive SWAP counts.
Qlosure (Benbetka et al. 2026)	Polyhedral models/transitive closure for global SWAP impact.	Reduces depth/gates significantly by avoiding local optima.	High $\mathcal{O}(N^3)$ initialization/precomputation latency.

While Table 2.1 shows the details of each technique, comparing the three main approaches, exact, learning-based, and heuristic, reveals clear differences in how they are used. The fundamental trade-offs among optimality, scalability, and compilation runtime govern the selection of these approaches for modern quantum compilation.

Early exact approaches are distinguished by their ability to provide formal mathematical optimality guarantees for feasible instances, ensuring the absolute minimum mapping overhead or maximum execution fidelity. However, this certainty comes at a severe computational

cost. Exact methods are strictly bounded by exponential scaling limits, typically restricting their use to very small instances of up to 8 to 12 logical qubits and shallow circuits containing only tens to hundreds of gates. Furthermore, they are only viable for small-scale quantum processing unit (QPU) topologies, generally capped at around 12 physical qubits. Even for these minimal-scale circuits, compilation times can range from hours to days, rendering exact methods entirely impractical for routine compilation, though they remain a vital tool for offline benchmarking and establishing theoretical lower bounds.

Learning-based approaches represent a modern shift, leveraging machine learning to discover mapping policies directly from training data or environmental interaction. Rather than offering formal optimality guarantees, they aim to learn highly efficient mapping strategies. These models scale considerably better than exact methods, comfortably handling intermediate parameters: circuits with 20 to 50 qubits and small to medium operation scales ranging from hundreds to thousands of gates. The primary limitation of learning methods lies in their training bottleneck and topological rigidity. They require a substantial upfront computational investment, often taking hours or days to train, even though inference (compilation) can subsequently be executed in seconds to minutes. Moreover, they suffer from limited generalization, often failing to adapt to unseen QPU hardware layouts or significantly larger circuit structures without requiring complete retraining.

Heuristic methods abandon formal optimality guarantees entirely in favor of achieving robust, "good enough" approximations within polynomial time. Consequently, heuristics represent the most practical and widely adopted solution, striking a balance between scalability and runtime efficiency. Because they avoid exponential state-space explosions, they possess unmatched scalability, comfortably handling massive circuits containing up to 50 to 200 qubits and routing hundreds of thousands of gates.

Most importantly, heuristics support modern NISQ hardware characterized by complex topologies and hundreds of qubits. By delivering compilation times that range from mere milliseconds to minutes, they completely bypass the crippling bottlenecks of exact search trees and AI training phases. While their primary limitation is an occasional susceptibility to local optima, their polynomial time efficiency makes them indispensable. Due to this optimal balance, heuristic algorithms are the undisputed industry standard and are natively integrated into major quantum compilation toolchains such as Qiskit, `t|ket`, and MQT QMAP.

2.7 Conclusion

This chapter surveyed exact, learning-based, and heuristic methods for qubit mapping. While classical heuristics remain the industry standard for modern NISQ devices due to their polynomial-time scalability, hand-crafting these rules for increasingly complex quantum hardware has become a severe, manual bottleneck.

Recently, Large Language Models for Automated Heuristic Design (LLM-AHD) have emerged as a powerful paradigm to autonomously generate and refine algorithmic logic. Notably, despite its success in other combinatorial optimization domains, applying LLM-AHD to quantum compilation, specifically qubit mapping, remains largely unexplored in the current literature.

Bridging this gap represents a highly promising research direction. By leveraging LLMs to automatically discover and optimize hardware-aware routing policies, we can eliminate the manual design bottleneck and develop agile quantum compilers capable of seamlessly adapting to future, complex hardware topologies.

Chapter 3

Large Language Models

3.1 Introduction

Large Language Models (LLMs) have emerged as one of the most significant advances in artificial intelligence, fundamentally transforming natural language processing and enabling increasingly capable intelligent systems. Built upon large-scale transformer architectures and trained on massive corpora, LLMs demonstrate strong capabilities in language understanding, reasoning, planning, code generation, and knowledge synthesis (Vaswani et al. 2017; Brown et al. 2020; Touvron et al. 2023; Achiam et al. 2023).

Unlike earlier task-specific neural models, modern LLMs are considered *foundation models* because they can generalize across a broad range of downstream tasks through prompting, instruction tuning, and adaptation (Bommasani et al. 2021). Their ability to perform zero-shot and few-shot learning has significantly reduced the dependence on task-specific supervised training pipelines (Brown et al. 2020). Consequently, LLMs now serve as the computational backbone of many contemporary AI systems, including conversational assistants, code-generation tools, autonomous agents, and scientific discovery frameworks.

Recent progress in LLM research has also accelerated the development of agentic systems capable of reasoning, planning, memory utilization, and tool interaction (Yao et al. 2023b; Park et al. 2023; Luo et al. 2025). In these systems, the LLM acts as a general-purpose cognitive engine that orchestrates interactions with external tools, environments, and other agents. As a result, understanding the foundations, capabilities, and limitations of LLMs is essential for studying modern agentic architectures and LLM-driven optimization systems.

This chapter presents an overview of the core principles underlying LLMs, including transformer architectures, training paradigms, emergent capabilities, and current limitations relevant to intelligent agent systems.

3.2 Large Language Models

3.2.1 From NLP Models to Foundation Models

Traditional Natural Language Processing (NLP) systems relied heavily on manually engineered features and task-specific pipelines (Jurafsky and Martin 2009). Early machine learning approaches, including statistical language models and recurrent neural networks, achieved success in limited domains but struggled to generalize across heterogeneous tasks and long contextual dependencies (Mikolov et al. 2010).

The introduction of **the Transformer architecture** marked a major turning point in NLP research (Vaswani et al. 2017). This architectural shift facilitated large-scale pretraining on internet-scale corpora, leading to increasingly capable language models such as GPT, BERT, PaLM, and LLaMA (Devlin et al. 2019; Brown et al. 2020; Chowdhery et al. 2023; Touvron et al. 2023).

As model scale increased, researchers observed the emergence of generalized capabilities that transferred across multiple tasks without explicit retraining (Kaplan et al. 2020; Wei et al. 2022a). This led to the concept of **foundation models**, which are large pretrained models adaptable to diverse downstream applications (Bommasani et al. 2021). Foundation models are typically trained using self-supervised learning objectives on massive multimodal or textual datasets and later adapted through prompting or fine-tuning.

Scaling laws further demonstrated that increasing model parameters, training data, and computational resources systematically improve model performance (Kaplan et al. 2020; Hoffmann et al. 2022). These findings motivated the development of increasingly large LLMs capable of performing complex reasoning, code synthesis, and natural language interaction with minimal task-specific supervision.

3.2.2 Transformer Architecture

Modern LLMs are predominantly based on the Transformer architecture (Vaswani et al. 2017). The transformer replaces recurrent processing with **self-attention mechanisms** that enable efficient modeling of contextual dependencies across input sequences.

Tokenization and Embeddings Before processing, textual inputs are converted into discrete **tokens** using tokenization algorithms such as Byte Pair Encoding (BPE) or SentencePiece (Sennrich, Haddow, and Birch 2016; Kudo and Richardson 2018). Each token is mapped into a continuous vector representation known as an **embedding**. Positional embeddings are then added to preserve sequential ordering information, which is otherwise absent in attention-based architectures.

Self-Attention Mechanism The self-attention mechanism allows each token to dynamically attend to other tokens within the sequence. Given query (Q), key (K), and value (V) matrices, attention weights are computed using **Scaled Dot-Product Attention** (Figure 3.2) as follows:

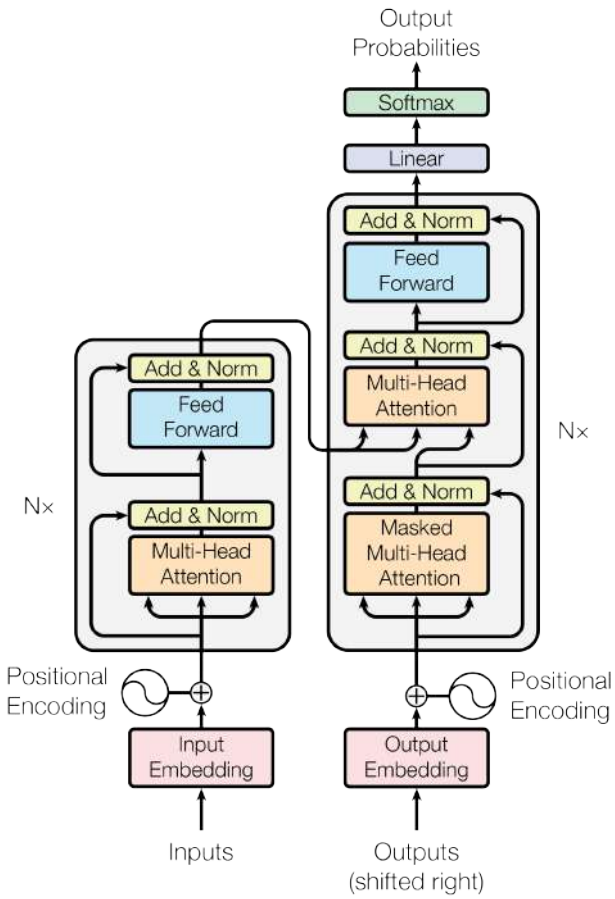


Figure 3.1: The Transformer Architecture (Vaswani et al. 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k denotes the dimensionality of the key vectors (Vaswani et al. 2017). **Multi-head attention** (Figure 3.3) extends this mechanism by enabling the model to learn multiple contextual relationships simultaneously.

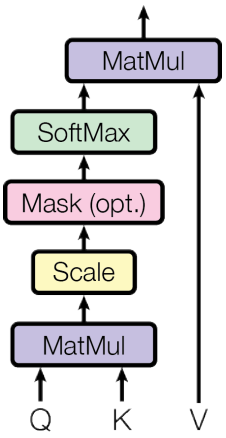


Figure 3.2: Scaled Dot-Product Attention (Vaswani et al. 2017)

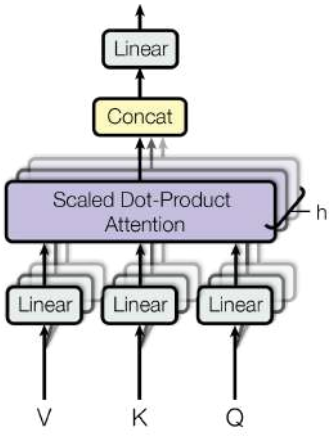


Figure 3.3: Multi-Head Attention (Vaswani et al. 2017)

Self-attention provides several advantages over recurrent architectures, including improved parallelization, better long-range dependency modeling, and greater scalability.

Autoregressive Language Modeling Most generative LLMs employ autoregressive training objectives in which the model predicts the next token conditioned on preceding tokens (Brown et al. 2020). Formally, given a token sequence $(x_1, \ldots, x_n)$, the objective is to maximize:

$$P(x) = \prod_{t=1}^{n} P(x_t | x_{<t})$$

This next-token prediction paradigm enables LLMs to learn syntactic, semantic, and contextual patterns directly from large corpora without explicit supervision.

3.2.3 Training Paradigms of Large Language Models

The capabilities of modern Large Language Models (LLMs) emerge through several successive training stages designed to improve linguistic competence, task generalization, and alignment with human preferences.

First, **pretraining** exposes the model to massive text corpora via self-supervised objectives such as next-token prediction, allowing it to acquire broad linguistic knowledge, factual information, and general reasoning patterns (Brown et al. 2020; Bommasani et al. 2021). Second, **instruction tuning** fine-tunes the pretrained model on structured instruction-response datasets, improving its ability to follow user directives and generalize across tasks (Ouyang et al. 2022; Chung et al. 2024). Third, **alignment optimization** further refines model behavior using Reinforcement Learning from Human Feedback (RLHF) or its variants such as Direct Preference Optimization (DPO), steering outputs to be helpful, accurate, and consistent with human preferences (Christiano et al. 2017; Rafailov et al. 2023). Together, these stages transform a raw statistical model into a capable, instruction-following, and preference-aligned system.

3.2.4 Emergent Capabilities of Large Language Models

As LLMs scale in size and training complexity, they exhibit emergent behaviors that are not explicitly programmed during training (Wei et al. 2022a). These capabilities are particularly important for agentic systems and autonomous reasoning frameworks.

Reasoning and Planning Modern LLMs demonstrate increasingly sophisticated reasoning abilities, particularly when guided through prompting strategies (Wei et al. 2022b; Yao et al. 2023a; Yao et al. 2023b).

In-Context Learning LLMs can perform new tasks by conditioning on demonstrations provided directly within the prompt (Brown et al. 2020). Rather than updating parameters, the model adapts behavior dynamically based on contextual examples.

Code Generation and Tool Use LLMs also exhibit strong code synthesis and tool utilization capabilities (M. Chen et al. 2021). Models can generate executable programs, interact with APIs, retrieve information from external sources, and orchestrate computational workflows. Tool-augmented reasoning frameworks substantially extend the operational capabilities of LLM-based systems (Schick et al. 2023).

3.3 Intelligent Agents

3.3.1 Fundamentals of Agency

The concept of an intelligent agent is a foundational paradigm in artificial intelligence, defining a system that is not merely a tool, but an autonomous entity. An agent is defined as anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators** (Russell and Norvig 2010).

This relationship is characterized by a continuous **perception-action loop** (Figure 3.4). At each discrete or continuous time step, the agent receives an observation, updates its internal state, and selects an action according to its **agent function** (the mapping from percept sequences to actions). The primary objective is to maximize a performance measure, often formalized through utility functions or goal-state attainment (Sutton and Barto 1998).

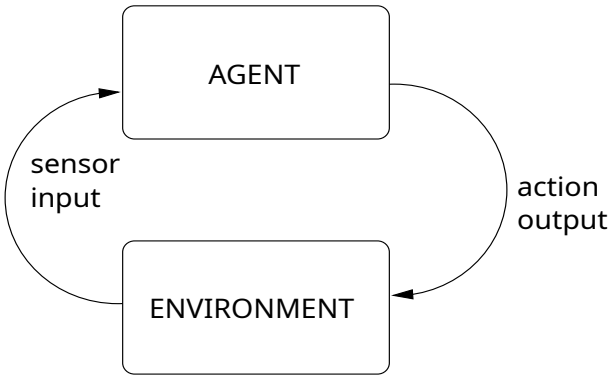


Figure 3.4: The perception-action loop in intelligent agents (Wooldridge 1999)

3.3.2 The Archetypal Properties of Intelligent Agents

According to the classical Wooldridge-Jennings definition, an intelligent agent must exhibit four key properties (Wooldridge and Jennings 1995):

- **Autonomy:** The ability to operate without direct human intervention and have control over its internal state and behavior.
- **Reactivity (Perception):** The ability to perceive the environment and respond in a timely fashion to changes that occur within it.
- **Proactiveness (Goal-directedness):** The ability to exhibit goal-directed behavior by taking the initiative, rather than simply responding to the environment.

- **Social Ability:** The ability to interact with other agents (and humans) via some kind of agent-communication language to complete tasks.

3.3.3 Taxonomy of Agent Architectures

Agents are categorized by how they process information and represent their environment:

Reactive Agents Map percepts directly to actions via condition-action rules (Brooks 2003). Without an internal world model, they remain computationally efficient and robust but lack long-term foresight.

Deliberative Agents Maintain an **internal state** for **symbolic reasoning** and planning (Ghallab, Nau, and Traverso 2004). They simulate potential outcomes to evaluate complex, multi-stage action sequences.

Learning Agents Designed for unknown environments, these agents improve over time using four components: a *learning element* (improvement), a *performance element* (execution), a *critic* (feedback), and a *problem generator* (exploration) (Russell and Norvig 2010; Sutton and Barto 1998).

3.3.4 Core Components of an Agentic System

Regardless of the underlying implementation (be it symbolic logic, neural networks, or reinforcement learning), agents share a common structural blueprint (Wooldridge 1999).

1. **Perception and Sensor Fusion** Perception is the mapping of raw data from sensors (e.g., cameras, thermometers, or data streams) into a meaningful internal representation. In complex or ”inaccessible” environments, where the agent lacks complete, up-to-date information, systems must utilize sensor fusion to combine disparate data sources and reduce uncertainty.

2. **Knowledge Representation and Memory** To exhibit flexible behavior, agents often maintain an internal data structure to track the state and history of their environment.

    - **Procedural Knowledge:** The internal functions (often referred to as the *next-state* function) that dictate how the agent updates its internal state and selects actions.
    - **Declarative Knowledge (Beliefs):** Factual information about the world, represented as symbolic models in deliberative agents.
    - **Episodic Memory:** A record of past experiences and state transitions, which is essential for reasoning about the interactions between current and future scenarios.

3. **Reasoning, Planning, and Decision Making** The core of the agent is its decision-making logic, which maps internal states to actions. In the *Belief-Desire-Intention* (BDI) architecture, this involves ”deliberation” (deciding what goals to pursue) and ”means-ends reasoning” (deciding how to achieve them). This process evaluates the current state

against a desired **goal state** (Desires) to select an action sequence with high expected utility.

4. **Action Execution and Control** Once a decision is reached, the agent employs its "effectoric capability" to modify the environment. Because environments are often non-deterministic and dynamic, this component must monitor for "execution failures," where actions do not result in predicted outcomes.

5. **The Feedback and Learning Loop** Sophisticated agents utilize a feedback loop to facilitate adaptation. This is formalized in hybrid architectures like *InteRRaP*, which coordinate reactive behavior-based layers with higher-level planning layers to ensure the agent remains robust in non-stationary environments.

3.4 LLM-Based Agents

3.4.1 LLMs as Cognitive Engines

Large Language Models (LLMs) serve as the central cognitive engine or "brain" of modern autonomous agents, bridging the gap between passive text generation and active environmental engagement (L. Wang et al. 2024; Luo et al. 2025). As illustrated in Figure 3.5, this modern architecture maps directly to the core agentic components discussed in 3.3.4. Within this framework, the LLM functions as a general-purpose controller that orchestrates perception, memory retrieval, planning, and action execution (Dong et al. 2024).

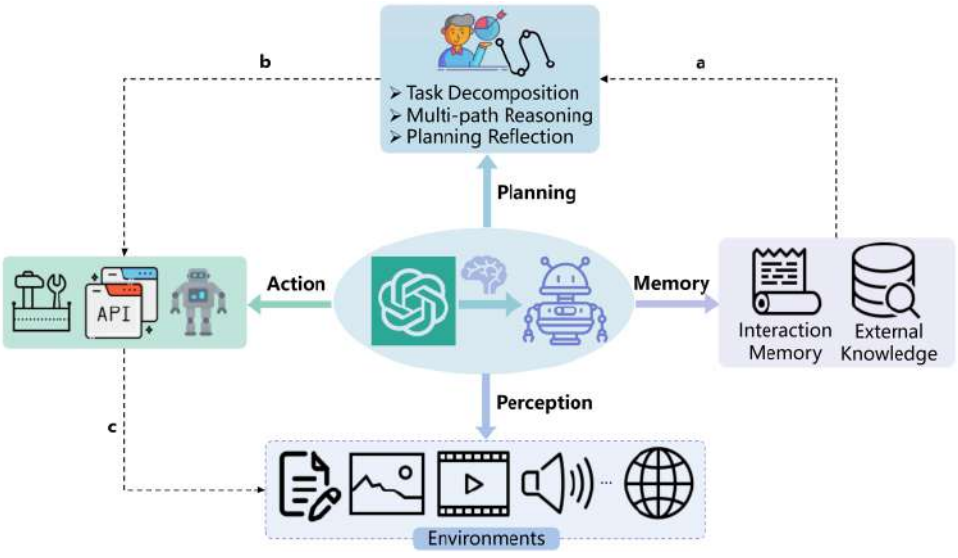


Figure 3.5: Overview of LLM-based agent architecture (Dong et al. 2024)

Unlike traditional symbolic AI, which relied on rigid logic and manually defined rules (Nwana 1996), LLM-based agents leverage vast parametric knowledge and zero-shot reasoning to exhibit the three hallmarks of intelligent agency: reactivity, pro-activeness, and social ability (Wooldridge 1999). By using natural language as a "universal interface," these systems can interpret ambiguous goals and decompose complex tasks into executable sub-steps (Dong et al. 2024; G. Wang et al. 2024).

3.4.2 Agent Architectures

Single-Agent Systems Utilize a unified LLM as an autonomous controller managing the reasoning-action cycle (Yao et al. 2023b). While efficient for bounded tasks, they are constrained by the context window and "short-term memory" of the underlying transformer (L. Wang et al. 2024). Despite the potential for long-term goal pursuit seen in frameworks like AutoGPT, these systems are prone to reasoning loops and error accumulation (Luo et al. 2025).

Multi-Agent Systems (MAS) Distribute cognitive load across specialized LLM entities to leverage modularity, parallelism, and reliability (Hong et al. 2024; Nwana 1996). Through collaborative strategies like "society of mind" debates, MAS facilitate cross-verification and error correction, often outperforming single agents in complex software and scientific domains (Du et al. 2024; G. Li et al. 2023).

Hierarchical Agent Architectures Separate strategic planning from tactical execution through layered control (L. Wang et al. 2024). By utilizing "manager" agents for task decomposition and "worker" agents for environmental interaction, this structure improves scalability and manages high-dimensional workflows more effectively than flat architectures (Hong et al. 2024; Nwana 1996).

3.4.3 Tool-Augmented LLM Systems

Retrieval-Augmented Generation (RAG) Grounds agent inference in external knowledge, allowing models to bypass static training data limitations (Lewis et al. 2020). By accessing real-time corpora, agents reduce hallucinations and adapt to current information streams (Dong et al. 2024).

Function Calling and API Interaction Agents utilize "effectoric capabilities" to interact with the environment through APIs and tools (Wooldridge 1999; Qin et al. 2024). Function calling enables transition from text generation to digital actions, such as database queries and web navigation (Patil et al. 2023; Luo et al. 2025).

Code Generation and Execution Agents solve complex problems by synthesizing and executing code within "trial-and-error" loops (M. Chen et al. 2021). This iterative process, generating code, observing logs, and refining solutions, is essential for autonomous data science and optimization (G. Wang et al. 2024; Luo et al. 2025).

3.4.4 Reasoning and Planning Strategies

Chain-of-Thought (CoT) and ReAct Elicits reasoning via intermediate "thinking" steps (Figure 3.6) (Wei et al. 2022b). The ReAct framework extends this by synchronizing reasoning traces with environmental actions, enabling dynamic plan updates based on external feedback (Yao et al. 2023b).

Tree-of-Thoughts (ToT) and Branching Search Treats reasoning as a search problem, exploring multiple paths simultaneously (Figure 3.6) (Yao et al. 2023a). This allows for "look-ahead" planning and backtracking when trajectories are deemed suboptimal (Besta et al. 2024).

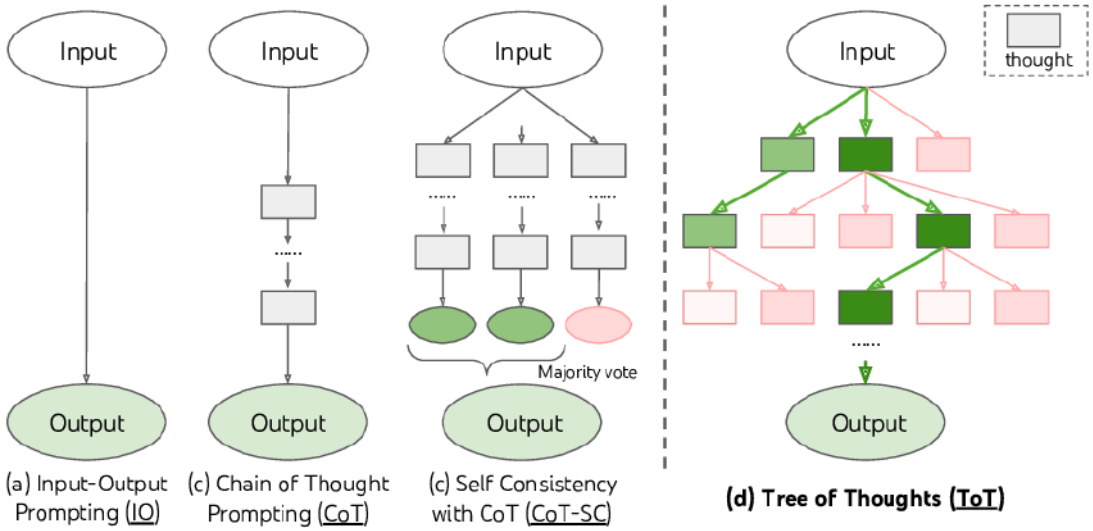


Figure 3.6: Visual comparison of CoT and ToT reasoning strategies (Yao et al. 2023a).

Reflection and Self-Improvement These methods empower agents to critique their own outputs and intentions (Figure 3.7) (Shinn et al. 2023). This meta-reasoning facilitates iterative self-improvement and the use of "long-term memory" to avoid repeating past failures (L. Wang et al. 2024; Luo et al. 2025).

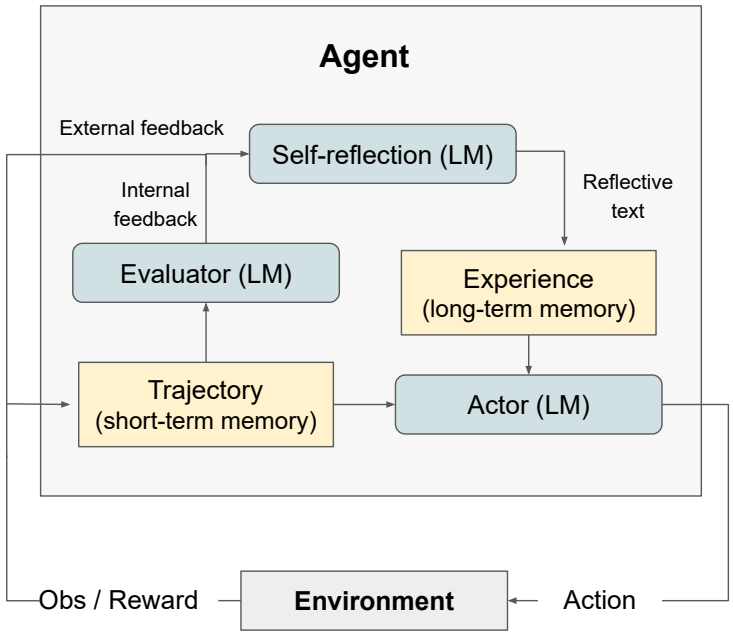


Figure 3.7: The iterative self-reflection loop for error correction (Shinn et al. 2023).

3.5 Limitations and Challenges of LLM-Based Agents

Despite rapid progress, LLM-based agents face several significant limitations.

Reliability and Hallucination Agentic systems inherit hallucination issues from underlying language models. Incorrect reasoning or fabricated information may propagate through planning and execution pipelines, potentially causing cascading failures (Ji et al. 2023).

Long-Horizon Planning Difficulties Although LLMs demonstrate strong short-term reasoning, maintaining coherent long-term plans remains challenging (N. F. Liu et al. 2024). Complex multi-step workflows often suffer from context fragmentation and accumulated reasoning errors.

Memory Limitations Current memory architectures remain limited in scalability, persistence, and retrieval accuracy. Efficient long-term memory integration remains an active research challenge (Packer et al. 2023).

Safety and Security Risks Agentic systems introduce new attack surfaces, including prompt injection, tool misuse, unsafe autonomous actions, and memory poisoning (K. Wang et al. 2025). The integration of external tools and autonomous decision-making increases the importance of robust alignment and safety mechanisms.

Computational Cost LLM-based agents often require repeated inference, retrieval, tool invocation, and multi-agent coordination, leading to substantial computational and financial costs.

3.6 LLM Agents for Scientific Discovery and Optimization

The integration of Large Language Models (LLMs) with autonomous agent architectures has enabled new paradigms in scientific discovery and optimization. By leveraging reasoning, tool-use, and iterative feedback loops, these agents can navigate vast search spaces that are traditionally inaccessible to manual human effort.

3.6.1 Key Examples of LLM Usage in Scientific Discovery and Optimization

Current research highlights several landmark applications of LLM agents in specialized scientific and optimization domains:

- **Chemical Synthesis and Lab Automation (Coscientist):** An autonomous agent that uses LLMs to design, plan, and execute complex chemical reactions by browsing the web for documentation and controlling robotic laboratory hardware (Boiko et al. 2023).
- **Scientific Reasoning (ChemCrow):** This agent integrates specialized chemistry tools (such as molecular docking and synthesis planners) to assist in drug discovery and materials synthesis through high-level reasoning (M. Bran et al. 2024).

- **LLM in Compilation:** In systems engineering, LLMs are used to optimize the compilation stack, automating the selection of optimization passes and the generation of compiler backends to handle the rapid expansion of diverse hardware architectures (S. Zhang et al. 2026; Zhong et al. 2025).

3.6.2 LLMs in Compilation: Taxonomy and Roles

In the domain of systems programming, the role of LLMs has evolved from simple code completion to core architectural integration. According to the taxonomy proposed in (S. Zhang et al. 2026), LLMs are integrated into compilers through three primary design philosophies:

- **LLM as Selector:** The model acts as a policy engine to choose the best optimization flags or pass sequences from a predefined set of valid compiler operations.
- **LLM as Translator:** The model directly rewrites code (e.g., source-to-source or source-to-assembly), treating compilation as a sequence-to-sequence translation task.
- **LLM as Generator:** The model is used to generate the source code of the compiler components themselves, rather than operating on the user's input program.

3.6.3 The LLM-as-Generator: Prioritizing Development Speed

A distinct trend in recent research is the use of LLMs to automate the labor-intensive process of compiler development. In this "Generator" paradigm (3.8), the primary goal is not to improve the quality of the compiler's output (e.g., code performance) but to **accelerate the implementation of compiler logic** and democratize development. Currently, only two major methods explore this approach for core compiler tasks (S. Zhang et al. 2026):

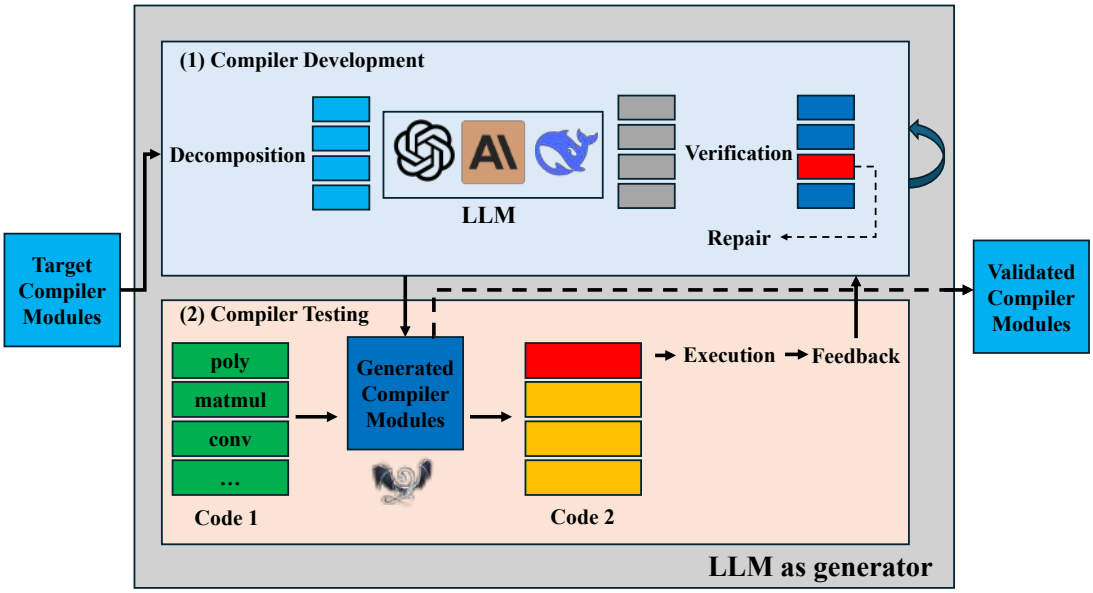


Figure 3.8: Overview of LLM-as-Generator methodology (S. Zhang et al. 2026)

1. **VEGA (Backend Generation):** VEGA leverages Transformers to automatically generate compiler backends for new processor architectures by synthesizing instruction templates and schedulers from hardware specifications (Zhong et al. 2025). Its main contribution is the ability to produce a complete backend in under an hour, solving the "hardware explosion" problem where manual backend development takes months.

2. **Code Transformers (Synthesizing Transformations):** Following the philosophy of "Don't Transform the Code, Code the Transforms," this method uses LLMs to synthesize the logic of code transformations as executable scripts (Cummins et al. 2024). By generating a deterministic transformation script rather than rewriting the code directly, the system provides a faster, more reliable, and debuggable development workflow for compiler engineers compared to manual implementation or error-prone direct LLM translation.

3.7 Conclusion

This chapter explored the foundational principles and rapid evolution of Large Language Models (LLMs), tracing their trajectory from traditional NLP architectures to the powerful, general-purpose cognitive engines that drive modern autonomous agents. By examining the underlying Transformer architecture, training paradigms, and the essential components of agentic systems, we established how LLMs can effectively orchestrate reasoning, planning, and tool interaction in complex environments.

Furthermore, we highlighted the application of LLM-based agents in scientific discovery and systems optimization. In the realm of compiler design, integrating LLMs presents a transformative opportunity. Specifically, utilizing the "LLM-as-Generator" paradigm emerges as a highly promising path forward. While existing generator frameworks primarily focus on accelerating the development speed of compilation toolchains, steering this generative capability toward the automated synthesis of complex, custom optimization strategies offers immense potential for directly improving overall execution performance.

Chapter 4

LLMs as Heuristic Designers

4.1 Introduction

The field of combinatorial optimization has long relied on heuristics and meta-heuristics to find high-quality solutions for computationally intractable problems. While these methods are powerful, designing them requires significant human expertise and empirical tuning. Hyper-heuristics emerged as a methodology to automate this design process, searching over a space of heuristics rather than the solution space directly (E. K. Burke et al. 2013). However, traditional hyper-heuristics often struggle with semantic blindness and a heavy reliance on predefined building blocks, limiting their performance and generality (F. Liu et al. 2026).

Recently, the integration of Large Language Models (LLMs) has sparked a paradigm shift in automated algorithm design (Fei Liu et al. 2024a; Romera-Paredes et al. 2024). By leveraging their vast pre-trained knowledge of programming languages and algorithmic logic, LLMs can reason about problem spaces, generate semantically rich code, and iteratively refine heuristic strategies. This chapter explores this transition, formally defining the role of LLMs as Heuristic Designers (LLM-AHD). The chapter begins by reviewing the theoretical foundations and limitations of traditional hyper-heuristics, followed by an analysis of how LLMs integrate into the algorithm design pipeline. Finally, a survey and synthesis of state-of-the-art LLM-AHD frameworks are presented, highlighting their evolving search structures, representations, and diversity mechanisms.

4.2 Theoretical Foundations of Hyper-Heuristics

4.2.1 The Heuristic Concept

A heuristic is a problem-solving device designed to operate under uncertainty. It provides no guarantee of completeness or optimality, but is expected to improve performance based on partial knowledge of a task environment (Romanycia and Pelletier 1985). This stands in contrast to an algorithm, which guarantees an optimal solution or determines insolvability within finite steps (Korfhage 2003). As (Feigenbaum and Feldman 1963) notes, a heuristic "may solve a given problem, but offers no guarantees of doing so."

The utility of a heuristic derives from its ability to capture and exploit structural regularities within a specific problem space (Lenat 1982). By leveraging these constraints, a heuristic can guide decision-making either by actively selecting among alternatives or by providing prioritized options for an executive process (Romanycia and Pelletier 1985). Rather than exhaustively searching for a globally optimal result, the heuristic prioritizes speed to find a sufficient or ”good enough” solution. Ultimately, a heuristic is a device added to a system under the reasoned belief, though not the certainty, that it will improve average performance on a targeted class of problems (Romanycia and Pelletier 1985).

4.2.2 Definition and Scope of Hyper-Heuristics

A hyper-heuristic (HH) operates on a search space of heuristics (or heuristic components) rather than directly on the solution space of the underlying problem (E. K. Burke et al. 2013). It is defined as “a search method or learning mechanism for selecting or generating heuristics to solve computational search problems” (E. Burke et al. 2010). Unlike a standard heuristic, which seeks a ”good enough” solution for a specific task, a hyper-heuristic seeks a good enough methodology for a class of tasks.

Following (Chakhlevitch and Cowling 2008), a hyper-heuristic is characterized by three key criteria:

(i) **Management:** It governs a library of low-level heuristics
(ii) **Search Level:** It searches for a high-performing method rather than a direct problem solution
(iii) **Abstraction:** It uses limited problem-specific information, effectively operating behind a ”domain-independent interface” to promote generality

The ultimate goal is to automate heuristic design and tuning, producing reusable, ”off-the-peg” methodologies that can be applied across different task environments with minimal reconfiguration (E. Burke et al. 2003; E. K. Burke et al. 2013).

4.3 Taxonomy of Hyper-Heuristics

The taxonomy of hyper-heuristics is generally divided into two main dimensions: the nature of the search space (what the system is doing) and the source of feedback (how the system learns). Following the standard classification by (E. K. Burke et al. 2010), the taxonomy can be broken down as follows (see Figure 4.1):

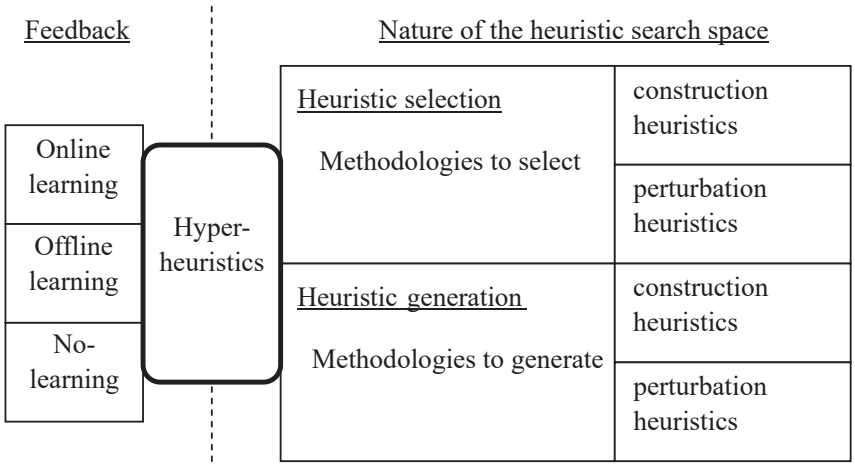


Figure 4.1: A classification of hyper-heuristic approaches. (E. K. Burke et al. 2010)

The Nature of the Search Space

This dimension distinguishes between systems that choose from existing tools and those that build new ones.

(i) **Heuristic Selection:** These methods move between existing low-level heuristics. The hyper-heuristic decides which heuristic to apply at any given state based on its current effectiveness.

(ii) **Heuristic Generation:** Instead of selecting, these methods use ”building blocks” (components of heuristics) to evolve or construct entirely new heuristic algorithms.

The Nature of the Underlying Search

Within those categories, hyper-heuristics are further classified by the type of process they manage:

(i) **Construction Hyper-heuristics:** These start with an empty solution and use low-level heuristics to build a complete solution piece by piece (e.g., bin packing or scheduling).

(ii) **Perturbation Hyper-heuristics:** These start with a complete (often sub-optimal) solution and use heuristics to ”perturb” or change it, searching for improvements (e.g., local search or ”move” operators).

Source of Feedback (Learning Mechanism)

This describes how the hyper-heuristic ”gets smarter” over time:

(i) **Online Learning:** The learning happens while solving a specific instance of a problem. The system tracks which heuristics are working well right now and adjusts.

(ii) **Offline Learning:** The system is trained on a set of representative problems to "learn" a general strategy. Once trained, the resulting "off-the-peg" methodology can be applied to new, unseen instances.

(iii) **No Learning:** The system uses a simple fixed strategy (like a round-robin) to toggle between heuristics without adapting to their performance.

4.4 Limitations of Traditional Hyper-Heuristics

Traditional hyper-heuristics (HH) face critical theoretical and practical constraints that limit their performance compared to human-designed algorithms.

Semantic Blindness and Syntactic Constraints

Generation HHs, particularly those using **Genetic Programming (GP)**, rely on **crossover** and **mutation** operators (Koza 1994). These are "semantically blind," manipulating code structure without functional logic (F. Liu et al. 2026). This lack of **semantic awareness** often leads to **code bloat**, excessive growth without utility, which can cause the search to stall in suboptimal regions (Poli et al. 2008).

The Meta-Design Burden

HH performance is strictly bounded by human-defined **low-level heuristics (LLH)** and **terminal/function sets** (E. K. Burke et al. 2013). If the necessary logic is not provided in the initial pool, the HH cannot discover it. This creates a **meta-design burden**, shifting human effort from algorithm design to the configuration of the search space (E. K. Burke et al. 2010).

Generality-Performance Trade-offs

To ensure **generality**, HHs use a **Domain Shield** to hide problem-specific details from the high-level strategy (E. Burke et al. 2003). However, according to the **No Free Lunch (NFL) theorem**, peak performance requires an algorithm to be deeply tailored to a specific problem (Wolpert and Macready 2002; Lenat 1982). Traditional HHs struggle with this trade-off: even though their heuristics contain domain-specific logic, the Domain Shield blocks the high-level mechanism from seeing, changing, or fixing that logic on the fly (E. K. Burke et al. 2013). This structural wall ultimately prevents traditional systems from achieving true, "made-to-measure" performance (E. K. Burke et al. 2013)

Complexity and Lack of Transferability

The search space for algorithm design is subject to the **"curse of dimensionality,"** where potential heuristic combinations grow exponentially (F. Liu et al. 2026). Traditional optimization methods often fail to navigate these vast spaces efficiently. Furthermore, these systems

exhibit poor **transferability**, lacking the high-level conceptual understanding required to apply design principles learned in one domain (e.g., scheduling) to another (e.g., routing) (F. Liu et al. 2026).

4.5 Theoretical Foundations of Large Language Models as a Hyper-Heuristic

4.5.1 Large Language Models in Algorithm Design

The integration of Large Language Models (LLMs) into Algorithm Design (AD) represents a paradigm shift from traditional, human-intensive development to an automated, data-driven synthesis of algorithmic logic (Fei Liu et al. 2024a; Fei Liu et al. 2026). Unlike classical Genetic Programming (GP), which evolves algorithms based on a predefined set of primitives and structural constraints, LLM-based approaches leverage the models' pre-trained knowledge of diverse programming languages and algorithmic paradigms (Fei Liu et al. 2024a; Romera-Paredes et al. 2024).

A critical enabler for this shift is **In-Context Learning (ICL)**, which allows an LLM to adapt its behavior and outputs based solely on instructions and examples provided within the prompt, without requiring formal parameter updates (F. Liu et al. 2026). In automated algorithm design, ICL enables the model to reason about problem specifications and iteratively refine code implementations using performance feedback (H. Ye et al. 2024; F. Liu et al. 2026). This iterative prompting loop effectively transforms the LLM into a "high-level search engine" that explores the space of executable programs, a concept known as **Language-based Program Search** (Romera-Paredes et al. 2024).

4.5.2 Taxonomy of LLM Roles in Algorithm Design

Recent systematic surveys categorize the utility of LLMs in the algorithm design pipeline into four primary roles based on their interaction with the problem-solving process (F. Liu et al. 2026; Fei Liu et al. 2026):

LLM as Optimizer (LLMaO): In this role, the LLM directly acts as a black-box optimizer. It receives a problem description and a history of previous candidate solutions, then generates a new solution aiming for a higher objective value (Yang et al. 2024). A hallmark framework for this role is **Optimization by PROmpting (OPRO)**, which utilizes the LLM's ability to identify patterns in the "optimization trajectory" to propose superior solutions (Yang et al. 2024; Fei Liu et al. 2026) (see Figure 4.2).

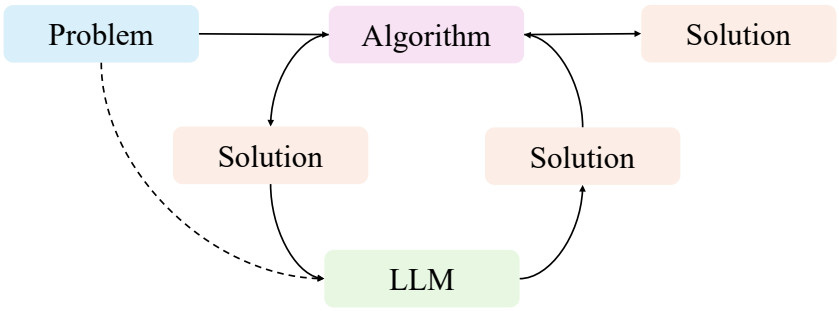


Figure 4.2: LLM as Optimizer (LLMaO) (F. Liu et al. 2026)

LLM as Predictor (LLMaP): LLMs function as Surrogate Models by predicting the performance or fitness of a candidate algorithm without requiring its actual execution. This role is crucial for filtering out low-quality candidates in high-throughput search frameworks, thereby reducing computational overhead (F. Liu et al. 2026; Stein and Bäck 2025) (see Figure 4.3).

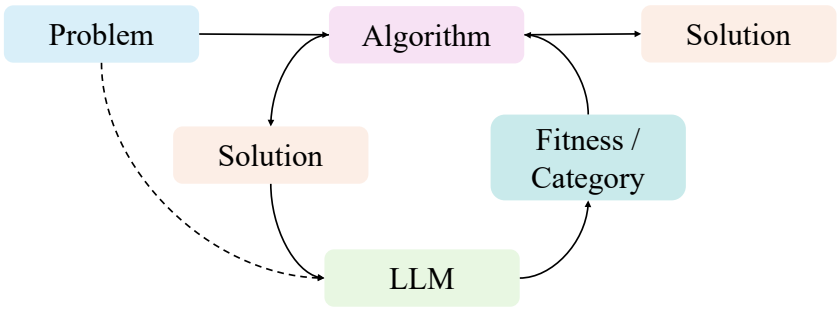


Figure 4.3: LLM as Predictor (LLMaP) (F. Liu et al. 2026)

LLM as Extractor (LLMaE): This involves using LLMs to parse unstructured, natural-language problem descriptions to extract formal Structural Features, mathematical constraints, or objective functions. This serves as a bridge between human-readable problem definitions and the structured inputs required by automated solvers (F. Liu et al. 2026) (see Figure 4.4).

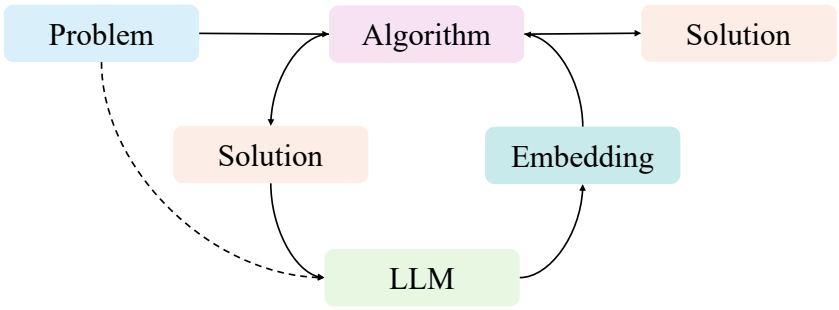


Figure 4.4: LLM as Extractor (LLMaE) (F. Liu et al. 2026)

LLM as Designer (LLMaD): This is the most complex role, where the LLM is responsible for synthesizing the core logic of the algorithm itself, such as a heuristic rule, a local search

operator, or a complete metaheuristic framework (Fei Liu et al. 2024a; H. Ye et al. 2024; Romera-Paredes et al. 2024) (see Figure 4.5).

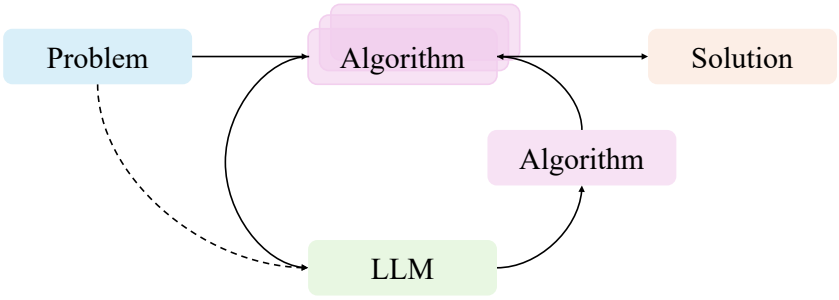


Figure 4.5: LLM as Designer (LLMaD) (F. Liu et al. 2026)

4.6 State-of-the-Art LLM-as-a-Heuristic-Designer (LLM-AHD) Frameworks

This section surveys five representative LLM-AHD frameworks, progressing from simple baselines to increasingly structured search strategies. Their design choices collectively expose the key axes of the field: search structure, heuristic representation, diversity management, and the degree to which LLM reasoning is exploited beyond raw code generation.

4.6.1 FunSearch: Evolutionary Program Search

FunSearch (Romera-Paredes et al. 2024) was among the first systems to demonstrate that an LLM paired with a principled search loop can surpass the best-known human solutions on open mathematical problems. Its core mechanism maintains a *programs database* of evaluated heuristics ranked by fitness. At each step, a pair of high-scoring programs is sampled as context, and the LLM is asked to complete or extend a partially specified function, performing recombination in program space. An *island model* of independent sub-populations guards against premature convergence. The key representational insight is that FunSearch searches in *function space* rather than solution space: the LLM generates the logic that constructs solutions, not the solutions themselves, producing programs that are interpretable and generalizable (see Figure 4.6).

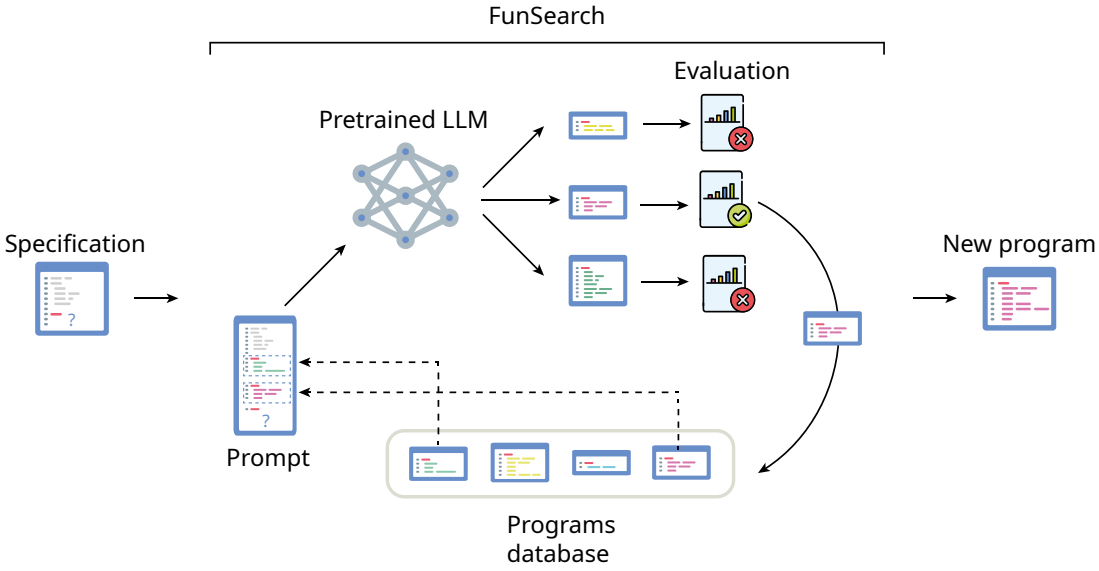


Figure 4.6: Overview of FunSearch (Romera-Paredes et al. 2024)

4.6.2 EoH: Evolution of Heuristics

EoH (Fei Liu et al. 2024a) introduces a *dual representation*: each heuristic is encoded as a natural-language *thought* capturing its conceptual intent and a Python *code* implementation. Both evolve together via *crossover* (the LLM synthesizes two parents' thoughts and codes into a novel offspring) and *mutation* (the LLM refines a single parent's thought and regenerates its code). This thought-to-code paradigm follows a Chain-of-Thought discipline (F. Liu et al. 2026): the model articulates design intent before committing to syntax, substantially reducing logically inconsistent implementations. EoH surpasses both hand-crafted heuristics and FunSearch on standard combinatorial benchmarks with a lower query budget (see Figure 4.7).

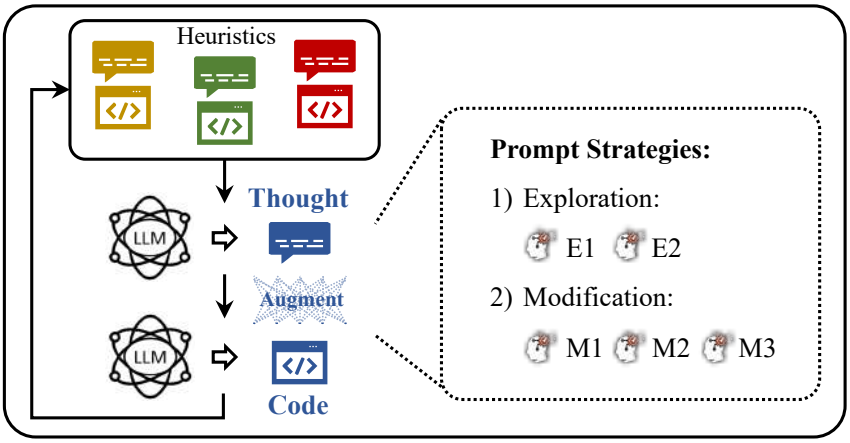


Figure 4.7: Overview of EoH (Fei Liu et al. 2024a)

4.6.3 ReEvo: Reflective Evolution

ReEvo (H. Ye et al. 2024) replaces numerical fitness signals with *verbal gradients*: the LLM is explicitly asked to diagnose why one heuristic outperforms another and to articulate improve-

ment directions in prose. Two complementary reflection levels are maintained, *short-term reflection* contrasts an offspring with its immediate parent, while *long-term reflection* compares the current population best against the best-ever, diagnosing persistent structural limitations. These verbal analyses are recycled as context in subsequent queries, forming a continuously updated improvement signal. Unlike most frameworks, ReEvo deliberately omits explicit diversification mechanisms, delegating population spread entirely to LLM stochasticity, and yet achieves state-of-the-art results across six combinatorial optimization problems (see Figure 4.8).

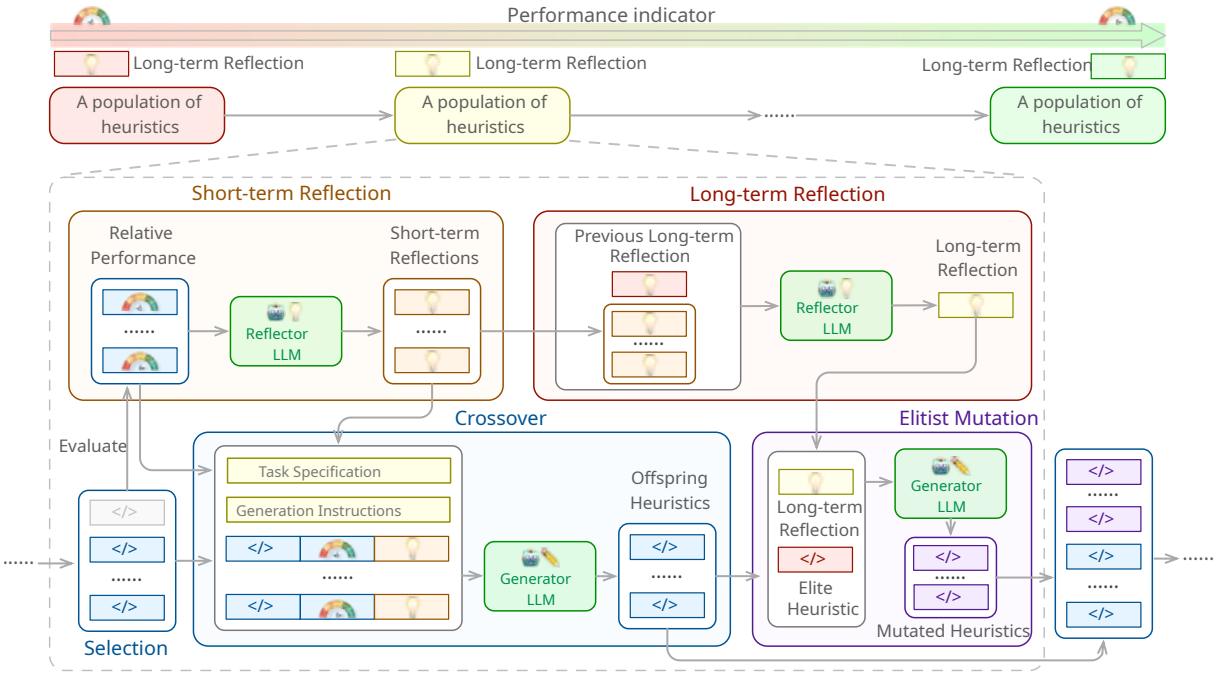


Figure 4.8: Overview of ReEvo (H. Ye et al. 2024)

4.6.4 MCTS-AHD: Monte Carlo Tree Search for Heuristic Design

MCTS-AHD (Z. Zheng et al. 2025) replaces the flat evolutionary population with a tree-structured search: each node represents a heuristic and edges encode LLM-generated transitions. Selection follows the UCT criterion, balancing exploitation of high-scoring subtrees with exploration of less-visited branches. *Progressive Widening* controls branching factor dynamically to prevent combinatorial explosion, while *Exploration-Decay* shifts the search from broad discovery toward focused refinement over time. Each node expansion uses a *Thought-Alignment Procedure*: two sequential LLM calls where a thought is generated first and code is conditioned on it and the full path from root to current node is supplied as context (*Tree-Path Reasoning*), enabling the LLM to reason about the cumulative design history rather than treating each generation in isolation (see Figure 4.9).

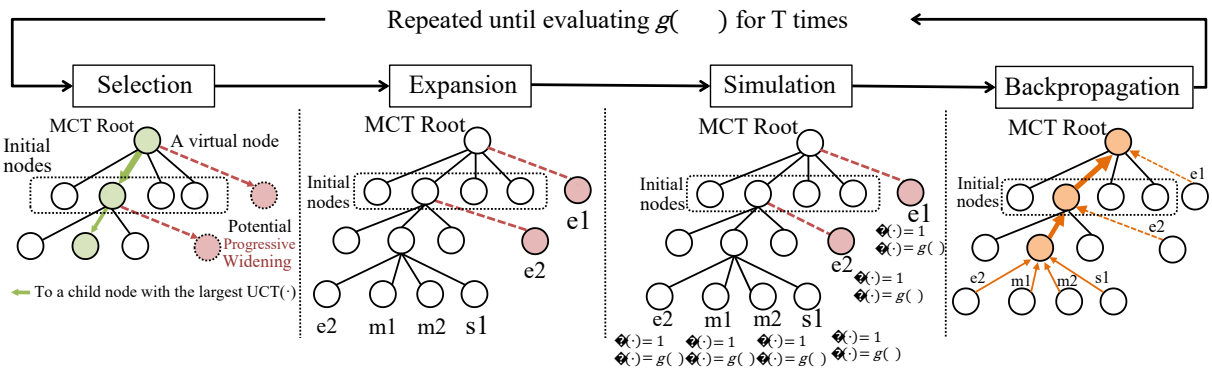


Figure 4.9: Overview of MCTS-AHD (Z. Zheng et al. 2025)

4.6.5 PartEvo: Niching-Enhanced Evolution

PartEvo (Hu and Q. Zhang 2026) addresses diversity collapse by introducing explicit *niching*: the population is partitioned into specialized sub-groups before any operator is applied, so that distinct heuristic strategies cannot be eliminated by global selection pressure. Each heuristic is described by a *triple representation*, a *Code Similarity Vector* (CSV) from Abstract Syntax Tree-based (AST) structural features, a *Thought Embedding* from BERT, and a fitness score, and K-Means clustering over the first two dimensions constructs the niches. Four operators are then applied within and across niches: *Random Evolution* (RE) for broad exploration, *Structured Evolution* (SE) for similarity-guided crossover, *Crossover within Niches* (CN) for local exploitation, and *Local Genetic Evolution* (LGE) for fine-grained mutation of each niche's incumbent. Batch generation amortizes LLM query cost, and together these mechanisms sustain rich population diversity throughout the run (see Figure 4.10).

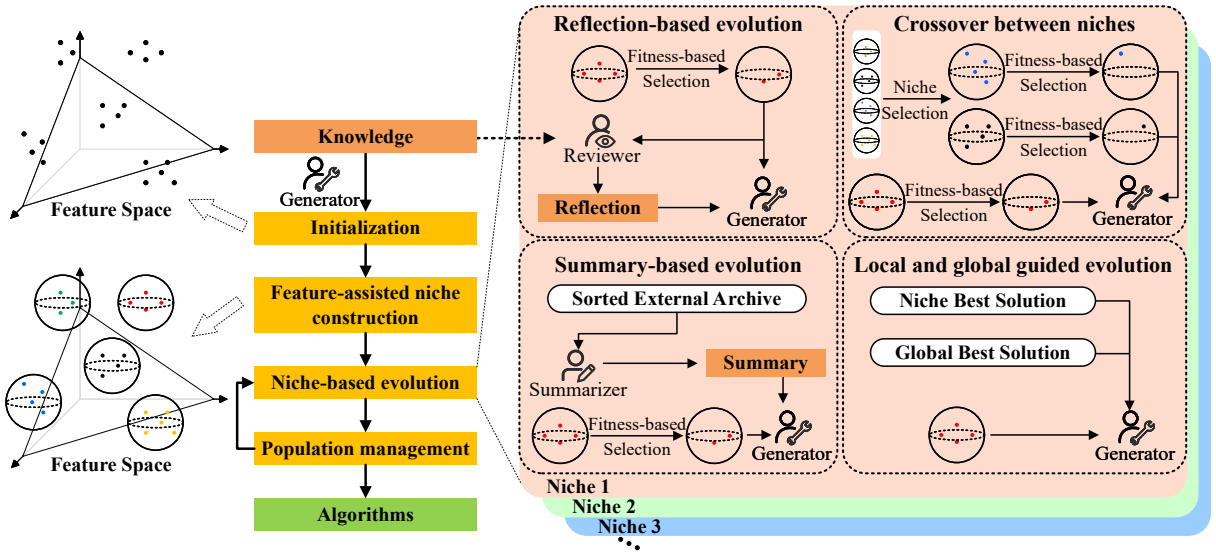


Figure 4.10: Overview of PartEvo (Hu and Q. Zhang 2026)

4.7 Synthesis

4.7.1 Taxonomy: Three Generations of LLM-AHD

Three generations emerge from the surveyed frameworks, each expanding what the LLM is asked to do. Table 4.1 summarizes the resulting classification.

Generation 1 (FunSearch) uses the LLM purely as a code generator the loop operates on numerical fitness alone and is semantically blind to *why* programs differ.

Generation 2 (EoH, ReEvo, MCTS-AHD) elicits structured reasoning, thought scaffolds, verbal gradients, or tree-path histories and feeds it back into the loop, substituting for gradient information classical optimizers rely on.

Generation 3 (PartEvo) inherits this reasoning and adds explicit diversity management, recognizing that the LLM's prior is concentrated enough to collapse any loop that does not actively counteract it.

Table 4.1: Taxonomy of the five surveyed LLM-AHD frameworks.

Framework	Gen.	Search Structure	Representation	Diversity Mechanism	Feedback beyond fitness
FunSearch	1	Island-based evolution	Code only	Island model	None
EoH	2	Population evolution	Thought + Code	Population crossover	Chain-of-Thought
ReEvo	2	Population evolution	Code only	LLM stochasticity	Short- & long-term verbal gradient
MCTS-AHD	2	Monte Carlo Tree Search	Thought + Code	Progressive Widening	Tree-Path Reasoning
PartEvo	3	Niching-based evolution	Thought + Code	K-Means niching	Structured Evolution

4.7.2 Evaluation

LLM-AHD frameworks are evaluated on classical combinatorial optimization (CO) benchmarks such as the Traveling Salesman Problem (TSP), Capacitated Vehicle Routing (CVRP), and online Bin Packing (BPP). Three criteria are commonly used:

Solution quality. The gap between the objective value achieved by the generated heuristic and that of a reference, either a well-known hand-crafted baseline or the output of an exact solver. This is the primary measure of whether the designed algorithm is practically useful.

Sample efficiency. The number of LLM queries required to reach a target quality level. Since each query carries a non-trivial computational and financial cost, this metric captures the practical viability of the search process and allows frameworks with different designs to be compared on equal footing.

Population diversity. A framework (PartEvo) explicitly tracks the variety of distinct heuristic strategies maintained throughout the run, using metrics such as AST-based pairwise dissimilarity. High diversity indicates that the search is genuinely exploring the space rather than converging on a single idiom.

4.7.3 Limitations of the Surveyed LLM-AHD Frameworks

Despite empirical results, the surveyed frameworks expose several critical gaps. These limitations directly motivate the design choices in Chapter 5:

Weak domain grounding and redundant exploration. Most systems rely on generic prompts and short-term population context, which encourages rediscovery of baseline ideas or repetition of previously failed mechanisms.

Unstructured knowledge and semantic drift. Verbal gradients or free-form reflections provide feedback but are difficult to verify and are prone to drift across iterations.

Operators not tied to causal claims. Crossover and mutation typically recombine code without explicitly testing why a change should help, making improvement signals hard to interpret.

Diversity control decoupled from accumulated knowledge. Even when diversity is tracked, it is often not guided by explicit negative evidence, leading to stagnation around local optima.

4.8 Conclusion

This chapter positioned LLMs as heuristic designers by grounding them in the hyper-heuristic tradition, surveying representative LLM-AHD frameworks, and synthesizing their design trends. The analysis highlights a clear evolution toward structured reasoning and diversity-aware search, while also exposing persistent gaps in grounding, knowledge representation, operator semantics, and stagnation control.

Furthermore, it is important to note that these state-of-the-art LLM-based automatic algorithm design methods have not yet been explored in the context of quantum compilation, nor have LLMs been extensively used to directly improve compiler performance. These limitations and the untapped potential in the quantum domain motivate the architecture presented in Chapter 5.

Chapter 5

Design and Implementation

5.1 Introduction

The preceding chapters established complementary halves of a single opportunity. Chapter 2 showed that qubit mapping in the NISQ era is dominated by hand-designed heuristics whose scoring functions must be re-tuned by hand for every new optimization objective, turning heuristic design into a recurring manual bottleneck. Chapter 4 showed that LLM-based Automated Heuristic Design (LLM-AHD) has matured into a paradigm capable of automating exactly this kind of design effort, yet has never been applied to quantum compilation. This chapter brings the two together by presenting **QPilot**, an LLM-based AHD framework that automates the design of initial qubit mapping heuristics, and by detailing the architecture, search pipeline, and concrete implementation through which it synthesizes them.

Section 5.2 makes the gap explicit, tracing the manual-design bottleneck inherited from the mapping literature and the limitations inherited from existing LLM-AHD frameworks, and maps each onto a corresponding QPilot design choice. Section 5.3 then describes the modular architecture and the role of each component, before Section 5.4 formalizes the four-stage closed-loop search pipeline at a high level. Section 5.5 develops each stage in detail, from knowledge-grounded ideation through the Hypothesis-Driven Knowledge Graph to evolutionary refinement and re-ideation. Finally, Section 5.6 reports the concrete realization of the framework, covering its problem scope, software stack, model selection, and configuration parameters.

5.2 Problem Presentation and Motivation for LLM-based AHD

Before introducing the architecture of QPilot, we make explicit the gap that the framework is designed to fill. This gap has two components: one inherited from the qubit mapping literature of Chapter 2, and one inherited from the LLM-AHD literature of Chapter 4.

5.2.1 The Manual-Design Bottleneck in Qubit Mapping

The synthesis in Chapter 2 showed that, among exact, learning-based, and heuristic methods, heuristics are the family that scales to the circuit and device sizes of practical NISQ compilation. Exact formulations guarantee optimality but are confined to a handful of qubits, while learning-based models incur substantial training cost and tend to generalize poorly to unseen topologies. Heuristics such as SABRE (Gushu, Ding, and Xie 2019) and dependence-driven methods such as Qlosure (Benbetka et al. 2026) therefore remain the default in production toolchains.

This reliance carries a cost that the survey made apparent: the scoring functions at the core of these heuristics are designed and tuned by hand. Each cost function encodes expert assumptions about which structural signals (physical distance, lookahead depth, dependence weight) should drive mapping policies, and adapting these assumptions to a new device topology or optimization objective requires renewed manual effort (Murali et al. 2019). As hardware diversifies, this manual-design step becomes a recurring bottleneck rather than a one-time investment.

LLM-based AHD is a natural candidate for relieving this bottleneck: it frames heuristic design as an automated search over executable, human-readable code (Romera-Paredes et al. 2024; Fei Liu et al. 2024a), so that the resulting heuristics remain interpretable and directly deployable within existing compilers rather than hidden inside an opaque model. However, as noted in the conclusion of Chapter 2, this paradigm has not yet been applied to quantum compilation to date, the qubit mapping heuristic has remained a human artifact. Whether existing LLM-AHD frameworks are ready for this transfer is the question the next subsection examines.

5.2.2 Limitations of Existing LLM-AHD Frameworks

Chapter 4 surveyed the frameworks that could, in principle, automate this design effort and identified four limitations that any application must confront:

(L1) **Weak domain grounding and redundant exploration.** Generic prompts and short-horizon population context lead the search to rediscover baseline ideas or to repeat previously failed mechanisms.

(L2) **Unstructured knowledge and semantic drift.** Feedback is accumulated as free-form natural-language reflections or verbal gradients, which are difficult to verify and tend to drift across iterations (H. Ye et al. 2024).

(L3) **Operators decoupled from causal claims.** Crossover and mutation recombine code without stating, or testing, why a given change is expected to help, leaving improvement signals hard to interpret.

(L4) **Diversity control decoupled from accumulated knowledge.** Even when diversity is tracked, it is rarely steered by explicit negative evidence, so the search tends to stagnate around local optima.

A further limitation concerns scope rather than mechanism. As discussed in the synthesis of Section 4.7, these frameworks have been validated almost exclusively on classical combinatorial benchmarks (TSP, CVRP, BPP) and have not been exercised under the constraints specific

to qubit mapping, namely a fixed hardware coupling map, a downstream routing backend, and SWAP-count and circuit-depth objectives.

5.2.3 From Limitations to QPilot Design Choices

QPilot is positioned as a direct response to the two preceding gaps. Its first contribution is one of scope: it instantiates the LLM-AHD paradigm for the qubit mapping problem, coupling the generator to the Qlosure routing backend so that every candidate heuristic is evaluated under realistic hardware constraints (Section 5.6). This addresses the unexplored application gap identified in Chapter 2. Its remaining contributions map onto the limitations (L1)–(L4) of existing frameworks, as summarized in Table 5.1 and detailed across Sections 5.4–5.5.

Table 5.1: Mapping between the limitations of existing LLM-AHD frameworks (Chapter 4) and the corresponding QPilot design choices.

Limitation (Chapter 4)	QPilot design response	Detailed in
(L1) Weak domain grounding and redundant exploration	Knowledge-grounded ideation: a literature-derived *Contextual Digest* and a persistent cross-run memory that enforces an orthogonality constraint against baselines and recorded failures.	Sec. 5.5.1
(L2) Unstructured knowledge and semantic drift	Hypothesis-Driven Knowledge Graph (HD-KG): discrete traits and falsifiable hypotheses carrying explicit confidence scores and validation status.	Sec. 5.5.3
(L3) Operators decoupled from causal claims	Hypothesis-conditioned crossover and targeted mutation, where each offspring is a controlled test of a stated claim whose outcome updates the hypothesis confidence.	Sec. 5.5.4
(L4) Diversity control decoupled from accumulated knowledge	Stagnation-triggered re-ideation that reuses falsified and exhausted hypotheses as explicit negative constraints.	Sec. 5.5.4

Concretely, QPilot addresses (L1) by grounding generation in a literature-derived *Contextual Digest* and a persistent memory of prior runs, so that new candidates are constrained to be orthogonal to both established baselines and recorded failures (Section 5.5.1). It addresses (L2) by replacing free-form reflection with a Hypothesis-Driven Knowledge Graph (HD-KG) whose discrete traits and falsifiable hypotheses carry explicit confidence scores and a validation status (Section 5.5.3). It addresses (L3) by conditioning each crossover and mutation on a specific hypothesis, so that every generated offspring is a controlled test whose measured performance updates that hypothesis confidence (Section 5.5.4). Finally, it addresses (L4) by triggering re-ideation on stagnation and using the falsified and exhausted hypotheses of the HD-KG as explicit negative constraints, so that renewed exploration is steered by accumulated evidence rather than by model stochasticity alone (Section 5.5.4).

The remainder of this chapter develops each of these design choices in turn, beginning with the overall architecture in Section 5.3.

5.3 QPilot Framework Architecture and Components

The QPilot framework is organized into a modular architecture where specialized components interact under the coordination of a central orchestrator. This design ensures a separation of concerns between the generative reasoning of the LLM, the evaluation of qubit mapping heuristics, and the management of experimental state. Figure 5.1 illustrates the structural relations between these components.

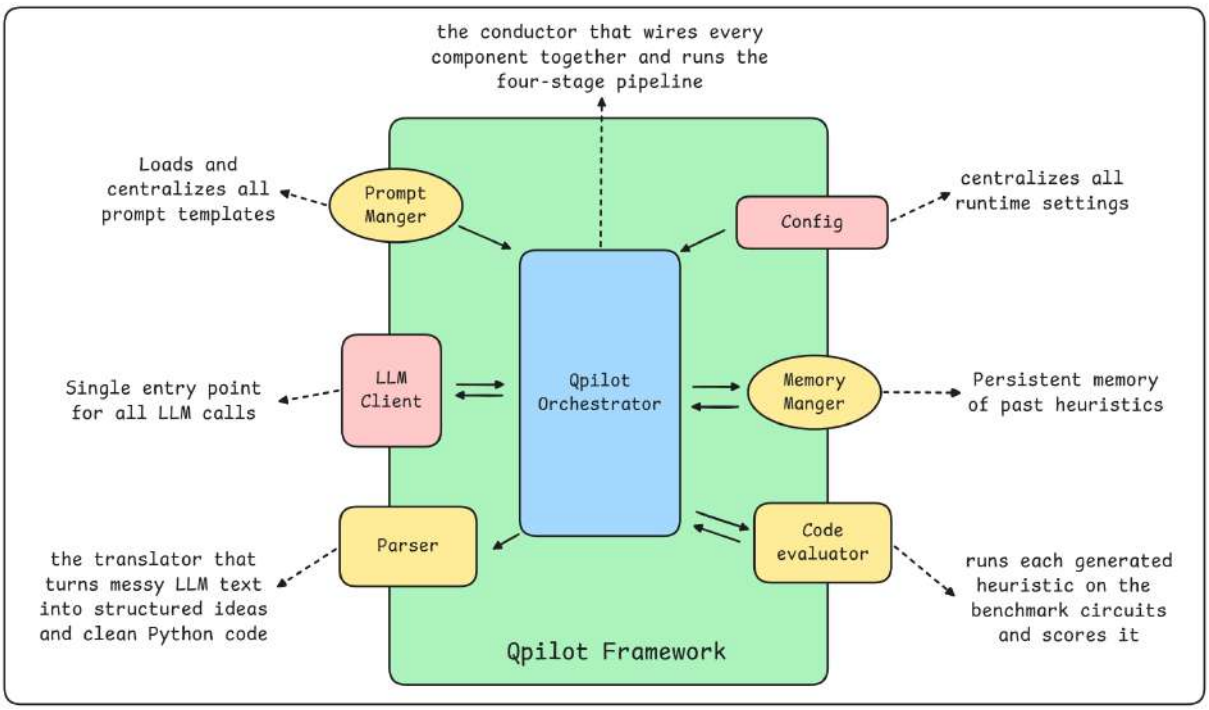


Figure 5.1: Architecture of the QPilot framework.

In the following paragraphs, we define the theoretical role and functional responsibility of each component within the QPilot Automatic Heuristic Design (AHD) loop.

1. **QPilot Orchestrator:** The central control unit of the framework. Its primary responsibility is to manage the operational workflow and facilitate data exchange between the modules. It dictates the transition between different functional phases, ensuring that the insights generated by the LLM are correctly parsed, evaluated, and stored. By acting as the ”brain” of the framework, it maintains the high-level logic of the iterative design process.

2. **Configuration Management:** Defines the static and dynamic parameters that govern an AHD experiment. It encapsulates environmental constraints, such as the target quantum hardware topology and the hyperparameters for the LLM and the evolution process. This component provides the necessary boundaries and objectives that guide the framework's search for optimal heuristics.

3. **Prompt Manager:** Responsible for the construction of instructions sent to the LLM. It acts as a context engine that translates the current state of the experiment, previous failures, and successful strategies into structured natural language prompts.

4. **LLM Client:** Provides an interface to interact with LLMs. Its role is to manage the communication lifecycle. Theoretically, this component abstracts the specific underlying model, allowing the framework to leverage different generative engines while maintaining consistent tracking of computational resources and reasoning outputs.

5. **Parser:** Serves as the bridge between unstructured natural language reasoning and formal algorithmic logic. Its function is to extract executable code and structured ideas from the LLM's responses. By isolating the mathematical and logical components of a proposed heuristic, it transforms a linguistic suggestion into a deterministic artifact that can be rigorously tested.

6. **Code Evaluator:** This component quantifies the "fitness" of a heuristic by measuring objective performance metrics, such as swap gate count and circuit depth on representative quantum circuit benchmarks, providing the Framework Orchestrator with the data required to guide subsequent iterations.

7. **Memory Manager:** This component maintains a historical record of evaluated strategies to prevent redundant exploration and manages a structured knowledge base of algorithmic traits. This allows the framework to transition from a stochastic search to a hypothesis-driven evolution, where past empirical evidence informs the generation of future heuristic candidates.

5.4 QPilot Automatic Heuristic Search Pipeline

The QPilot framework formalizes Automatic Heuristic Design (AHD) for the qubit mapping problem as an iterative, closed-loop interaction between a Large Language Model and an empirical evaluation environment. Rather than treating the LLM as a one-shot code generator (Romera-Paredes et al. 2024), QPilot organizes its reasoning capabilities into a structured discovery pipeline in which heuristics are progressively synthesized, empirically scored on representative quantum circuits, and refined through evolutionary operators guided by accumulated knowledge. The objective of this orchestration is to converge towards heuristics that minimize SWAPs and depth overhead while remaining interpretable, executable in polynomial time, and generalizable across hardware topologies.

Figure 5.2 depicts the global pipeline. The framework is decomposed into four conceptual stages: (i) an *Ideation* stage that grounds the LLM in domain literature and translates this knowledge into a diverse pool of candidate algorithmic strategies, (ii) an *Implementation and Empirical Selection* stage that materializes those strategies into executable heuristics and ranks them through empirical benchmarking, (iii) a *Traits Extraction and Hypothesis Generation* stage that builds a Hypothesis-Driven Knowledge Graph (HD-KG) to capture discrete algorithmic traits and causal claims, and (iv) an *Evolutionary Refinement and Re-Ideation* stage that applies evolutionary operators conditioned on the HD-KG while periodically re-injecting diversity to prevent premature convergence.

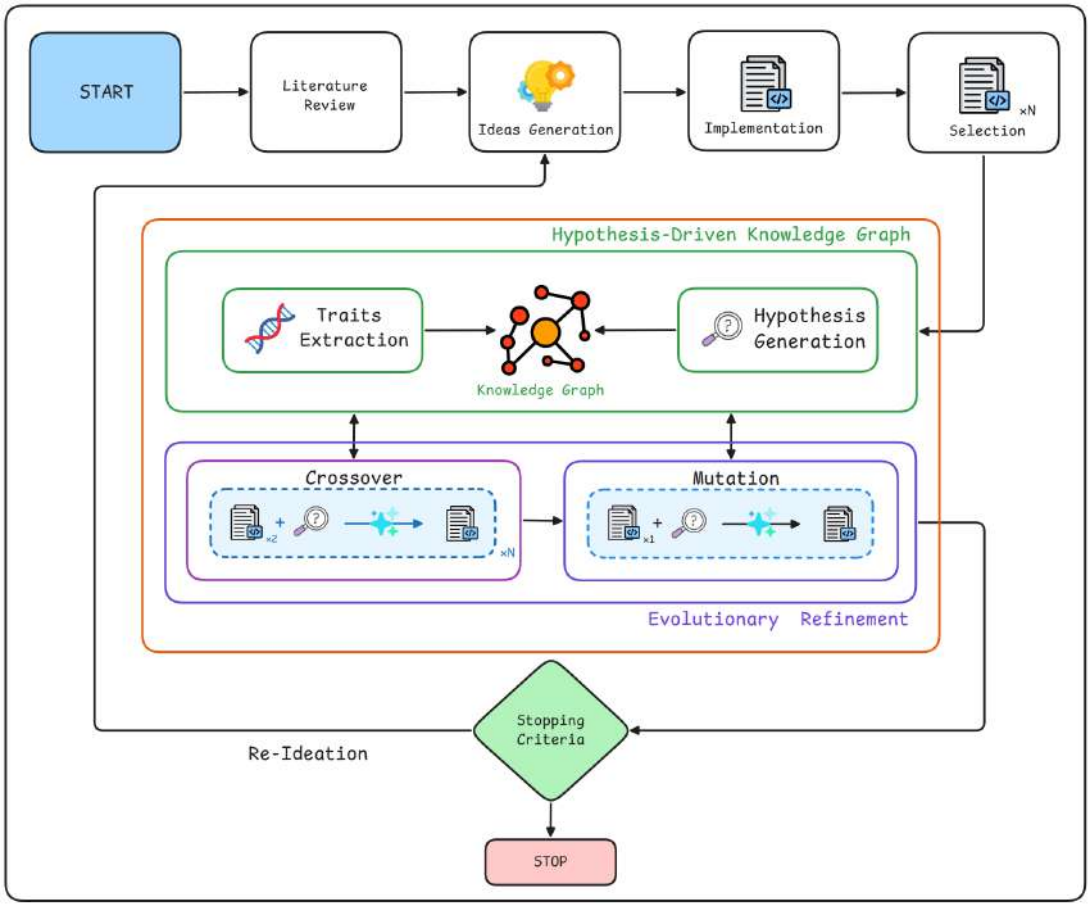


Figure 5.2: Global pipeline of the QPilot framework for Automatic Heuristic Design.

The next few paragraphs provide a high-level description of each of these stages :

1. **Knowledge-Grounded Ideation:** The ideation stage anchors generation in a relevant context to escape irrelevant heuristics for ideas. The LLM first performs a literature review of qubit mapping methods, producing a contextual summary used in later stages. Building on this, the model generates algorithmic primitives for qubit mapping. A persistent memory of past experiments is provided as context to discourage redundancy, enabling exploration of new regions of the design space.

2. **Implementation and Empirical Selection:** Each idea is converted into an executable heuristic under fixed interface constraints and evaluated on a benchmark suite using the Qlosure backend, measuring SWAP count and circuit depth. This step grounds proposals in empirical performance. The best-performing heuristics seed the evolutionary loop, while all results (successful or not) are stored in persistent memory with metrics. This enables long-term reuse of empirical evidence and prevents repeated exploration of poor strategies.

3. **Knowledge Extraction and Hypothesis Generation:** A central contribution of the framework is the introduction of a Hypothesis-Driven Knowledge Graph (HD-KG) that guides the evolutionary process. Current works on LLM-based heuristic search rely on unstructured natural-language reflections (H. Ye et al. 2024), which are difficult to verify

and prone to drift. In contrast, the HD-KG imposes a discrete, falsifiable representation of the knowledge accumulated across iterations.

The graph is populated in two steps. A *trait extraction* step asks the LLM to act as a reflective analyst, to identify algorithmic traits across the top and bottom performing heuristics in the current population. A *hypothesis generation* step then formulates testable causal claims to link specific traits. Each hypothesis is registered in the graph with an associated confidence score, status, and trial history. Confidence of the hypothesis is updated empirically whenever a hypothesis is invoked during generation, the resulting offspring is compared against its parents, and the outcome is interpreted as evidence supporting or weakening the hypothesis.

4. **Evolutionary Refinement and Re-Ideation:** The refinement stage implements an evolutionary loop in which the population of heuristics is improved through two operators conditioned on the HD-KG. A *crossover* operator selects pairs of parent heuristics and instructs the LLM to synthesize an offspring that combines the strengths of both with respect to a sampled hypothesis. A *mutation* operator perturbs the elite individual under the guidance of a high-confidence hypothesis, inducing a local refinement of the best-known heuristic. Both operators reformulate generation as a controlled experiment: each offspring constitutes a concrete test of a stated algorithmic claim (hypothesis), and its empirical performance feeds directly back into the confidence update of the corresponding hypothesis.

   To prevent stagnation and diversity loss that typically affects evolutionary search, the framework periodically interleaves a *re-ideation* phase. Where the LLM is prompted to propose new ideas explicitly orthogonal to all directions already explored. The resulting candidates are implemented, evaluated, and merged with the elite of the previous generation to seed the next iteration. This mechanism re-establishes exploration without discarding previously acquired knowledge, allowing the framework to escape local optima while remaining anchored in its empirical history.

5.5 QPilot Automatic Heuristic Design Pipeline in Details

5.5.1 Knowledge-Grounded Ideation Stage

The first operational phase of the QPilot framework is the ideation stage, designed to anchor the generative capabilities of the LLM in established domain expertise. Traditional zero-shot generation often leads to the proposal of mathematically unsound or trivially suboptimal strategies (Romera-Paredes et al. 2024). To counteract this, QPilot formalizes the generation of initial heuristic seeds as a context-aware, memory-augmented process. This phase produces a diverse pool of algorithmic concepts and ideas that respect quantum hardware constraints and avoid previously exhausted regions of the design space.

Automated Literature Synthesis

Prior to generating any novel ideas for the qubit mapping problem, the framework executes a structured literature review phase. The LLM is prompted to assume the role of a domain expert and synthesizes core strategies from seminal qubit mapping literature (e.g., SABRE). The output is a *Contextual Digest* that encapsulates known algorithmic primitives and strictly defines hardware constraints, such as the topological limits of the target QPU's coupling map. By injecting this digest into the system prompt of subsequent generative tasks, the framework aligns the LLM's contextual reasoning with the state-of-the-art, thereby preventing the re-invention of rudimentary baselines.

Memory-Augmented Seed Generation

Building upon the literature digest, the framework initiates the generation of candidate heuristic strategies. To guarantee that the search process continuously explores novel algorithmic paradigms, QPilot integrates a persistent *Memory Manager*. This component maintains a historical archive of all previously evaluated ideas across prior runs, tracking their structural descriptions alongside empirical performance metrics (e.g., mean SWAP overhead, critical path depth, and execution status).

During the prompt formulation for seed generation, the Memory Manager computes a structured summary comprising three partitions:

- **Top Performers:** The k-best historical heuristics, establishing an empirical performance target.
- **Recent Experiments:** The immediate predecessors in the active run to maintain search trajectory context.
- **Archived Failures:** Summaries of heuristic concepts that previously yielded runtime errors or severe SWAP overheads.

By explicitly injecting this historical context, the LLM is conceptually constrained. It is instructed to enforce an orthogonality constraint, ensuring that newly proposed strategies utilize mechanisms fundamentally distinct from both the established literature and the framework's recorded failures.

Idea Parsing and Structuring

As illustrated in Figure 5.3, the natural language output generated by the LLM must be systematically converted into a standardized format before programmatic implementation. LLMs inherently produce conversational filler alongside technical content. To resolve this, QPilot employs an *Idea Parser* that applies robust regular expressions and block-detection algorithms to extract discrete "Idea Blocks."

Each extracted Idea Block is rigorously formatted to contain:

1. **Strategy Name:** A distinct identifier for the proposed heuristic.

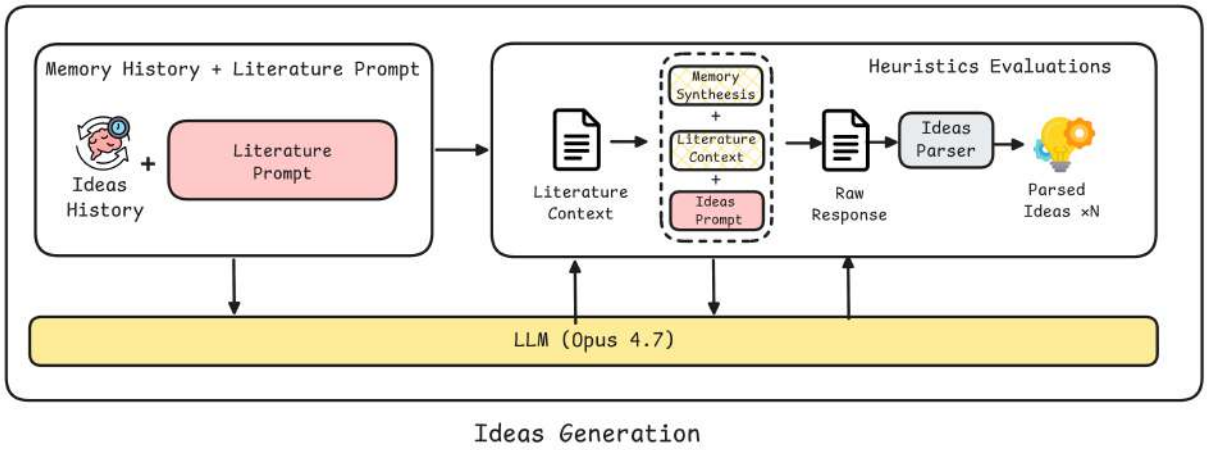


Figure 5.3: Internal structure of the ideas generation stage.

2. **Conceptual Logic:** A rigorous natural language formulation of the heuristic's underlying decision-making mechanism (e.g., defining exactly how topological distance or node degrees map to penalty scores).
3. **Self-Critique and Risks:** An LLM-generated reflection regarding the potential computational bottlenecks or edge-case failure modes inherent to the proposed logic.

This structured extraction effectively decouples the conceptual algorithmic intent from the raw code synthesis. It ensures that each strategy can be independently registered within the framework's long-term memory graph before being passed downstream to the automated implementation and evaluation pipelines.

5.5.2 Implementation and Empirical Evaluation Stage

Following the extraction of algorithmic ideas, the framework transitions to the Implementation and Empirical Evaluation stage. This phase bridges the abstraction gap between natural language reasoning and concrete, low-level quantum circuit manipulation. By materializing conceptual strategies into executable programs, the framework subjects the LLM's heuristic proposals to rigorous empirical validation.

Constrained Code Synthesis

The transition from text to code is governed by a strict interface contract designed to ensure compatibility with the underlying compilation backend. For each structured *Idea Block* generated in the previous stage, the LLM is prompted to produce a corresponding Python implementation. To guarantee that the resulting function can be seamlessly integrated into the evaluation pipeline, the prompt explicitly defines the expected method signature and details the available class attributes, such as the hardware coupling map, distance matrices, and the current logical-to-physical mapping state.

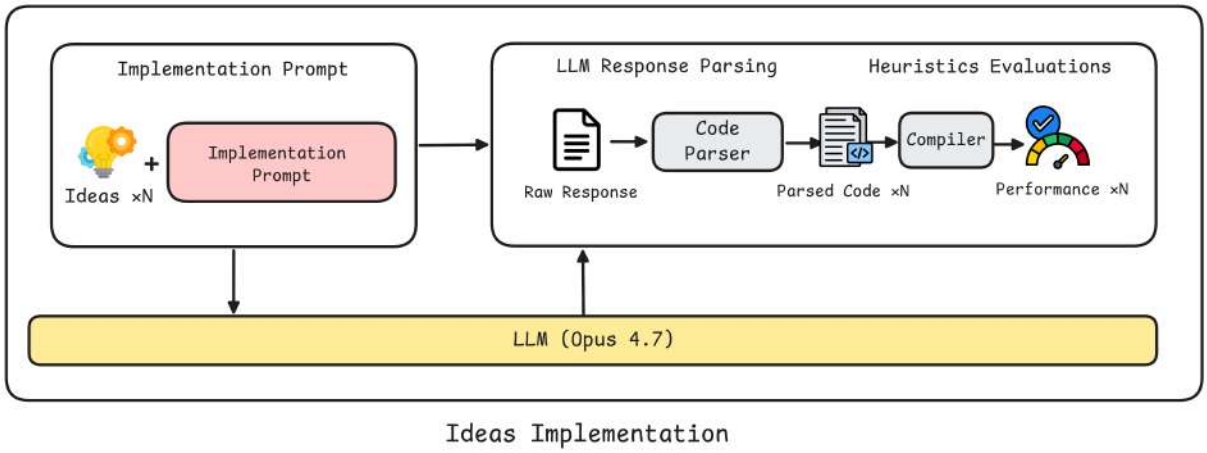


Figure 5.4: Internal structure of the implementation stage.

As illustrated in Figure 5.4, the raw output generated by the LLM is processed by a dedicated parsing module. This module utilizes regular expressions to isolate the executable Python block, successfully decoupling the programming logic from any surrounding conversational text.

Isolated Execution Environment

A practical challenge in Automatic Heuristic Design is the robust execution of dynamically generated code. Because each candidate heuristic is produced by the LLM, it may contain non-terminating loops, unbounded memory growth, or unhandled exceptions that would otherwise stall or crash the main Orchestrator. To contain such faults, QPilot evaluates every heuristic in a separate worker process spawned through Python's `multiprocessing` module.

Within this environment, each candidate heuristic is loaded with Python's `exec()` into a dedicated local namespace, and the worker process instantiates the `Qlosure` routing environment, binds the generated function as the routing method, and executes the routing protocol on a target quantum circuit under a fixed wall-clock timeout. Running each evaluation out of process provides *fault isolation*: a worker that crashes, hangs, or exceeds its time budget is terminated and recorded as a failed trial, leaving the Orchestrator and the remaining evaluations unaffected.

We stress that this mechanism is a robustness measure against faulty heuristics, not a security boundary. The combination of `multiprocessing` and `exec()` does not constitute a secure sandbox: a generated heuristic still runs with the full privileges of the host process and could, in principle, read or write the file system or open network connections. This is acceptable in our setting because the executed code originates from a trusted, controlled LLM rather than an adversary; hardening the evaluator into a genuine sandbox, for instance via an OS-level container or a seccomp-restricted subprocess, is left as a direction for future work.

Empirical Benchmarking and Fail-Fast Optimization

Once successfully loaded, each heuristic is benchmarked against a standardized suite of quantum circuits. The evaluation focuses on two primary topological compilation metrics:

- **Mean SWAP Overhead:** The average number of supplementary SWAP operations inserted to satisfy the physical connectivity constraints of the target QPU.
- **Mean Circuit Depth:** The average increase in the critical path length of the fully mapped and routed circuits.

Given the significant computational overhead associated with evaluating routing algorithms across multiple circuits, the evaluator implements a critical *fail-fast* optimization to preserve the system's compute budget. The framework continuously monitors the worker processes for runtime failures or logical validation errors. If a candidate heuristic fails to successfully compile three distinct circuits, the evaluation process is immediately aborted. The heuristic is subsequently assigned an infinite penalty score (representing a worst-case baseline) and categorized as a mathematically or logically invalid approach.

At the conclusion of this stage, all successfully executed heuristics are ranked primarily by their capacity to minimize SWAP overhead. The top-performing candidates are selected to form the initial "elite" population, effectively seeding the subsequent Hypothesis-Driven evolutionary loop. Concurrently, all execution outcomes ranging from state-of-the-art successes to catastrophic failures are permanently registered in the Memory Manager. This preserves a comprehensive dataset of empirical evidence that the framework leverages to guide and refine future evolutionary operations.

5.5.3 Knowledge Extraction and Hypothesis Generation Stage

Following the empirical evaluation of the generated heuristics, the framework must synthesize the outcomes to inform subsequent iterations. A critical limitation of early Large Language Model (LLM) based evolutionary algorithms is their reliance on unstructured natural-language reflection. To resolve this, QPilot introduces a Hypothesis-Driven Knowledge Graph (HD-KG), which operates as an automated scientific analyst, transforming raw code and performance metrics into discrete, testable algorithmic theories.

The Hypothesis-Driven Knowledge Graph (HD-KG)

The HD-KG stores what the framework has learned as a graph built from two kinds of entities: *Traits* and *Hypotheses*.

1. **Traits:** A trait is a single, reusable building block of a heuristic, such as one particular way of weighting the distance matrix. Formally, a trait is a tuple $T = (\text{id}, \text{label}, E)$, where E is a set of *exemplars*: the actual code snippets in which the trait was found. These snippets tie each abstract trait back to concrete, working code.

2. **Hypotheses:** A hypothesis is a testable claim that links one or more traits to an expected effect on performance. Formally, a hypothesis is a tuple $H = (\text{id}, S, \mathbf{T}, C, \phi)$, where S is the claim written in plain language, $\mathbf{T}$ is the set of traits it involves, $C \in [0, 1]$ is its confidence score, and $\phi \in \{\text{open}, \text{confident}, \text{falsified}, \text{exhausted}\}$ records whether the claim is still untested, supported, disproved, or exhausted.

Figure 5.5 shows an example HD-KG. Grey nodes are traits and coloured nodes are hypotheses, where the colour encodes the status ϕ and the label reports the confidence C. Each edge links a hypothesis to the traits $\mathbf{T}$ it refers to. Because supported and rejected claims stay in the same graph, the framework keeps a visible record of what has worked and what has not as it accumulates knowledge across iterations.

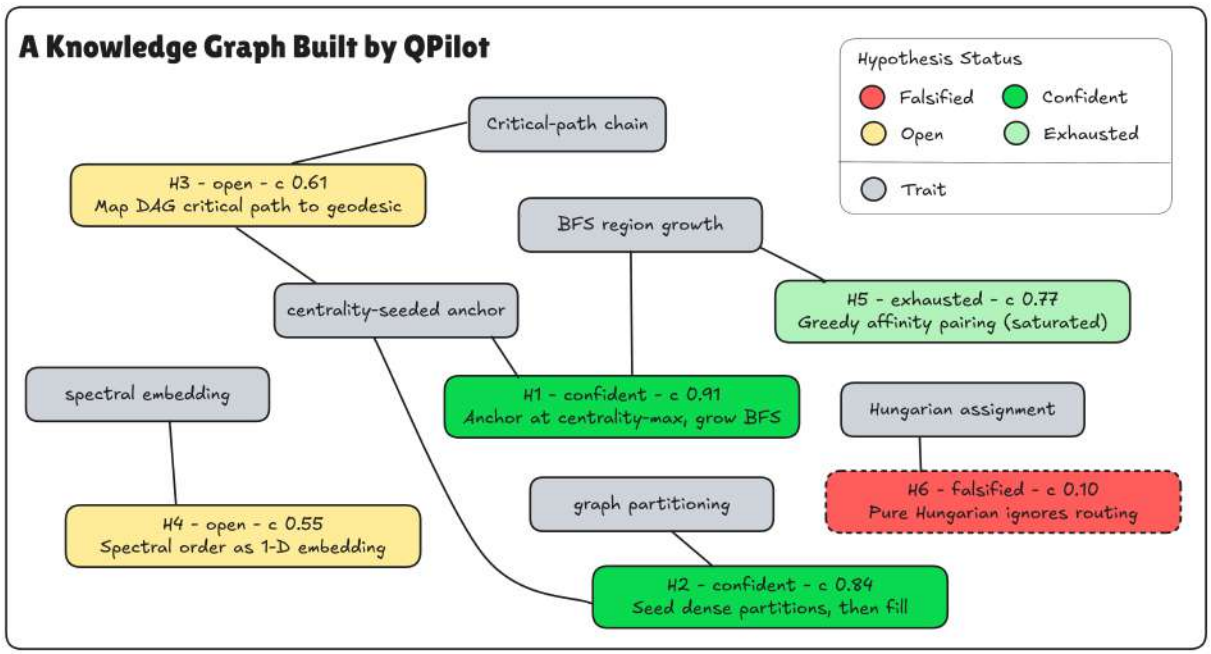


Figure 5.5: An illustrative Hypothesis-Driven Knowledge Graph.

Automated Trait Extraction

The first step in building the HD-KG is to find the traits. As shown in Figure 5.6, the framework starts by splitting the evaluated heuristics into two groups: the best-performing ones (the ”elite”) and the worst-performing ones (the ”failures”). The LLM then compares the two groups to spot the patterns that separate good heuristics from bad ones. By reading the code, it extracts the recurring traits. Each trait is stored together with the code snippets it came from, so the framework gradually builds a library of proven implementation patterns.

Hypothesis Formulation

Once the traits are known, the framework moves on to forming hypotheses, as shown in Figure 5.7. Here the LLM examines the traits and proposes claims about how they affect performance. Rather than vaguely asking to ”improve the code,” each claim is specific and testable, such as:

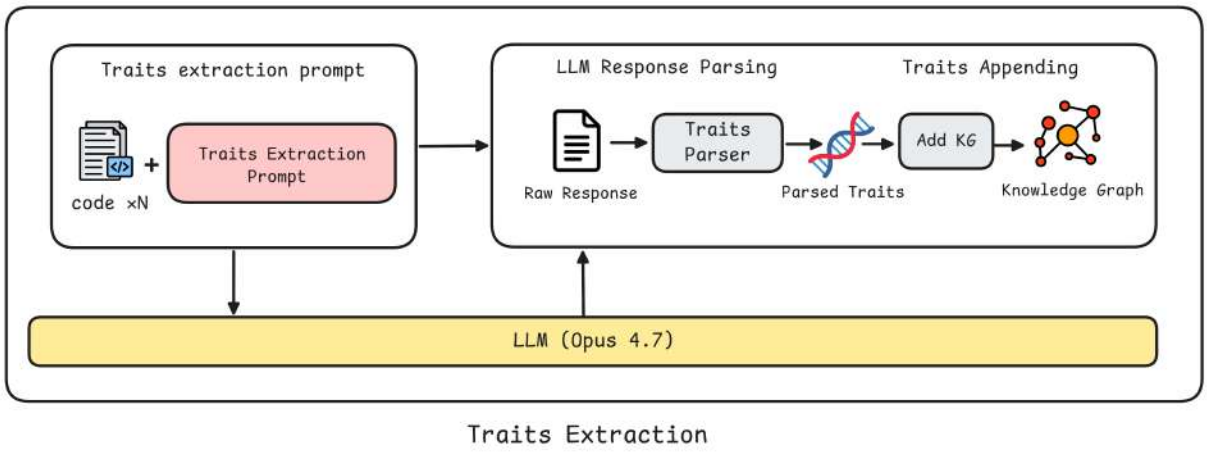


Figure 5.6: Internal workflow of the Trait Extraction step

"Combining Trait A with Trait B will reduce the mean SWAP overhead on heavy-hex topologies." Every new claim enters the HD-KG as $\phi =$ "open" with a neutral starting confidence ($C_0 = 0.5$). In this way, the reason behind a code change is recorded separately from the code itself, so that it can later be tested on its own.

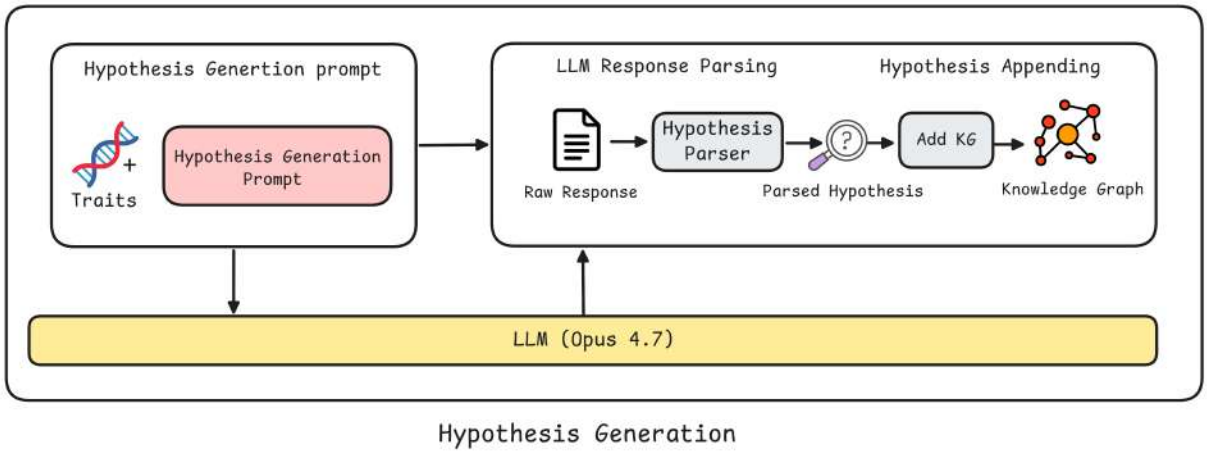


Figure 5.7: Internal workflow of the Hypothesis Generation phase

Empirical Confidence Updates

The intelligence of the HD-KG lies in its dynamic validation mechanism. Hypotheses are actively tested during the evolutionary refinement stage. Whenever an evolutionary operator generates an offspring conditioned on a specific hypothesis, the empirical performance of that offspring acts as a concrete "trial."

Each hypothesis is created with a neutral baseline confidence $C_0 = 0.5$. When a conditioned offspring is benchmarked, the confidence of the guiding hypothesis is updated by a reinforcement-style rule driven by the *normalized* change in mean SWAP overhead between the offspring and its parent. Let o_{par} and o_{off} denote the mean SWAP overhead of the parent and the offspring, respectively, and let $o^\star$ be the best (lowest) overhead recorded in the population

at the start of the current iteration. The normalized performance delta is

$$\Delta_{\text{perf}} = \frac{o_{\text{par}} - o_{\text{off}}}{o^{\star}}, \tag{5.1}$$

so that $\Delta_{\text{perf}} > 0$ when the offspring reduces SWAP overhead and $\Delta_{\text{perf}} < 0$ when it regresses. Dividing by $o^{\star}$ expresses the change as a fraction of the current best score, making the update scale-invariant across circuits of differing sizes; on the first iteration, where $o^{\star}$ is not yet defined, the denominator falls back to $\max(o_{\text{par}}, \varepsilon)$ with $\varepsilon = 10^{-9}$. The updated confidence after the n-th trial is

$$C_n = \text{clip}_{[0,1]}\big(C_{n-1} + \alpha\, \Delta_{\text{perf}}\big), \qquad \text{clip}_{[0,1]}(x) = \min\big(1,\ \max(0,\ x)\big), \tag{5.2}$$

where $\alpha = 0.2$ is the learning rate governing the sensitivity of the graph to new evidence, and the clipping keeps C_n a valid confidence in $[0, 1]$. A trial counts as an *improvement* only when the offspring strictly lowers the parent overhead ($\Delta_{\text{perf}} > 0$), which is the sole case that raises confidence; a tied trial ($o_{\text{par}} = o_{\text{off}}$) leaves the confidence unchanged.

Based on this update cycle, the validation status ϕ of the hypothesis transitions as follows:

- **Confident:** An *open* hypothesis is promoted once $C_n \geq \tau_{\text{conf}}$, with $\tau_{\text{conf}} = 0.75$. It is then treated as a robust optimization strategy and prioritized for exploiting the search space.
- **Falsified:** A hypothesis is falsified as soon as a conditioned offspring fails to produce a valid routing, that is, an execution error, a validation failure, or a fail-fast abort as defined in Section 5.5.2. Its confidence is then pinned to the falsification floor $\tau_{\text{fals}} = 0.1$, and the hypothesis is retained as a negative constraint that discourages the LLM from revisiting the strategy.
- **Exhausted:** A *confident* hypothesis is archived as exhausted once repeated testing yields no further gain, namely once (i) it has accumulated at least $2\,N_{\text{pop}}$ trials and (ii) at least N_{pop} consecutive trials have elapsed since its confidence last increased, where N_{pop} is the population size ($N_{\text{pop}} = 10$, giving a 20-trial minimum and a 10-trial stall window). Here "no further gain" denotes a run of trials in which no offspring achieved $\Delta_{\text{perf}} > 0$; rather than invoking a formal statistical test, the rule treats a hypothesis as saturated when it stops producing strictly improving offspring over a full population's worth of trials. Archiving frees the search to pursue unexplored theories while the statement is kept as a negative constraint for re-ideation.

5.5.4 Evolutionary Refinement and Re-Ideation Stage

The final stage of the QPilot framework operationalizes the knowledge captured within the Hypothesis-Driven Knowledge Graph (HD-KG). While standard genetic programming relies on stochastic permutations (e.g., random abstract syntax tree modifications) that often produce non-compiling or mathematically nonsensical code, QPilot frames evolutionary operators as intentional, theory-driven experiments. In this phase, the population of heuristics is iteratively improved through controlled crossover and mutation operators, interspersed with periodic re-ideation to maintain population diversity.

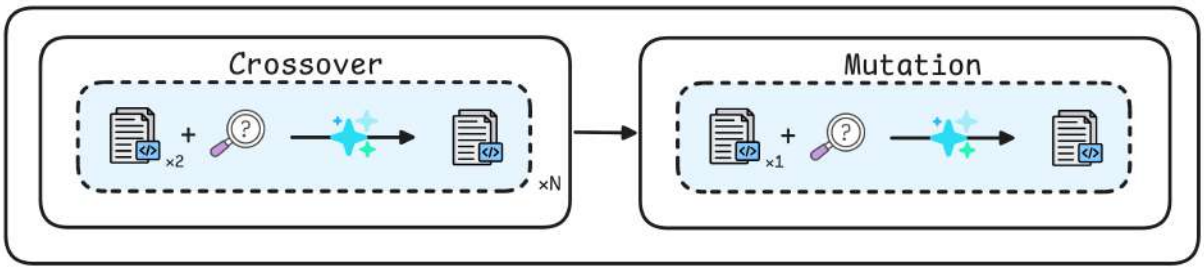


Figure 5.8: Internal workflow of the Evolutionary Refinement stage.

Hypothesis-Conditioned Crossover

The crossover operator in QPilot is designed to synthesize the strengths of distinct algorithmic lineages. During each iteration, pairs of heuristics are selected from the current population. Unlike traditional crossover, which blindly swaps code fragments, QPilot's crossover is *hypothesis-conditioned*. The LLM is provided with the source code of both parent heuristics alongside a specific, sampled hypothesis from the HD-KG (typically one with an *open* or *confident* status). The prompt instructs the LLM to write a new Python heuristic that explicitly embodies the causal claim of the hypothesis while integrating the functional advantages of both parents. This approach ensures that the resulting offspring represent a logical amalgamation of ideas rather than a fractured syntax tree.

Targeted Mutation

While crossover explores combinations of existing strategies, the mutation operator is designed for localized exploitation. In this step, the framework isolates the single best-performing heuristic in the population (the "elite" individual). The LLM is instructed to perturb the logic of this elite heuristic under the strict guidance of a high-confidence hypothesis from the HD-KG. By constraining the mutation to empirically verified algorithmic traits, such as aggressively penalizing gate congestion or refining look-ahead bounds, the framework ensures that the elite individual undergoes localized, monotonic improvements without losing its foundational efficacy.

Closing the Empirical Feedback Loop

Both the crossover and mutation operators yield a new generation of candidate heuristics. These offspring are immediately dispatched to the isolated execution environment (detailed in Section 5.5.2) for empirical benchmarking. The resulting performance metrics are compared against the scores of the parent heuristics. This performance delta is seamlessly fed back into the HD-KG to update the confidence scores of the hypotheses that guided the generation, as defined in Section 5.5.3. Consequently, every generated line of code acts as a verifiable experiment that either fortifies or dismantles the framework's internal algorithmic theories.

Re-Ideation and Stagnation Recovery

A prevalent challenge in evolutionary algorithms is premature convergence, where the population collapses into a local optimum, and operators fail to yield further improvements. To counteract this diversity loss, QPilot continuously monitors the iteration-over-iteration improvement rate. If the population exhibits signs of stagnation, the framework interrupts the standard evolutionary loop to trigger a *Re-Ideation* phase. The Orchestrator queries the LLM to generate entirely novel algorithmic seeds, explicitly utilizing the *falsified* and *exhausted* hypotheses from the HD-KG as negative constraints. This forces the LLM to explore orthogonal regions of the design space. The newly implemented strategies are benchmarked and injected into the active population alongside the historical elite. This mechanism successfully revitalizes the search process, enabling the framework to escape local minima while preserving the hard-won empirical knowledge accumulated in the HD-KG.

5.6 Implementation of the QPilot Framework

Following the architectural design discussed in Section 5.5, this section details the concrete realization of QPilot. We focus on the software stack, the nuances of the LLM-orchestration layer, and the specific configurations required for the experimental pipeline.

5.6.1 Problem Scope: Initial Qubit Mapping

The qubit mapping problem is conventionally split into two interdependent stages: *initial mapping* (pre-execution placement) and *routing* (dynamic SWAP insertion). While prior work on Qlosure (Benbetka et al. 2026) achieved state-of-the-art results on the routing problem, QPilot targets the complementary and under-explored initial mapping problem. Instead of modifying the routing logic, QPilot automates the synthesis of initial placement heuristics while keeping the mature Qlosure routing cost function as a fixed downstream backend. This strict separation of concerns ensures that any reduction in SWAP count or circuit depth is directly attributable to the quality of the generated placement, while allowing the optimization search to focus entirely on the mapping decisions previously left to manual design.

5.6.2 Libraries and Tools Used

The development and functionality of the QPilot framework rely on the following key external libraries and tools.

- **Python 3:** Python serves as the primary language for the framework. It provides the necessary infrastructure for complex data structure management and file handling.
- **Qlosure:** (Benbetka et al. 2026) A state-of-the-art framework designed for scalable quantum circuit compilation. Within QPilot, Qlosure serves a dual purpose: it is both the target for heuristic optimization and the primary evaluation environment. QPilot's LLM-driven engine specifically designs and injects novel mapping heuristics into the Qlosure

pipeline to improve mapping decisions. By executing these custom heuristics within the Qlosure infrastructure, the framework can directly evaluate their impact on circuit depth and gate overhead across diverse hardware topologies.

- **Qiskit:** (IBM 2024) As a leading open-source quantum computing framework, Qiskit is leveraged within the Qlosure module to interface with backend hardware specifications.
- **Claude Code:** This interface provides access to the Claude family of Large Language Models (LLMs), serving as the core reasoning engine for the framework. It is responsible for interpreting natural language prompts and autonomously designing the optimization heuristics.
- **LLM4AD Framework:** (Fei Liu et al. 2024b) A specialized platform designed for automated Algorithm Design via Large Language Models. This framework served as the architectural foundation for benchmarking state-of-the-art LLM-based heuristic generation methods.

5.6.3 Large Language Model Selection

The efficacy of the QPilot framework, specifically the generation and reflection phases of the Automated Heuristic Design (AHD) loop, is tied to the underlying Large Language Model's (LLM) reasoning depth and architectural proficiency. To identify the optimal engine for our framework, we conducted a comparative analysis of several state-of-the-art models: **Claude 4.7 Opus** (Anthropic 2026a), **Claude 4.6 Sonnet** (Anthropic 2026b), **GPT-5.2** (OpenAI 2025), **DeepSeek V4 Pro Max** (DeepSeek-AI 2026), and **Gemini 3.1 Pro** (Google DeepMind 2025).

The selection process was guided by two primary criteria: *code generation quality* and *multi-step logical reasoning*. We utilized two high-signal benchmarks to quantify these capabilities:

- **SWE-bench Verified:** A benchmark suite designed to assess an LLM's capacity to resolve real-world software engineering issues. High performance on this suite indicates a model's ability to maintain syntactic integrity while navigating complex codebase dependencies and logical requirements (Jimenez et al. 2024).
- **Humanity's Last Exam:** To assess abstract problem-solving and cognitive flexibility, we evaluated models against this general reasoning benchmark. It tests limits across mathematics, logic, and common sense-cognitive skills that are indispensable for synthesizing novel heuristics in specialized domains such as quantum computing (Phan et al. 2025).

According to benchmark results reported by the model providers, **Claude 4.7 Opus** was selected as the primary driver for the QPilot framework. While GPT-5.2 and Gemini 3.1 Pro showed competitive performance in general reasoning, Claude 4.7 Opus promises a superior ability to bridge the gap between high-level heuristic conceptualization and functional code implementation. Its higher pass rate on the SWE-bench Verified makes it uniquely suited for the iterative requirements of the AHD loop.

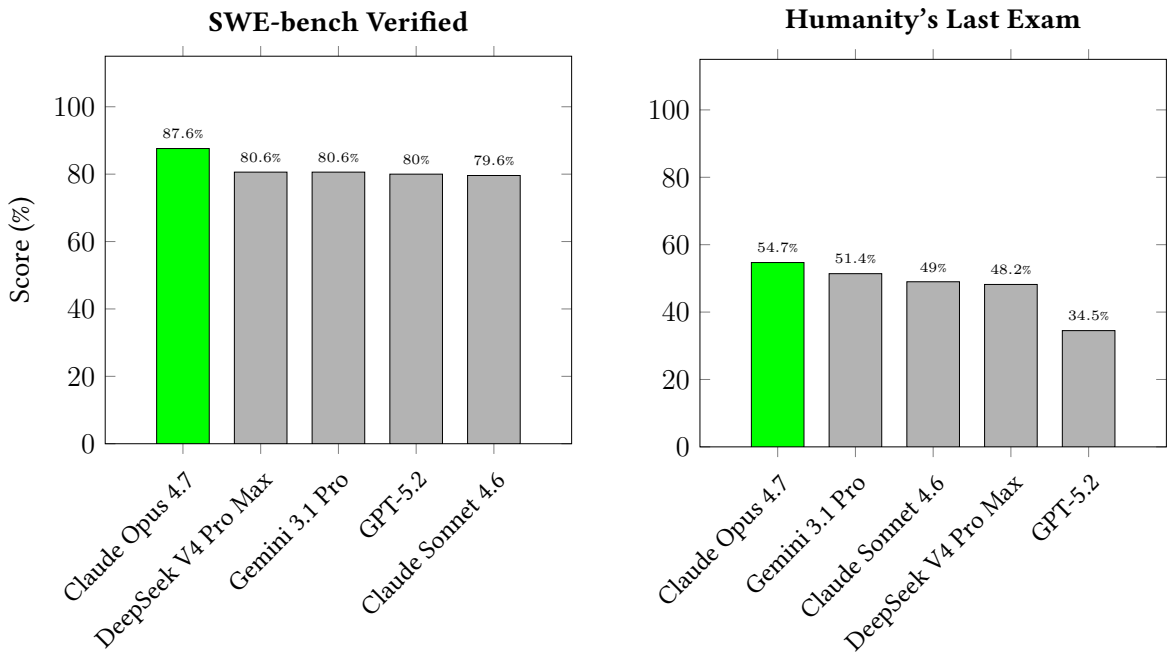


Figure 5.9: Performance of LLMs on the SWE-bench and Humanity's Last Exam benchmarks.

5.6.4 Anthropic API Integration

The interaction between the Orchestrator and the Claude Opus 4.7 model (Anthropic 2026a) is mediated by a dedicated *LLM Client* (Section 5.3) that encapsulates the communication lifecycle. Rather than binding the pipeline to the raw Anthropic REST endpoints, QPilot routes every request through the Claude Code command-line interface invoked in a *headless* fashion, under the invocation `claude --model claude-opus-4-7 -p --output-format json`. The print flag (`-p`) forces a non-interactive, single-shot completion, while `--output-format json` wraps the reply in a structured envelope from which the client deterministically recovers the generated text. This design inherits the authentication and model-routing facilities of the official tooling while exposing a single, model-agnostic entry point. The client operates in either a *conversational* mode that serializes the dialogue history into successive `User`/`Assistant` turns, or a stateless mode that resets the context per call. Since the implementation and evolution stages dispatch many candidates concurrently (`implementation_workers`, `evolution_workers`), the client is thread-safe: the subprocess call runs outside any lock, so requests proceed truly in parallel, and only the subsequent state mutations are serialized behind a mutex.

To quantify the computational and financial cost of the search, the LLM Client maintains a transparent token-accounting scheme. For every call, consumption is measured as the whitespace-delimited word count of the prompt and completion combined; this proxy is preferred over the provider-reported billing fields because it remains stable across CLI versions and directly comparable across all benchmarked AHD methods. Each call updates a running `usage_totals` accumulator and is snapshotted into a per-call `call_log` entry recording its timestamp, originating pipeline stage (ideation, implementation, crossover, mutation, or re-ideation), per-call cost, and cumulative total at commit time. These counters are joined with the evaluation trace so that cumulative token expenditure is aligned with each heuristic's mea-

sured SWAP and depth metrics, yielding the efficiency comparison reported in Chapter 6.

5.6.5 LLM4AD Framework Adaptation

To ensure a fair and rigorous comparison between QPilot and existing state-of-the-art LLM-based Automated Heuristic Design (LLM-AHD) methods, we utilized the LLM4AD framework (Fei Liu et al. 2024b) as our primary benchmarking platform. Originally tailored as a general-purpose platform for algorithm design via LLMs, LLM4AD required specific adaptations to support the demands of our quantum mapping experiments. We implemented the following key modifications to the framework:

- **Integration of the Claude CLI:** We modified the LLM4AD execution engine to interact with the Claude CLI natively, mirroring the asynchronous and economically efficient token-consumption strategy developed for QPilot.
- **Quantum Mapping Task Formulation:** We explicitly formulated the initial qubit mapping task within the LLM4AD environment. This involved developing and injecting custom prompt templates tailored to the quantum domain, as well as integrating the Qlosure-based evaluation backend. By establishing these problem-specific boundaries, the SOTA methods could synthesize mapping heuristics evaluated under the same strict routing constraints as QPilot.
- **Comprehensive Logging and Telemetry System:** We introduced a persistent telemetry and logging module to capture the full lifecycle of each experiment. This system was engineered to meticulously track all generated heuristics, their corresponding evaluation results (e.g., SWAP counts and circuit depth metrics), and the exact token consumption per generation block. These artifacts provided the vital data footprint necessary to analyze not only the algorithmic performance of the generated heuristics but also the computational and financial efficiency of each benchmarked AHD method.

5.6.6 QPilot Configuration Parameters

A QPilot run is governed by a compact set of configuration parameters that control the search budget, the scale of the candidate population, the behavior of the evolutionary operators, and the dynamics of the HD-KG. We describe here the parameters that materially shape the search; their values, together with the operator-specific hyperparameters of the baseline methods, are reported in Table 5.2 and are applied identically across all benchmarked frameworks to ensure a transparent and standardized comparison.

- **Search budget (`max_time_seconds`, `max_fe`).** Each run is bounded jointly by a wall-clock limit and a function-evaluation (FE) cap, the search halting on whichever is reached first. We fix the wall-clock budget to 7200 s (2 h) so that every method is afforded an identical exploration window. A per-heuristic evaluation timeout of 360 s further bounds the cost of any single candidate, which, combined with the fail-fast policy of Section 5.5.2, prevents pathological heuristics from monopolizing the budget.

- **Population and ideation scale (`pop_size`, `top_ideas_to_implement`).** The Knowledge-Grounded Ideation stage produces ten candidate strategies, all of which are implemented and benchmarked; the `pop_size` $= 10$ best-performing heuristics then seed the evolutionary loop and define the elite carried across generations.
- **Evolutionary operators (`crossover_count`).** Each evolutionary iteration generates `crossover_count` hypothesis-conditioned offspring through crossover, together with a single targeted mutation of the elite individual, so that one iteration debits three evaluations from the FE budget.
- **HD-KG dynamics (`kg_alpha`, `kg_confidence_threshold`, `kg_open_sample_prob`).** The reinforcement-style confidence update of Section 5.5.3 uses a learning rate $\alpha = 0.2$ (`kg_alpha`), trading responsiveness to new evidence against the stability of accumulated theories. A hypothesis is promoted to the *confident* status once its confidence reaches $\tau_{\text{conf}} = 0.75$ (`kg_confidence_threshold`). When an operator samples a guiding hypothesis, it draws an *open* (exploratory) claim with probability 0.3 (`kg_open_sample_prob`) and otherwise exploits a *confident* one, balancing the validation of new theories against the reuse of verified strategies.

To evaluate the efficacy of the QPilot engine, we compare its performance against existing LLM-AHD methods. The operational configurations and architectural parameters for these comparative AHD frameworks are summarized in the table below 5.2 to ensure a transparent and standardized benchmark.

Table 5.2: Description of experimental settings

Description of Setting	Value
LLM (generator and reflector)	Claude-Opus-4.7
LLM temperature (generator and reflector)	1
Run time	7200 sec
Maximum evaluation time for each heuristic	360 sec
# of samplers and # of evaluators	1, 1
Population size (for EoH, ReEvo, MCTS-AHD, and PartEvo)	10
Max generations (for EoH, ReEvo, and PartEvo)	15
Mutation rate (for ReEvo)	0.5
Alpha and Lambda (for MCTS-AHD)	0.5, 0.1
# of islands, # of samples per prompt (for FunSearch)	10, 1
Maximum # of initial sample, partition method, and partition number (for PartEvo)	10, kmeans, 4

5.7 Conclusion

This chapter introduced QPilot, an LLM-based Automated Heuristic Design framework that automates the design of initial qubit mapping heuristics. Starting from the two gaps identified in

Chapters 2 and 4, the unexplored application of LLM-AHD to qubit mapping and the four recurring limitations of existing frameworks, we derived a set of design choices and realized them as a modular, orchestrated architecture. The resulting closed-loop pipeline couples knowledge-grounded ideation, sandboxed empirical evaluation against the Qlosure routing backend, a Hypothesis-Driven Knowledge Graph that records discrete traits and falsifiable causal claims, and hypothesis-conditioned evolutionary operators interleaved with stagnation-triggered re-ideation, so that every generated heuristic doubles as a controlled test whose outcome refines the framework's accumulated knowledge.

Beyond the conceptual design, we detailed the concrete implementation that makes these choices operational: the restriction of scope to the initial mapping behind a fixed state-of-the-art routing backend, the software stack and Claude-CLI integration layer, the model-selection rationale, and the configuration parameters applied identically across all benchmarked methods. With the framework and its implementation now fully specified, the next chapter subjects the heuristics it discovers to a systematic empirical evaluation against both production quantum compilers and competing LLM-AHD frameworks (Chapter 6).

Chapter 6

Evaluation

6.1 Introduction

Chapter 5 introduced **QPilot**, an LLM-based Automated Heuristic Design framework for qubit mapping, and detailed the architecture, search pipeline, and concrete implementation through which it synthesizes initial mapping heuristics. This chapter subjects that framework to a systematic empirical evaluation, organized around two questions: whether the heuristics QPilot discovers are competitive with the mapping routines embedded in production quantum compilers, and whether its Hypothesis-Driven Knowledge Graph yields better heuristics than alternative LLM-driven AHD strategies operating under an equivalent search budget.

Section 6.2 establishes the experimental setup, defining the quality and cost metrics, the two families of baselines, the synthetic and application-oriented benchmark suites, the target hardware architectures, and the computational environment. Section 6.3 then presents the results and their analysis: it first traces how a single search run converges on the heuristic carried forward into the evaluation (Section 6.3.1), before assessing that heuristic on the sparse heavy-hexagon lattice of IBM Sherbrooke (Section 6.3.2) and the dense square lattice of Rigetti Ankaa-3 (Section 6.3.3), in each case against both production compilers and competing AHD frameworks. The remaining subsections quantify the per-circuit mapping time of the discovered heuristic and the token cost of the search (Section 6.3.5), and close with an ablation study that isolates the contribution of the Hypothesis-Driven Knowledge Graph (Section 6.3.6).

6.2 Experimental Setup

To evaluate the performance and efficacy of the QPilot framework and the heuristics it generates, a comprehensive experimental setup was designed. This section details the evaluation metrics, the baseline methods used for comparison, the benchmark circuits, the target quantum hardware architectures, the computational environment, and the results.

6.2.1 Evaluation Metrics

Assessing an Automatic Heuristic Design framework requires capturing two complementary dimensions: the *quality* of the initial mapping heuristics that QPilot discovers, and the *cost* of the automatic search process that produces them. The quality dimension is measured on the mapped circuits produced when a generated heuristic is applied to a benchmark, while the cost dimension reflects the resources consumed by the LLM-driven evolutionary loop. To quantify both, the following metrics were employed:

- **SWAP Gate Count**: The total number of SWAP gates inserted by a generated heuristic to make a logical circuit compliant with the connectivity of the target device. Since each SWAP gate decomposes into three CNOT gates and contributes substantially to the accumulated error of the circuit, a lower SWAP count is a primary indicator of mapping quality.
- **Final Circuit Depth**: The length of the critical path of the mapped circuit, representing the minimum number of sequential time steps required for execution under parallel gate scheduling. A lower depth shortens the exposure of qubits to decoherence and is therefore preferable.
- **Compilation Time**: The time required to apply a generated heuristic and route a single benchmark circuit onto the target topology. This metric characterizes the practical efficiency and scalability of the discovered heuristic, independent of the time spent searching for it.
- **Average Improvement in Depth and SWAPs**: To enable a fair, scale-independent comparison across benchmark suites of heterogeneous sizes, we report the mean relative improvement of a heuristic over a reference method, averaged across a benchmark set $\mathcal{B}$. For the SWAP count, it is defined as

  $$\Delta_{\text{SWAP}} = \frac{1}{|\mathcal{B}|} \sum_{c \in \mathcal{B}} \frac{S_{\text{ref}}(c) - S_{\text{QPilot}}(c)}{S_{\text{ref}}(c)} \times 100\%,$$

  and analogously for circuit depth,

  $$\Delta_{\text{Depth}} = \frac{1}{|\mathcal{B}|} \sum_{c \in \mathcal{B}} \frac{D_{\text{ref}}(c) - D_{\text{QPilot}}(c)}{D_{\text{ref}}(c)} \times 100\%,$$

  where $S(c)$ and $D(c)$ denote the SWAP count and depth obtained on circuit c. Positive values indicate that the QPilot-generated heuristic improves upon the reference method, with larger values reflecting stronger and more consistent gains across the suite.
- **Token Consumption**: The cumulative number of tokens exchanged with the Large Language Model over the course of a complete search run, aggregating prompt and completion tokens across all ideation, implementation, reflection, and evolutionary queries.

6.2.2 Baselines for Comparison

To position QPilot within the existing landscape, its generated heuristics are evaluated against two distinct families of baselines, each addressing a different research question.

Established quantum compiler mapping methods. The first family answers whether the heuristics discovered by QPilot are competitive with the initial mapping routines embedded in production quantum compilers. These tools represent the current state of the practice and are routinely used in real compilation pipelines:

- **`SABRE` (LightSABRE in Qiskit):** The default routing pass of IBM's Qiskit transpiler stack (IBM 2024). We adopt the enhanced LightSABRE variant (Zou et al. 2024), which refines the original SABRE algorithm (Gushu, Ding, and Xie 2019) through a lighter-weight cost model and improved layout selection, and which remains actively maintained and widely deployed.
- **`MQT QMAP` (Munich Quantum Toolkit):** The mapping component of the MQT framework developed at the Technical University of Munich (Wille and Burgholzer 2023). QMAP is a robust, extensible compiler backend frequently used as a reference in academic research on quantum circuit mapping.
- **`Pytket` (Quantinuum's `tket`):** The Python interface to the `tket` compiler developed by Quantinuum (Sivarajah et al. 2020), widely used in both academic and industrial settings for hardware-aware circuit optimization and routing.
- **`Google Cirq`:** A quantum programming framework developed by Google, which includes routing and transformation passes and underpins Google's quantum software stack.

LLM-based AHD methods. The second family answers whether the design choices of QPilot, in particular its Hypothesis-Driven Knowledge Graph, yield better heuristics than alternative LLM-driven Automatic Heuristic Design strategies. Each of these frameworks, presented in Chapter 4, is configured to operate on the same qubit-mapping task, with the same LLM and an equivalent search budget, so that the comparison isolates the contribution of the search methodology itself:

- **Randsample** (Fei Liu et al. 2024b): Serves as a naive generation baseline that bypasses evolutionary or reflective feedback mechanisms.
- **FunSearch** (Romera-Paredes et al. 2024): Couples an LLM with a programs database and an island model, performing recombination directly in function space.
- **EoH (Evolution of Heuristics)** (Fei Liu et al. 2024a): Evolves a dual representation in which each heuristic is encoded jointly as a natural-language thought and a code implementation.
- **ReEvo** (H. Ye et al. 2024): Replaces numerical fitness signals with verbal gradients, maintaining short-term and long-term reflections that are recycled as context across queries.
- **MCTS-AHD** (Z. Zheng et al. 2025): Organises the search as a Monte Carlo Tree Search over heuristics, using progressive widening and exploration decay to balance discovery and refinement.

- **PartEvo** (Hu and Q. Zhang 2026): Addresses diversity collapse through explicit niching by clustering heuristics based on structural and semantic features before applying targeted evolutionary operators.

6.2.3 Benchmark Circuits

A diverse set of benchmark circuits was used to expose the generated heuristics to a broad range of sizes, structures, and interaction patterns. Two complementary suites were selected: one synthetic and one drawn from real quantum applications.

QUEKO Benchmarks: QUEKO (**QU**antum **E**xamples with **K**nown **O**ptimal) is a synthetic benchmark suite designed specifically to assess the optimality of qubit-mapping tools (Tan and J. Cong 2020). Each QUEKO instance is constructed so that a mapping achieving a known optimal depth on a given coupling graph provably exists, obtained by embedding a critical path of target depth into the device topology and then obfuscating it through a random permutation of the qubit labels. The inverse of this permutation constitutes the known optimal mapping, against which any candidate heuristic can be measured. This property makes QUEKO particularly well-suited for quantifying the overhead introduced by a mapping heuristic, since deviations from the optimum are attributable to the heuristic alone rather than to an unknown ground truth.

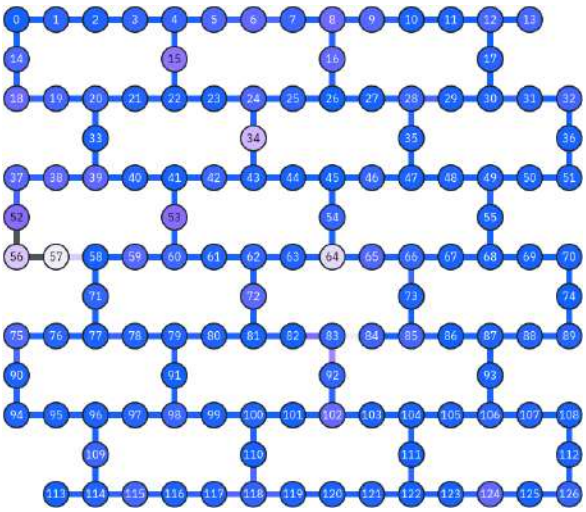
For this study, three QUEKO instance sets of increasing scale were employed: **queko-bss-16 qbt**, **queko-bss-54qbt**, and **queko-bss-81qbt**, comprising circuits of 16, 54, and 81 qubits, respectively. Within each set, the target circuit depth ranges from 100 to 900 cycles in steps of 100, and for every depth value, 10 distinct circuit instances are provided. This yields 90 circuits per set, ensuring both *depth diversity*, which probes how the generated heuristics behave under shallow and deep temporal constraints, and *structural variability*, which provides several randomized gate layouts per depth and thus supports a statistically meaningful estimate of average performance.

QASMBench: QASMBench (A. Li et al. 2023) is a comprehensive suite of quantum circuit benchmarks distributed in OpenQASM 2.0 format, derived from a wide range of quantum algorithms and application domains, including quantum chemistry, arithmetic, reversible logic, and oracle-based constructions. Unlike the synthetic QUEKO instances, QASMBench circuits reflect the gate-level structure and interaction patterns of realistic quantum kernels, and therefore complement QUEKO by testing the generality of the discovered heuristics beyond constructed worst cases. Two of its size-stratified categories were used in the evaluation: the **medium** category, containing 19 circuits of moderate qubit count and depth, and the **large** category, containing the most demanding application circuits in the suite (22 circuits). Circuits requiring more qubits than available on the target processors were excluded, leaving the subsets of each category that are mappable onto the architectures described below.

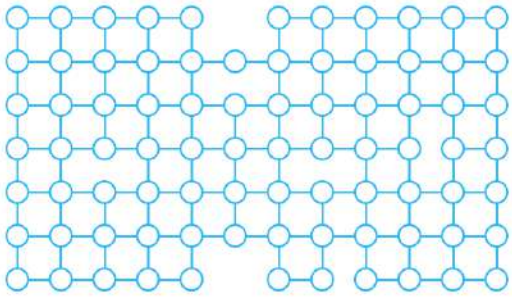
6.2.4 Target Quantum Hardware Architectures

The performance of a qubit-mapping heuristic is inseparable from the connectivity of the device it targets: sparser topologies force longer SWAP chains and amplify the effect of mapping decisions. To evaluate the generated heuristics under different connectivity regimes, two real superconducting architectures with contrasting lattice structures were modeled:

- **IBM Sherbrooke**: A 127-qubit superconducting processor built on a heavy-hexagon lattice (IBM 2025). The heavy-hexagon topology is comparatively sparse, with most qubits having degree two or three, which limits mapping flexibility and makes the placement of SWAP operations especially consequential. Its coupling graph is shown in Figure 6.1a.
- **Rigetti Ankaa-3**: An 84-qubit superconducting processor organized as a square-lattice of qubits linked by tunable couplers (Rigetti 2025). Its grid-like connectivity offers a higher average qubit degree than the heavy-hexagon lattice, providing a markedly different mapping environment and thereby testing the adaptability of the generated heuristics across topology families. Its coupling graph is shown in Figure 6.1b.



(a) IBM Sherbrooke.



(b) Rigetti Ankaa-3.

Figure 6.1: Coupling graphs of the target hardware architectures used in the evaluation.

6.2.5 Experimental Environment

The complete pipeline, covering both the QPilot search runs and the subsequent evaluation of the generated heuristics, was executed on the high-performance computing (HPC) facility of New York University Abu Dhabi (NYUAD), with jobs scheduled through the SLURM workload manager. Each allocated node was powered by an **AMD EPYC 7742 32-core processor** and provisioned with **64 GB of RAM**, running under CentOS 8.2. To keep runtimes within practical bounds, every mapping run was capped by a **24-hour wall-clock timeout** any method exceeding this limit on a given instance was recorded as a timeout rather than being allowed to run indefinitely.

6.2.6 Experimental Protocol

A **single QPilot search run** is performed, targeting the **IBM Sherbrooke** heavy-hexagon topology and scoring every candidate heuristic on the **QASMBench *Large*** suite through the Qlosure (Benbetka et al. 2026) routing backend, with mean SWAP count as the objective (Section 5.5.2) all empirical feedback that drives ideation, evolutionary refinement, and the Hypothesis-Driven Knowledge Graph is computed on this backend–benchmark pair, and the run is traced in Section 6.3.1. This pair, IBM Sherbrooke together with QASMBench *Large*, is the **only data and topology seen during the search**. The best heuristic produced by the search is then **fixed and evaluated** on both backends, IBM Sherbrooke (Section 6.3.2) and Rigetti Ankaa-3 (Section 6.3.3), across the QUEKO *16*, *54*, and *81*-qubit suites and the QASMBench *Large and Medium* categories, against both production compilers and alternative LLM-based AHD frameworks. Crucially, **no separate search is run per backend**: rather than re-searching a dedicated heuristic for Ankaa-3, we deploy the Sherbrooke optimized heuristic on it. The same heuristic is thus used throughout, so its evaluation on Ankaa-3 and on the QUEKO suites measures how a heuristic optimized on a single sparse topology and a single application benchmark generalizes to an **unseen, denser lattice** and to **unseen circuit families**.

6.3 Results and Analysis

6.3.1 Search Progress and Convergence

We first examine how a single search run converges on the heuristic that the rest of this section evaluates. As detailed in the experimental protocol (Section 6.2.6), this is the only search performed: it targets the IBM Sherbrooke topology and uses the QASMBench *Large* suite as the search objective, and the heuristic it selects is frozen and reused unchanged on both backends and all benchmarks. Figure 6.2 traces a complete QPilot run on the IBM Sherbrooke target, plotting the average SWAP count of every generated heuristic against the cumulative LLM tokens consumed. The 2-hour run produces 43 heuristics within a 142k-token budget (analyzed in Section 6.3.5), each colored by its originating stage: ideation heuristics and refinement heuristics from the Evolutionary Refinement, the two stages introduced in Chapter 5.

The descent is non-monotonic: the sharpest upward jumps coincide with each ideation phase, the intended effect of re-ideation injecting novel, untested directions (Section 5.5.4), into which refinement then descends through HD-KG-conditioned crossover and mutation. Ideation heuristics sit higher and cost more tokens, while refinement heuristics cluster lower, and the best-so-far envelope shows that durable improvements are crystallized during refinement rather than at ideation. The global-best heuristic (489.27 average SWAPs) emerges inside the third refinement phase after roughly 98k tokens, well before the time budget is exhausted, and is retained as the elite even as later phases keep exploring. This is the heuristic carried forward into the evaluation, which we begin on the IBM Sherbrooke.

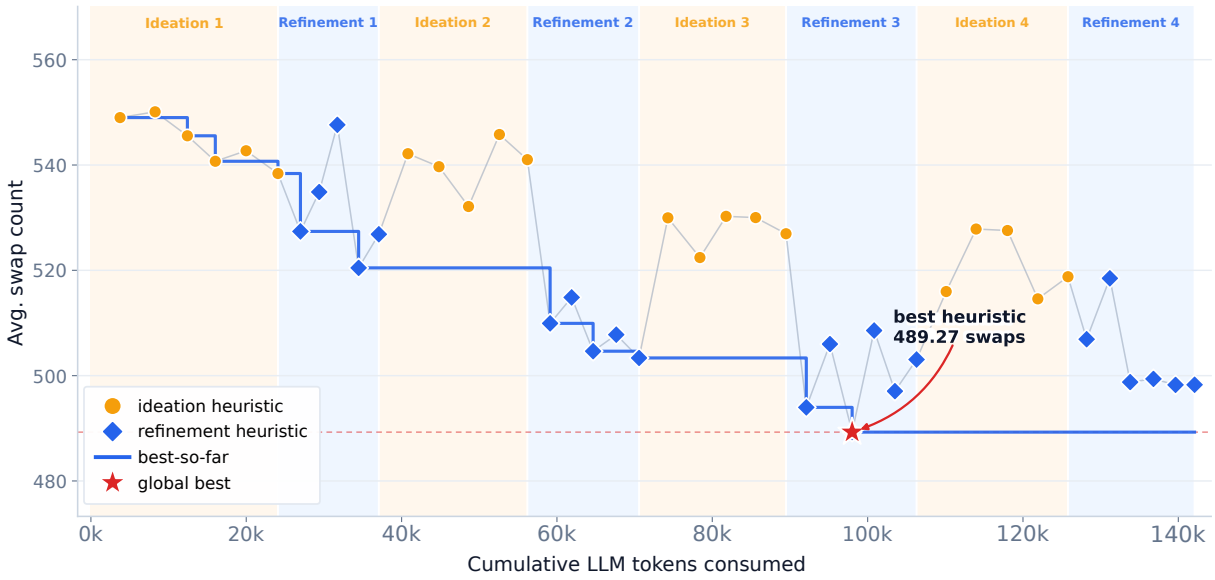


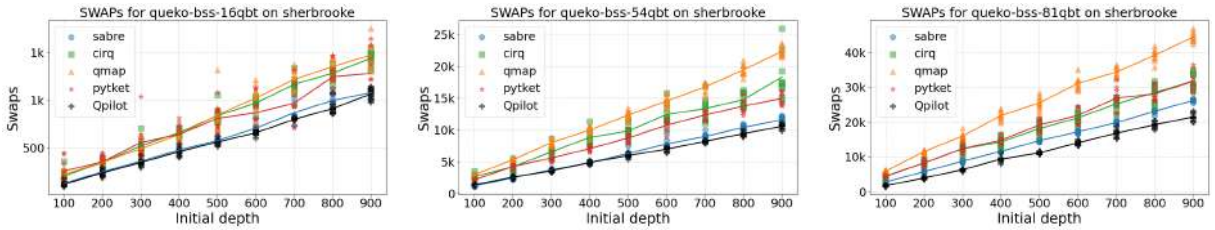
Figure 6.2: Search trajectory of a complete QPilot run on the IBM Sherbrooke backend.

6.3.2 Evaluation on IBM Sherbrooke backend

In this section, we evaluate the quality of the initial qubit mapping heuristics discovered by QPilot when targeting the IBM Sherbrooke architecture. This is the topology on which the heuristic was searched (Section 6.2.6), so the results below measure its performance on the backend it was optimized for. The heavy-hexagon lattice of the Sherbrooke processor poses a challenging mapping setting due to its low qubit connectivity. We first compare the QPilot best generated heuristic against the mapping passes of established production compilers (SABRE, QMAP, Pytket, and Cirq) to determine its competitiveness in a state-of-the-art context.

Comparison with State-of-the-Art Compilers

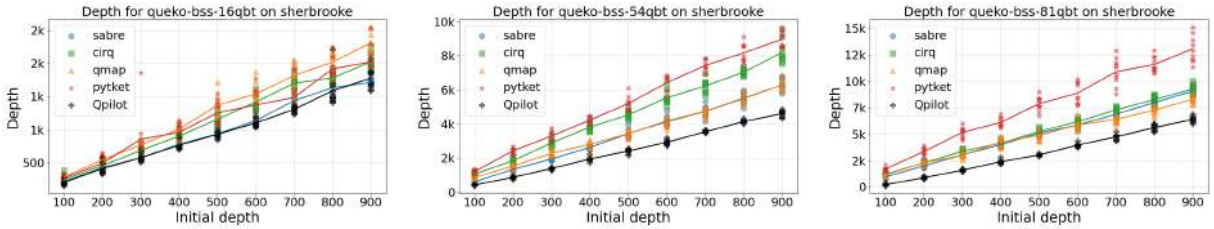
Performance on QUEKO Benchmarks. To observe how the generated heuristics scale with both circuit width (number of qubits) and initial circuit depth, we evaluated the methods across the 16 qubit, 54 qubit, and 81 qubit QUEKO benchmark sets. The results for SWAP gate counts and final circuit depths are visualized in Figure 6.3 and Figure 6.4, respectively.



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

Figure 6.3: SWAP gate counts on the IBM Sherbrooke backend across QUEKO benchmarks.

As shown in Figure 6.3, the routing overhead grows linearly with the initial circuit depth for all tested methods. However, the slope of this growth varies significantly. For smaller



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

Figure 6.4: Final circuit depths on the IBM Sherbrooke backend across QUEKO benchmarks.

instances (16 qubits), QPilot and SABRE exhibit similar baseline behaviors, though QPilot maintains an advantage. As the problem dimension scales to 54 and 81 qubits, the combinatorial space of possible mappings expands exponentially, and the performance gap between the methods becomes more pronounced. On the 81 qubit set, QPilot scales favorably; its synthesized mapping strategy achieves flatter growth curves for SWAP insertions compared to Pytket and QMAP, which struggle to contain the overhead at higher initial depths (e.g., depths 700 to 900).

A parallel trend is evident in the final circuit depth (Figure 6.4). Because SWAP gates effectively block the execution of subsequent logical operations, minimizing SWAP counts often correlates with shallower mapped circuits. QPilot yields the lowest final circuit depth across the three scales considered. By learning to prioritize initial mapping that does not disrupt the critical path of the circuit, the QPilot framework generates a heuristic that, on average, outperforms the manually engineered strategies embedded in state-of-the-art compilers on these benchmarks.

Performance on Application-Oriented QASMBench. While the QUEKO suite evaluates algorithms against synthetic structures, production compilers must also be tested against realistic algorithmic patterns. Table 6.1 summarizes the average absolute SWAP counts and final depths obtained on the *Large* category of the QASMBench suite, alongside the relative improvement (Δ_{SWAP} and Δ_{Depth}) achieved by QPilot over each baseline.

Table 6.1: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the IBM Sherbrooke backend (QPilot vs Production Compilers).

Method	**Absolute Performance**		**QPilot Improvement**	
	Average SWAPs	**Average Depth**	Δ_{SWAP} **(%)**	Δ_{Depth} **(%)**
Cirq	646.72	1032.36	24.35%	12.59%
Pytket	587.95	1028.77	16.78%	12.28%
SABRE	577.77	1066.91	15.32%	15.42%
QMAP	507.41	925.82	3.57%	2.53%
QPilot	**489.27**	**902.41**	–	–

QPilot produces the highest-quality mappings among the production compilers, achieving the lowest average SWAP count (489.27) and final depth (902.41). The largest margin of victory

is observed against Google Cirq, where the QPilot heuristic eliminates roughly a third of the inserted SWAPs ($\Delta_{\text{SWAP}} = 24.35\%$) and compresses the circuit depth by (12.59%). Against highly optimized heuristic passes like Pytket and SABRE, QPilot sustains double-digit gains, cutting SWAP counts by 16.78% and 15.32% and circuit depth by 12.28% and 15.42%, respectively.

MQT QMAP emerges as the strongest classical baseline on this suite, and QPilot's lead over it is correspondingly the slimmest: QPilot routes circuits with 3.57% fewer SWAPs and a 2.53% shorter depth. Although this margin is narrow, QPilot still occupies the top of the ranking on both metrics, outperforming every production compiler simultaneously. This confirms that the LLM-driven AHD loop discovers advanced initial mapping strategies that generalize from the search phase to unseen application topologies, even against a highly tuned mapper such as QMAP.

Table 6.2: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the IBM Sherbrooke backend (QPilot vs Production Compilers).

Method	**Absolute Performance**		**QPilot Improvement**	
	Average SWAPs	**Average Depth**	Δ_{SWAP} **(%)**	Δ_{Depth} **(%)**
Cirq	100.15	306.63	0.32%	7.31%
QMAP	92.05	299.84	-8.46%	5.21%
Pytket	139.73	308.26	28.55%	7.80%
SABRE	98.84	291.95	-1.01%	2.65%
QPilot	**99.84**	**284.21**	–	–

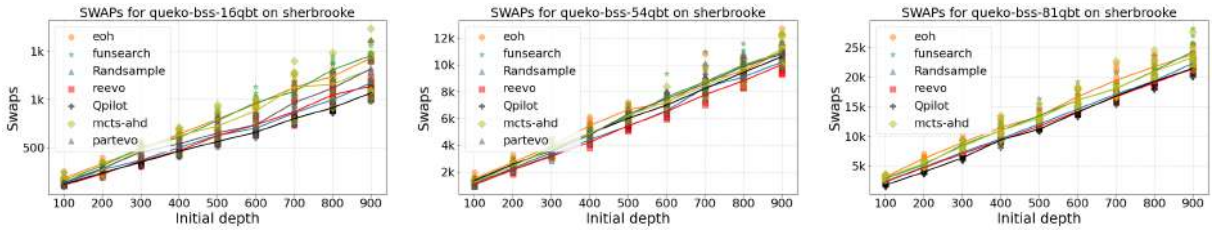
The *Medium* category, summarized in Table 6.2, paints a more nuanced picture of these shallower circuits. QPilot still returns the shallowest mappings (depth 284.21), cutting the critical path by 5.21% against QMAP and 7.31% against Cirq, but it no longer holds the outright SWAP lead: QMAP inserts fewer SWAPs (92.05 against 99.84), giving it an 8.46% gate-count edge, while QPilot remains 5.21% shallower. Against Cirq, QPilot keeps a marginal SWAP margin (0.32%), while maintaining a substantial lead over Pytket (28.55%). Where routing slack is plentiful, the strongest mappers converge to a near-tie on certain metrics, yet QPilot preserves the depth advantage that matters most for decoherence.

Comparison with LLM-based AHD Methods

Now we evaluate the effectiveness of QPilot search methodology. In this section, we compare the heuristics discovered by QPilot against those generated by other state-of-the-art LLM-driven Automatic Heuristic Design frameworks, namely EoH, FunSearch, ReEvo, MCTS-AHD, PartEvo, and a random sampling baseline (Randsample). By allocating an equivalent search time budget to each framework, this comparison directly evaluates the quality and efficiency of the search pipeline.

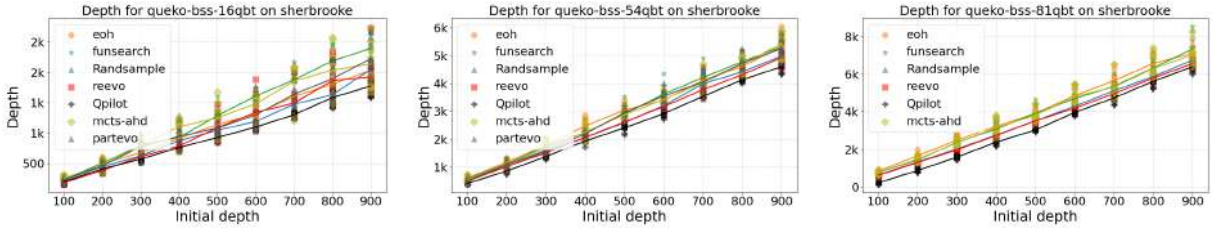
Performance on QUEKO Benchmarks. To assess how the heuristics synthesized by each AHD framework scale, we evaluate them across the synthetic QUEKO suites. Figure 6.5

presents the SWAP counts, while Figure 6.6 displays the final circuit depths for the 16 qubit, 54 qubit, and 81 qubit topologies.



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

Figure 6.5: SWAP gate counts on the IBM Sherbrooke backend across QUEKO benchmarks, QPilot against LLM-based AHD frameworks.



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

Figure 6.6: Final circuit depths on the IBM Sherbrooke backend across QUEKO benchmarks.

The visual trends in Figures 6.5 and 6.6 reveal a clear divergence in the quality of the heuristics found by the different search strategies. As the problem scales to 54 and 81 qubits, purely evolutionary frameworks like FunSearch and EoH, as well as tree-search methods like MCTS-AHD (yellow diamonds), exhibit steeper growth curves. This indicates that their search loops struggled to discover robust, scalable mapping policies within the allotted budget, often overfitting to smaller instances.

In contrast, the heuristic generated by QPilot maintains one of the flattest trajectories: it sits at the lower envelope of final circuit depth across all three scales and shares the lowest SWAP-insertion band with ReEvo, its closest competitor, while the remaining frameworks fan out above. By leveraging a structured, Hypothesis-Driven Knowledge Graph rather than relying on unstructured LLM mutations or random sampling, QPilot navigates the combinatorial heuristic space to synthesize a mapping policy that scales well across the tested sizes.

Performance on Application-Oriented QASMBench. To determine if the advantages observed on synthetic instances translate to real-world algorithmic structures, we evaluated the AHD-generated heuristics on the *Large* category of the QASMBench suite. Table 6.3 reports the average SWAP counts and final depths, alongside QPilot's relative improvement over each competing AHD framework.

The QASMBench empirical data places QPilot at the head of the AHD field on SWAP count, while revealing a genuine trade-off between gate count and critical-path scheduling among the strongest competitors. QPilot attains the lowest SWAP overhead of any LLM-based AHD

Table 6.3: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the IBM Sherbrooke backend (QPilot vs AHD Frameworks).

AHD Method	Absolute Performance		QPilot Improvement	
	Average SWAPs	Average Depth	Δ_{SWAP} (%)	Δ_{Depth} (%)
PartEvo	525.64	935.73	6.92%	3.56%
FunSearch	521.82	917.68	6.24%	1.66%
MCTS-AHD	521.55	921.64	6.19%	2.09%
EoH	519.95	897.68	5.90%	-0.53%
Randsample	526.18	922.09	7.01%	2.13%
ReEvo	498.36	922.18	1.82%	2.14%
QPilot	**489.27**	**902.41**	–	–

method, but it does not hold the outright lead on depth, where EoH proves marginally shallower.

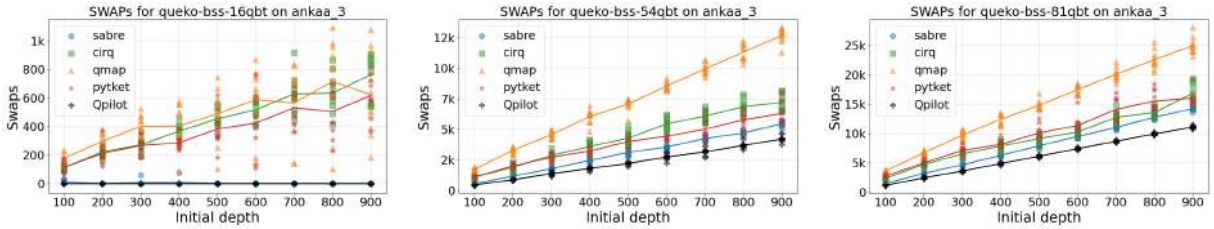
QPilot achieves the lowest average SWAP count (489.27), edging out its closest competitor ReEvo ($\Delta_{\text{SWAP}} = 1.82\%$) and opening wider margins over PartEvo (6.92%), FunSearch (6.24%), MCTS-AHD (6.19%), and EoH (5.90%). The final-depth ranking is tighter and no longer led outright by QPilot: EoH produces marginally shallower circuits on average (897.68 against 902.41), surpassing QPilot by 0.53% on depth, yet it does so while inserting roughly 6% more SWAP gates. QPilot, by contrast, keeps its depth within half a percent of the best AHD result while retaining the lowest gate count, and it remains shallower than every other framework, including ReEvo ($\Delta_{\text{Depth}} = 2.14\%$) and PartEvo ($3.56\%$).

These results temper the picture of clear dominance suggested by the comparison against production compilers, but they remain favorable to QPilot in the dimension that matters most for NISQ fidelity. Because each SWAP decomposes into three CNOT gates and is the dominant contributor to accumulated error, the SWAP-count advantage QPilot holds over every AHD baseline is the more consequential of the two objectives, and it is secured without conceding more than a negligible amount of depth. By systematically accumulating heuristic knowledge and steering the LLM through specific algorithmic hypotheses rather than relying on unstructured reflection or simple evolutionary operators, QPilot lands at or near the Pareto front of the two objectives on these suites, whereas competing frameworks such as EoH purchase a shorter depth at the price of a noticeably higher gate count.

On the *Medium* category, Table 6.4, QPilot recovers the clean two-objective lead that narrowly eluded it on the Large set, posting both the lowest SWAP count (99.84) and the shallowest depth (284.21) of any AHD framework. ReEvo is again the closest pursuer ($\Delta_{\text{SWAP}} = 2.67\%$, $\Delta_{\text{Depth}} = 4.73\%$), while the remaining methods trail by 3–6% on SWAPs and 5–8% on depth. The advantage therefore holds on moderately sized kernels, not only on the most demanding circuits.

Table 6.4: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the IBM Sherbrooke backend (QPilot vs AHD Frameworks).

AHD Method	Absolute Performance		QPilot Improvement	
	Average SWAPs	Average Depth	Δ_{SWAP} (%)	Δ_{Depth} (%)
MCTS-AHD	106.47	308.47	6.23%	7.87%
PartEvo	106.05	300.58	5.86%	5.45%
EoH	105.16	304.37	5.06%	6.62%
Randsample	103.84	298.89	3.85%	4.91%
FunSearch	103.26	299.37	3.31%	5.06%
ReEvo	102.58	298.32	2.67%	4.73%
QPilot	**99.84**	**284.21**	–	–



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

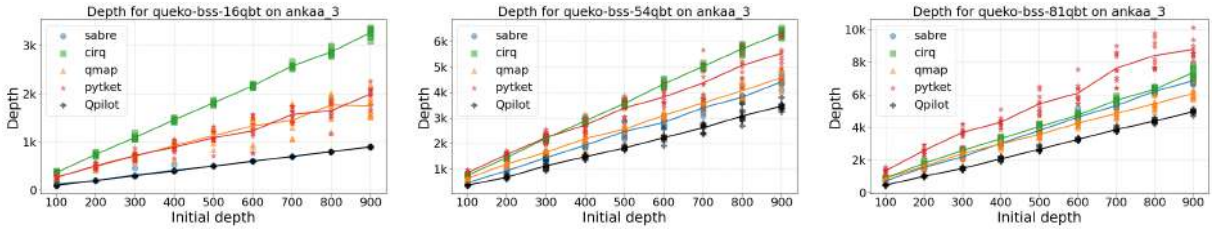
Figure 6.7: SWAP gate counts on the Rigetti Ankaa-3 backend across QUEKO benchmarks.

6.3.3 Evaluation on Rigetti Ankaa-3 backend

We now turn to the Rigetti Ankaa-3 architecture to assess whether the advantages observed on the sparse heavy-hexagon lattice of Sherbrooke carry over to a different backend. We stress that no new search is performed here: the heuristic evaluated is the very same one selected on Sherbrooke (Section 6.2.6), now deployed unchanged on a topology it never observed during generation, making this a held-out test of cross-topology generalization. In contrast to Sherbrooke, the Ankaa-3 processor is organized as a square lattice of tunable couplers, offering a higher average qubit degree. This dense connectivity affords mapping methods more placement flexibility. It therefore serves as a complementary test: whether a generated heuristic can still extract an advantage once the topology itself has relaxed many of the constraints that make mapping difficult. As before, we first benchmark the QPilot best generated heuristic against the mapping passes of established production compilers (SABRE, QMAP, Pytket, and Cirq), before comparing it against the alternative LLM-based AHD frameworks.

Comparison with State-of-the-Art Compilers

Performance on QUEKO Benchmarks. As with the Sherbrooke evaluation, we trace how the methods scale with circuit width and initial depth across the 16 qubit, 54 qubits, and 81 qubits QUEKO benchmark sets. The SWAP gate counts and final circuit depths are presented in Figure 6.7 and Figure 6.8, respectively.



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

Figure 6.8: Final circuit depths on the Rigetti Ankaa-3 backend across QUEKO benchmarks.

On the smallest 16 qubit instances, the abundant connectivity of the square lattice enables QPilot to insert exactly zero SWAP gates across all initial depths, reducing each synthetic circuit to its optimal layout with no routing overhead whatsoever. SABRE comes remarkably close, returning a zero-SWAP mapping for the majority of circuits, for an average of just 2.6 SWAPs. By contrast, Cirq, QMAP, and Pytket accumulate substantial overhead that climbs with depth, averaging several hundred inserted SWAPs over the set. These averages are summarized in Table 6.5. As the circuits widen to 54 and 81 qubits, the available routing slack no longer masks the differences between strategies. QPilot retains the lowest SWAP trajectory at every depth, with SABRE following as the closest classical competitor, while QMAP exhibits the steepest growth and becomes the most overhead-heavy method on these synthetic instances.

Table 6.5: Average SWAP count and final circuit depth on the 16 qubit QUEKO benchmark targeting the Rigetti Ankaa-3 backend.

Method	Average SWAPs	Average Depth
QMAP	473.53	1091.37
Cirq	440.93	1814.62
Pytket	370.84	1100.56
SABRE	2.60	506.02
QPilot	**0.00**	**500.00**

The same ordering governs the final circuit depth in Figure 6.8. QPilot produces the shallowest mapped circuits across all three scales at 16 qubits, it is effectively indistinguishable from SABRE, and on the larger 54 and 81 qubit sets it separates clearly from the rest of the field. The deepest circuits originate from Cirq on the smaller sets and from Pytket on the 81 qubit set, confirming that high SWAP overhead translates directly into a longer critical path on this topology. Even with the additional connectivity working in every method's favor, QPilot preserves a consistent lead on these instances, suggesting that its synthesized strategy does not merely exploit bottleneck links but captures a more general placement principle.

Performance on Application-Oriented QASMBench. We next evaluate the production compilers on the *Large* category of the QASMBench suite. Table 6.6 reports the average absolute SWAP counts and final depths, together with the relative improvement (Δ_{SWAP} and Δ_{Depth}) achieved by QPilot over each baseline.

Table 6.6: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the Rigetti Ankaa-3 backend (QPilot vs Production Compilers).

Method	Absolute Performance		QPilot Improvement	
	Average SWAPs	Average Depth	Δ_{SWAP} (%)	Δ_{Depth} (%)
Cirq	426.72	919.04	21.19%	9.08%
SABRE	358.55	860.27	6.20%	2.86%
Pytket	447.81	905.54	24.90%	7.72%
QMAP	368.63	862.22	8.77%	3.08%
QPilot	**336.32**	**835.64**	–	–

On realistic application circuits, QPilot again secures the best performance on both metrics simultaneously, achieving the lowest average SWAP count (336.32) and final depth (835.64). The widest margin in SWAP counts is recorded against Pytket ($\Delta_{\text{SWAP}} = 24.90\%$), while the widest margin in depth is against Cirq ($\Delta_{\text{Depth}} = 9.08\%$). Against Cirq, QPilot carries a substantial SWAP reduction ($\Delta_{\text{SWAP}} = 21.19\%$), while sustaining notable SWAP reductions against QMAP (8.77%) and SABRE (6.20%).

The most instructive comparison is with MQT QMAP. Whereas QMAP is the weakest method on the synthetic QUEKO instances, it emerges as a very strong classical competitor on structured application circuits, trailing QPilot by 8.77% in SWAPs and 3.08% in depth, though SABRE actually comes closest in depth (2.86%). This narrow gap shows that QPilot's edge over highly tuned classical mappers on this backend is real but slim. Where the backends differ more clearly is in the depth advantage QPilot holds over the remaining compilers: against Cirq, SABRE, and Pytket, it shortens the critical path by 9.08%, 2.86%, and 7.72% on Ankaa, down from the higher margins seen on the sparser Sherbrooke lattice. That QPilot's depth gains shrink on the denser grid is the expected signature of richer connectivity: as the lattice supplies more direct routing paths, a poor initial placement can be repaired more cheaply, leaving less room for a superior mapping to compress the schedule. That QPilot nonetheless retains the lead on both metrics across both benchmark families and both connectivity regimes suggests that its advantage is not an artifact of any single topology.

The *Medium* category on Ankaa, Table 6.7, mirrors the trade-off seen on Sherbrooke's medium circuits, sharpened by the denser lattice. QPilot delivers a highly competitive depth (260.32), trimming the critical path against Pytket (2.96%), Cirq (2.56%), and QMAP (4.98%). However, on this well-connected regime, SABRE achieves both the lowest SWAP count and the shallowest depth, undercutting QPilot by 16.40% on SWAPs and 2.11% on critical path depth. Cirq also inserts fewer SWAPs than QPilot, undercutting it by 7.93%. QPilot retains a commanding SWAP lead over Pytket (23.68%) and a marginal edge over QMAP (0.52%). As on the synthetic suites, abundant connectivity lets certain heuristic routers optimize aggressively on smaller kernels, showing that QPilot's primary advantages are balanced across both metrics depending on the specific classical baseline.

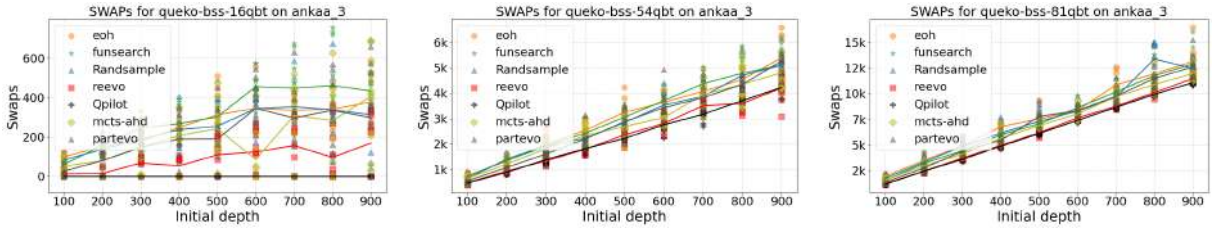
Table 6.7: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the Rigetti Ankaa-3 backend (QPilot vs Production Compilers).

Method	Absolute Performance		QPilot Improvement	
	Average SWAPs	Average Depth	Δ_{SWAP} (%)	Δ_{Depth} (%)
QMAP	61.21	273.95	0.52%	4.98%
Cirq	56.42	267.16	-7.93%	2.56%
SABRE	52.32	254.95	-16.40%	-2.11%
Pytket	79.79	268.26	23.68%	2.96%
QPilot	**60.89**	**260.32**	–	–

Comparison with Alternative LLM-based AHD Methods

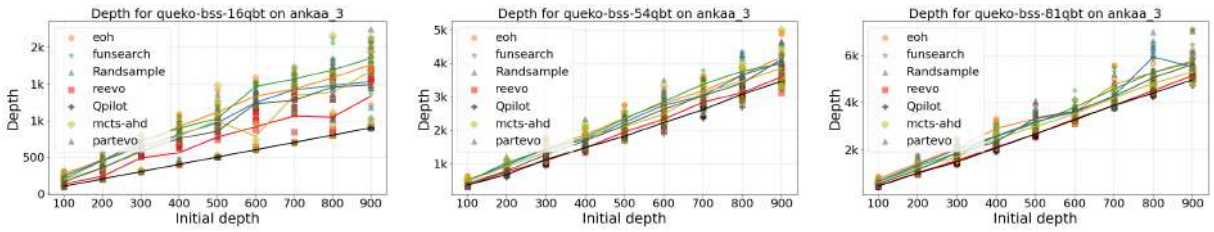
We now isolate the contribution of the QPilot search methodology by comparing its discovered heuristic against those produced by the alternative LLM-driven AHD frameworks, namely EoH, FunSearch, ReEvo, MCTS-AHD, PartEvo, and the random sampling baseline (Randsample), each allocated the same search-time budget on the Ankaa-3 target.

Performance on QUEKO Benchmarks. Figure 6.9 presents the SWAP counts and Figure 6.10 the final circuit depths obtained by each AHD framework across the 16 qubit, 54 qubit, and 81 qubit QUEKO suites.



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

Figure 6.9: SWAP gate counts on the Rigetti Ankaa-3 backend across QUEKO benchmarks, comparing QPilot against alternative LLM-based AHD frameworks.



(a) 16 qubit QUEKO (b) 54 qubit QUEKO (c) 81 qubit QUEKO

Figure 6.10: Final circuit depths on the Rigetti Ankaa-3 backend across QUEKO benchmarks.

The competitive landscape among AHD methods differs significantly from that observed on the Sherbrooke backend. On the 16 qubit set, QPilot stands apart from the entire field:

it alone recovers the known-optimal mapping of every instance with zero inserted SWAPs, whereas ReEvo, the nearest AHD competitor, still averages roughly 89 SWAPs, and the remaining frameworks trail considerably further behind, as summarized in Table 6.8. As the circuits widen to 54 and 81 qubits, QPilot and ReEvo converge into a clearly dominant pair, their SWAP and depth curves tightly clustered at the lower bound, with QPilot retaining a slim but consistent edge. The remaining frameworks, EoH, Randsample, FunSearch, PartEvo, and MCTS-AHD, trail with visibly steeper trajectories, a separation that widens at the 81 qubit scale, where the deepest and most SWAP-heavy curves belong to EoH and Randsample. The two-method lead is itself informative: ReEvo is QPilot's nearest pursuer here, and as the QASMBench comparison below shows, it occupies the same runner-up position on the sparse Sherbrooke lattice, so this leading pair reflects the search methodology rather than any one topology. What is distinctive about the dense grid is the 16 qubit regime, where the abundant connectivity lets QPilot solve every instance exactly and open the wide margin over ReEvo seen on the smallest set.

Table 6.8: Average SWAP count and final circuit depth on the 16 qubit QUEKO targeting the Rigetti Ankaa-3 backend.

AHD Method	Average SWAPs	Average Depth
FunSearch	315.03	1096.02
EoH	265.81	1058.90
Randsample	250.19	975.14
PartEvo	212.51	904.47
MCTS-AHD	201.38	900.90
ReEvo	89.33	726.40
QPilot	**0.00**	**500.00**

Performance on Application-Oriented QASMBench. Table 6.9 reports the average SWAP counts and final depths on the *Large* category of QASMBench, alongside QPilot's relative improvement over each competing AHD framework.

The QASMBench results confirm the picture drawn by the synthetic suites. QPilot delivers the lowest average SWAP count (336.32) and the shallowest final depth (835.64), retaining its lead on both metrics at once. The decisive observation is the proximity of ReEvo, which trails by a mere 1.60% in SWAPs and 0.94% in depth, by far the closest any competitor approaches QPilot across either backend. The remaining methods spread out behind this leading pair: EoH is overtaken by the widest margin ($\Delta_{\text{SWAP}} = 13.66\%$), followed by Randsample (10.52%), FunSearch (9.50%), PartEvo (7.30%), and MCTS-AHD (6.34%).

Relative to Sherbrooke, the dense lattice is in several respects more favorable to QPilot. Its SWAP-count lead over the weaker AHD frameworks is wider here, exceeding 13% against EoH and 10% against Randsample where the corresponding Sherbrooke gaps sit near 6% and 4%, while ReEvo remains its nearest pursuer on both backends (1.60% here against 1.82% on Sherbrooke). QPilot also leads on both metrics simultaneously on Ankaa, whereas on Sherbrooke EoH narrowly undercuts it on depth, so the clean two-objective win that eluded QPilot

Table 6.9: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Large* benchmark set targeting the Rigetti Ankaa-3 backend. Improvements indicate the percentage reduction achieved by QPilot compared to other LLM-based AHD methods.

AHD Method	Absolute Performance		QPilot Improvement	
	Average SWAPs	Average Depth	Δ_{SWAP} (%)	Δ_{Depth} (%)
EoH	389.55	905.32	13.66%	7.70%
Randsample	375.86	884.68	10.52%	5.54%
FunSearch	371.64	870.27	9.50%	3.98%
PartEvo	362.82	881.50	7.30%	5.20%
MCTS-AHD	359.09	883.36	6.34%	5.40%
ReEvo	341.77	843.55	1.60%	0.94%
QPilot	**336.32**	**835.64**	–	–

on the heavy-hexagon lattice is recovered on the square grid. Across both architectures QPilot is not displaced from the top of the SWAP ranking on the *Large* suite and sits at or above the depth frontier, and the consistency of this leadership across opposing connectivity profiles, sparse heavy-hexagon and dense square lattice, provides evidence that the heuristics it discovers encode transferable mapping principles rather than topology-specific exploits.

Among the AHD frameworks on the *Medium* category, Table 6.10, QPilot holds the shallowest depth (260.32) but cedes the SWAP lead to EoH, which inserts marginally fewer gates (60.00 against 60.89) for a 1.49% edge while running 1.83% deeper. ReEvo once more hugs QPilot closely ($\Delta_{\text{SWAP}} = 1.62\%$, $\Delta_{\text{Depth}} = 0.54\%$), whereas FunSearch, MCTS-AHD, and PartEvo trail by 4–6% on SWAPs. The depth advantage is the robust one: QPilot is shallower than every AHD competitor on both backends, even where a rival shaves a fraction off the gate count.

Table 6.10: Average SWAP count, final circuit depth, and relative improvements on the QASMBench *Medium* benchmark set targeting the Rigetti Ankaa-3 backend. Improvements indicate the percentage reduction achieved by QPilot compared to other LLM-based AHD methods.

AHD Method	Absolute Performance		QPilot Improvement	
	Average SWAPs	Average Depth	Δ_{SWAP} (%)	Δ_{Depth} (%)
FunSearch	64.63	269.79	5.78%	3.51%
MCTS-AHD	64.16	281.84	5.09%	7.64%
PartEvo	63.63	274.05	4.30%	5.01%
Randsample	62.11	272.00	1.95%	4.30%
ReEvo	61.89	261.74	1.62%	0.54%
EoH	60.00	265.16	-1.49%	1.83%
QPilot	**60.89**	**260.32**	–	–

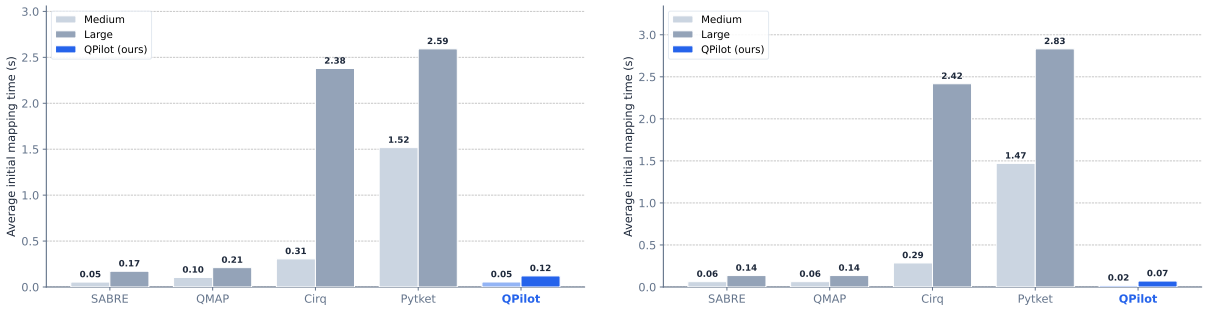
6.3.4 Mapping-Time Comparison.

The token budget analyzed above is a one-off cost, paid a single time when the heuristic is discovered. The cost that recurs at deployment is the *per-circuit mapping time*: the wall-clock time a finished heuristic spends computing an initial placement for each circuit it is asked to route. A heuristic that maps a circuit in milliseconds can sit inside an interactive compilation pipeline, whereas one that takes seconds becomes a bottleneck once thousands of circuits must be compiled. We therefore close the evaluation by measuring the average initial mapping time of the QPilot heuristic against both families of baselines, on the *Medium* and *Large* QASMBench suites, and on both backends.

Against production compilers. Table 6.11 and Figure 6.11 report the timings against the production compilers. QPilot is the fastest mapper in the entire comparison on the demanding *Large* suite, routing a circuit in 0.12 s on Sherbrooke and 0.07 s on Ankaa-3. This places it ahead of even the lightweight greedy passes of SABRE (0.17 s / 0.14 s) and QMAP (0.21 s / 0.14 s), and roughly twenty times faster than the search-heavy routines of Cirq (2.38 s / 2.42 s) and Pytket (2.59 s / 2.83 s). The ordering carries over to the shallower *Medium* circuits, where QPilot's 0.02 s on Ankaa-3 is the lowest of any compiler, and its 0.05 s on Sherbrooke matches SABRE while halving QMAP's 0.10 s. Cirq and Pytket remain one to two orders of magnitude slower throughout, reflecting the more expensive layout search they perform on every circuit. The QPilot heuristic thus delivers the quality advantages reported in Section 6.3.2 and Section 6.3.3 with no penalty in mapping latency on the contrary, it is among the most economical options in the field on these measurements.

Table 6.11: Average initial mapping time (in seconds) per compiler on the QASMBench *Medium* and *Large* benchmark sets, for the IBM Sherbrooke and Rigetti Ankaa-3 backends.

Mapper	**Sherbrooke**		**Ankaa-3**	
	Medium	**Large**	**Medium**	**Large**
SABRE	0.05	0.17	0.06	0.14
QMAP	0.10	0.21	0.06	0.14
Cirq	0.31	2.38	0.29	2.42
Pytket	1.52	2.59	1.47	2.83
QPilot	**0.05**	**0.12**	**0.02**	**0.07**



(a) IBM Sherbrooke

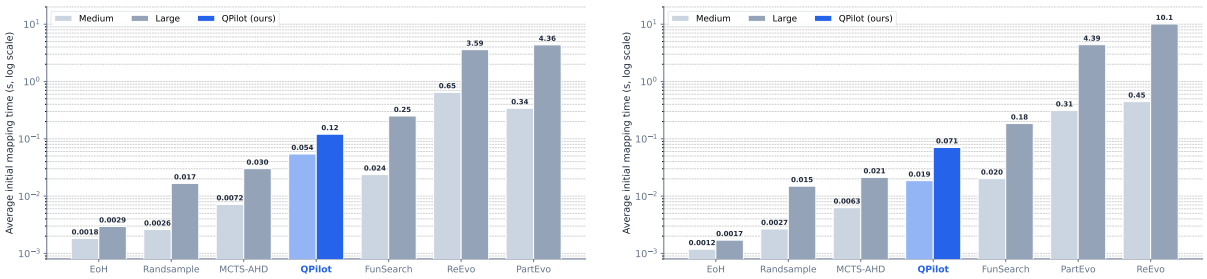
(b) Rigetti Ankaa-3

Figure 6.11: Average initial mapping time per production compiler on the QASMBench *Medium* and *Large* benchmark sets.

Against LLM-based AHD methods. Table 6.12 and Figure 6.12 repeat the measurement against the LLM-based AHD heuristics, whose timings span almost four orders of magnitude and are therefore plotted on a logarithmic axis. QPilot does not produce the single fastest heuristic: the simplest baselines, EoH, Randsample, and MCTS-AHD, map circuits in single-digit milliseconds because the policies they encode are correspondingly thin, but as the quality results of Tables 6.3 and 6.9 showed, that thrift is paid for in markedly worse mappings. The more revealing comparison is with the heuristics that actually rival QPilot on quality. ReEvo, QPilot's nearest competitor on both backends, generates a heuristic that is one to two orders of magnitude slower to apply: 10.1 s per *Large* circuit on Ankaa-3 and 3.59 s on Sherbrooke, against QPilot's 0.07 s and 0.12 s. PartEvo is similarly heavy (4.39 s / 4.36 s), and FunSearch trails by a factor of two to three (0.18 s / 0.25 s). QPilot therefore occupies a uniquely favorable position: it stays firmly in the sub-second mapping regime shared by the lightest heuristics, yet the only framework that approaches its mapping quality, ReEvo, does so with a heuristic that is tens to hundreds of times more expensive to run.

Table 6.12: Average initial mapping time (in seconds) of the LLM-based AHD heuristics on the QASMBench *Medium* and *Large* benchmark sets, for the IBM Sherbrooke and Rigetti Ankaa-3 backends.

AHD Method	**Sherbrooke**		**Ankaa-3**	
	Medium	**Large**	**Medium**	**Large**
EoH	0.0018	0.0029	0.0012	0.0017
Randsample	0.0026	0.017	0.0027	0.015
MCTS-AHD	0.0072	0.030	0.0063	0.021
FunSearch	0.024	0.25	0.020	0.18
PartEvo	0.34	4.36	0.31	4.39
ReEvo	0.65	3.59	0.45	10.1
QPilot	**0.054**	**0.12**	**0.019**	**0.071**



(a) IBM Sherbrooke

(b) Rigetti Ankaa-3

Figure 6.12: Average initial mapping time of the LLM-based AHD heuristics on the QASMBench *Medium* and *Large* benchmark sets.

Taken together with the search-cost analysis, this completes the efficiency picture. QPilot reaches state-of-the-art heuristics while consuming a fraction of the search tokens demanded by the strongest evolutionary competitors, and the heuristics it discovers are simultaneously among the cheapest to deploy, mapping application circuits faster than every production compiler and faster than every AHD method of comparable quality. The structured, knowledge-grounded search does not merely find better mappings it finds mappings that are also fast to compute, a combination that none of the evaluated baselines attains on both axes at once.

6.3.5 Token Consumption and Generation Efficiency

To evaluate search costs, we analyze cumulative token consumption and generation throughput under identical budgets (Figure 6.13). QPilot in the search consumed just 142k tokens, second only to the unguided, context-free Randsample framework (95k). In contrast, all other principled strategies incur heavy overhead from maintaining expanded prompt contexts, with PartEvo and FunSearch consuming $\sim$ 220k tokens, EoH 258k, and ReEvo and MCTS-AHD reaching 420k and 477k respectively (roughly $3\times$ QPilot's budget). High token spend does not yield proportional productivity while ReEvo and EoH generate the most raw heuristics (91 and 74), they burn massive budgets to do so. Normalized as throughput (heuristics per thousand tokens), QPilot leads all reflective and evolutionary methods at 0.30 (spending 3.3k tokens per heuristic), while EoH (0.29), ReEvo (0.22), FunSearch (0.18), MCTS-AHD (0.13), and PartEvo (0.12) lag significantly behind. These metrics indicate QPilot's favorable Pareto position: Randsample is cheaper but produces lower quality heuristics (e.g., a $\Delta_{\text{SWAP}} = 10.52\%$ deficit on the Ankaa lattice), while ReEvo and MCTS-AHD commit the largest budgets without outperforming QPilot on the metrics considered. Overall, by grounding its search in a compact *Hypothesis-Driven Knowledge Graph* rather than bloated, re-summarized prompt histories, QPilot extracts substantial guidance from each query, reaching state-of-the-art results at a fraction of the cost.

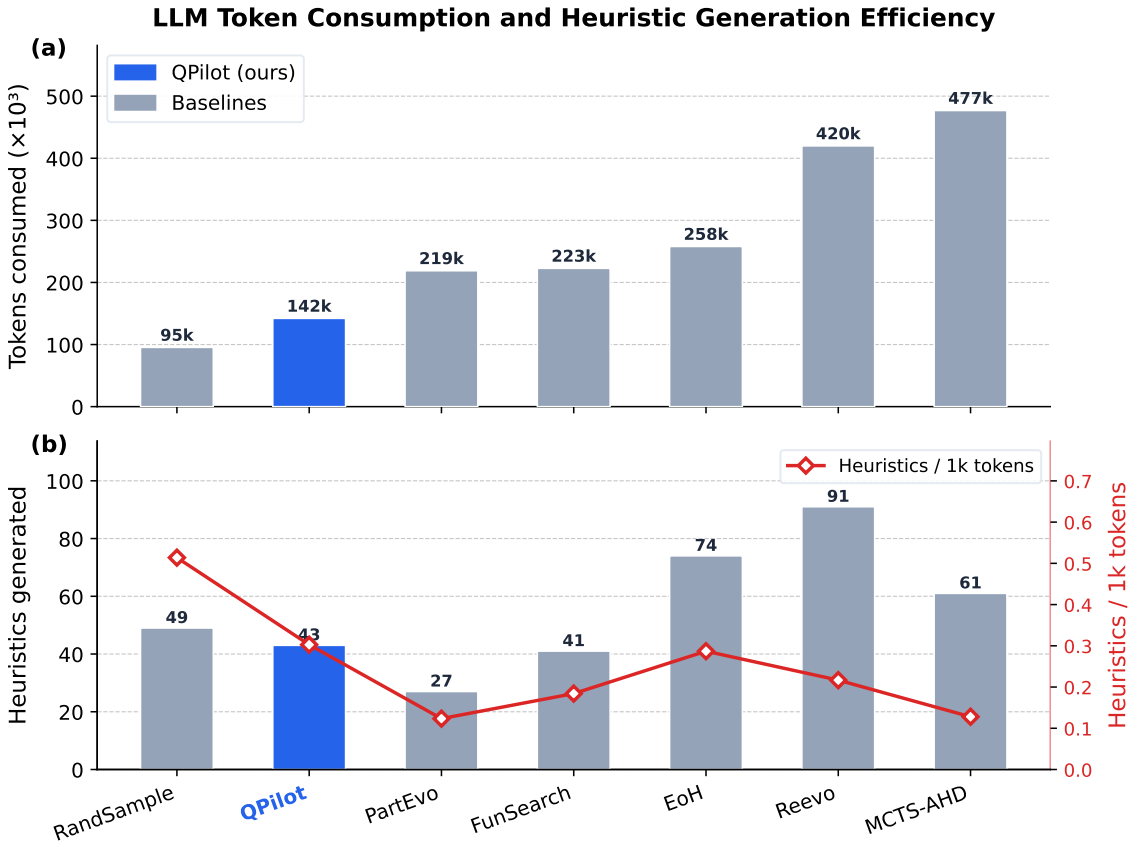


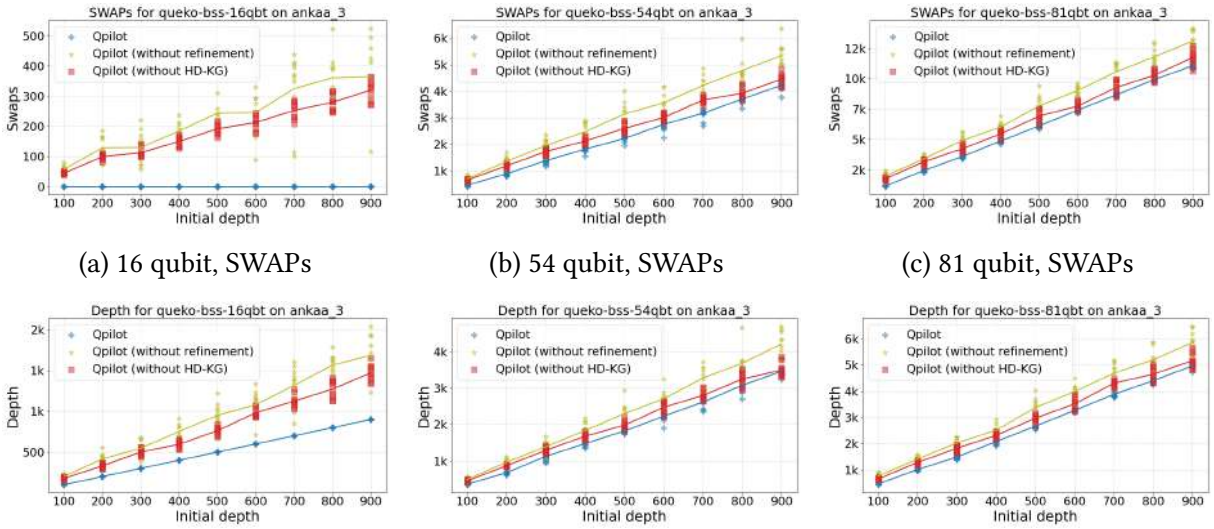
Figure 6.13: LLM token consumption and heuristic generation efficiency of the AHD frameworks.

6.3.6 Ablation Study: Hypothesis-Driven Knowledge Graph and Evolutionary Refinement

The Hypothesis-Driven Knowledge Graph and the Evolutionary Refinement it conditions are the two mechanisms that distinguish QPilot from a plain idea-generation loop. To isolate their individual contributions, we construct two reduced variants of the framework. The first, *without refinement*, removes crossover and mutation entirely, collapsing the search into an *ideation-only* loop of successive idea generation, implementation, and evaluation. The second, *without HD-KG*, retains the Evolutionary Refinement but strips the knowledge graph that conditions it, so that crossover and mutation proceed with no accumulated, structured knowledge to steer subsequent queries. Both variants are re-run on the same IBM Sherbrooke target under an equivalent configuration, and the heuristic each produces is then deployed unchanged on the Rigetti Ankaa-3 QUEKO suites, so that the quality cost of removing each component can be read directly from the deployed heuristic.

Figure 6.14 applies the full QPilot heuristic and its two ablated variants to the Ankaa-3 QUEKO suites across the three width scales. On every scale and for both metrics, the full heuristic dominates: its curve lies lowest throughout, inserting the fewest SWAPs and producing the shallowest circuits, while the *without HD-KG* curve sits consistently above it and the *without refinement* curve runs highest of all. The separation is most pronounced on the 16 qubits set, where QPilot recovers the known-optimal zero-SWAP mapping at every initial

depth and stays flat along the axis, whereas both ablated curves climb steadily with the initial depth and fan apart, with correspondingly deeper critical paths. On the 54 and 81 qubit suites all three curves grow with initial depth, but the two ablated curves remain consistently above QPilot's by margins that widen with circuit size, the ideation-only curve pulling away fastest. This ordering, full below *without HD-KG* below *without refinement*, holds across both SWAP count and final depth and across all three scales. The ablation therefore confirms that both mechanisms contribute to QPilot's mapping quality and that they are jointly responsible for the heuristics the framework discovers: removing either component yields a markedly weaker heuristic at deployment, and removing the refinement entirely is the more damaging of the two.



(a) 16 qubit, SWAPs (b) 54 qubit, SWAPs (c) 81 qubit, SWAPs

(d) 16 qubit, depth (e) 54 qubit, depth (f) 81 qubit, depth

Figure 6.14: SWAP counts and final circuit depths on the Rigetti Ankaa-3 QUEKO suites, comparing the full QPilot heuristic against its two ablated variants: *without refinement* (ideation-only) and *without HD-KG*.

6.4 Conclusion

This chapter evaluated the qubit-mapping heuristics synthesized by QPilot across two contrasting superconducting architectures, two benchmark families, and two families of baselines. Against the mapping passes of production compilers, the discovered heuristic returned the lowest SWAP counts and shallowest circuits on the demanding QASMBench *Large* suite on both backends, outperforming SABRE, Pytket, QMAP and Cirq on the synthetic QUEKO suites, it held the flattest growth curves at every scale, recovering the known-optimal zero-SWAP mapping on the well-connected 16 qubit instances of Ankaa-3. Against the alternative LLM-based AHD frameworks, QPilot occupied the head of the field on SWAP count and sat at or above the depth frontier, with ReEvo as its only consistent pursuer, while the margin narrowed on shallow, well-connected kernels where abundant routing slack lets the simplest mappers close the gap.

These quality gains were obtained efficiently on both of the axes that matter at deployment: on the suites considered, QPilot maps application circuits faster than every production

compiler and faster than most AHD heuristics of comparable quality, and it reaches its result while consuming a fraction of the search tokens demanded by the strongest evolutionary competitors. The concluding ablation traced this combination of efficiency and quality back to the Hypothesis-Driven Knowledge Graph, whose removal degrades the search into untrended random sampling and yields a noticeably weaker deployed heuristic. Taken together, the evidence suggests that QPilot discovers transferable mapping principles rather than topology-specific exploits, a finding we situate within the broader contributions of this thesis in the Conclusion Chapter.

Conclusion

The practical value of quantum computing relies heavily on compiling algorithms to respect strict hardware constraints like limited qubit connectivity. Within this pipeline, the **qubit mapping** problem is a major bottleneck, as topological mismatches require routing operations (SWAP gates) that increase circuit depth and reduce execution fidelity on NISQ devices. We focus specifically on the **initial mapping** problem: selecting the starting placement of logical qubits. The central challenge is producing scalable and robust mapping policies that minimize this overhead across diverse topologies. We address this by shifting the design of mapping heuristics from a manual, intuition-driven craft to an autonomous, evidence-driven discovery process.

In this manuscript, Chapter 1 established the foundations of quantum computation and the compilation constraints that motivate hardware-aware mapping. Chapter 2 surveyed the landscape of qubit mapping approaches, highlighting why scalable compilation in practice relies on heuristics and why hand-engineering their scoring functions becomes a recurring bottleneck as devices diversify. Chapters 3 and 4 then introduced Large Language Models and the emerging paradigm of LLM-based Automatic Heuristic Design (LLM-AHD), positioning them as a mechanism for generating interpretable, deployable heuristic code rather than opaque learned policies.

Building on this synthesis, Chapter 5 introduced **QPilot**, an LLM-driven AHD framework applied to the initial qubit mapping problem. QPilot organizes LLM generation into a closed-loop pipeline that is grounded in domain knowledge, verified by empirical benchmarking, and refined by structured evolution. Its central contribution is the **Hypothesis-Driven Knowledge Graph (HD-KG)**, which converts unstructured reflections into discrete traits and falsifiable causal hypotheses with explicit confidence and validation status. By conditioning crossover and mutation on these hypotheses, QPilot turns program evolution into a sequence of controlled experiments: each generated offspring tests an explicit claim about mapping behavior, and its measured performance updates the search knowledge rather than being discarded as anecdotal feedback.

Chapter 6 evaluated the best heuristic discovered by QPilot across two complementary benchmark families (synthetic QUEKO instances and application-oriented QASMBench circuits) and two contrasting superconducting architectures (IBM Sherbrooke's sparse heavy-hexagon lattice and Rigetti Ankaa-3's denser square grid). Against production compilers, QPilot achieved the strongest overall mappings on almost all benchmarks.

Beyond mapping quality, the evaluation also established that QPilot is practical in both search cost and deployment latency. QPilot reached its best-performing heuristic within a 142k-token budget and at a higher heuristic-per-token throughput than competing AHD

frameworks (Section 6.3.5), outperforming alternative LLM-driven method on SWAP count across both backends, by margins ranging from 1.60% over its closest competitor (ReEvo) to 13.66% over the weakest (EoH on the Ankaa-3 *Large* suite), while maintaining competitive depth. Importantly, the resulting heuristic does not increase initial mapping time, matching or outperforming every tested production compiler on mapping time (Table 6.11). Finally, an ablation study isolated the impact of the HD-KG itself (Section 6.3.6): removing the knowledge graph and evolutionary refinement degraded the search into untrended sampling and produced substantially weaker deployed heuristics, confirming that QPilot's gains derive from structured, hypothesis-driven refinement rather than from generation volume alone.

Several promising research directions follow naturally from this work. First, since QPilot currently tackles only the initial placement part of the quantum mapping stage, future work could extend it to co-design downstream routing and scheduling decisions, treating the compilation pipeline as a coupled multi-stage optimization problem rather than a sequence of isolated passes. Second, the framework could be generalized to support multi-objective optimization, optimizing simultaneously for SWAP count, circuit depth, and noise-aware metrics to align the search directly with execution fidelity. Third, QPilot could be enhanced to target multiple backends simultaneously during the training phase, learning reusable hypothesis libraries that generalize across diverse families of coupling graphs to ensure robust cross-topology transfer.

In summary, LLM-based Automatic Heuristic Design grounded in empirical evidence and explicit hypotheses can discover fast, interpretable, and architecture-aware quantum mapping heuristics that rival or exceed hand-engineered routines. This work also demonstrates the potential of LLMs to accelerate and improve the design of critical compiler components.

Bibliography

Achiam, Josh, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade ribbon Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal rectangle Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Pall Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley presidential Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, and Alec Radford (2023). "GPT-4 Technical Report." In: *arXiv e-prints arXiv:2303.08774*.

Alexander, Thomas, Naoki Kanazawa, Daniel J. Egger, Lauren Capelluto, Christopher J. Wood, Ali Javadi-Abhari, and David C. McKay (2020). "Qiskit Pulse: Programming Quantum Computers Through the Cloud with Pulses." In: *Quantum Science and Technology* 5.4, p. 044006.

Amy, Matthew, Dmitri Maslov, Michele Mosca, and Martin Roetteler (2013). "A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits." In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32.6, pp. 818–830.

Baghdadi, Riyadh (2015). "Improving tiling, reducing compilation time, and extending the scope of polyhedral compilation." PhD thesis. Paris 6.

Baghdadi, Riyadh, Albert Cohen, Tobias Grosser, Sven Verdoolaege, Javed Absar, Sven Van Haastregt, Alexey Kravets, Anton Lokhmotov, and Alastair Donaldson (2015). "PENCIL Language Specification." PhD thesis. INRIA.

Baghdadi, Riyadh, Albert Cohen, Serge Guelton, Sven Verdoolaege, Jun Inoue, Tobias Grosser, Georgia Kouveli, Alexey Kravets, Anton Lokhmotov, Cedric Nugteren, et al. (2013). "PENCIL: Towards a platform-neutral compute intermediate language for DSLs." In: *arXiv preprint arXiv:1302.5586*.

Baghdadi, Riyadh, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Patricia Suriana, Shoaib Kamil, and Saman P Amarasinghe (2018). "Tiramisu: A code optimization framework for high performance systems." In: *arXiv preprint arXiv:1804.10694*.

Barenco, Adriano, Charles H Bennett, Richard Cleve, David P DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A Smolin, and Harald Weinfurter (1995). "Elementary gates for quantum computation." In: *Physical review A* 52.5, p. 3457.

Benbetka, Marouane, Merwan Bekkar, Riyadh Baghdadi, and Martin Kong (2026). "Dependence-Driven, Scalable Quantum Circuit Mapping with Affine Abstractions." In: *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 268–280.

Besta, Maciej, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler (2024). "Graph of thoughts: Solving elaborate problems with large language models." In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 38. 16, pp. 17682–17690.

Bhattacharjee, Debjyoti and Anupam Chattopadhyay (2017). "Depth-Optimal Quantum Circuit Placement for Arbitrary Topologies." In: *arXiv e-prints arXiv:1703.08540*. arXiv: 1703.08540.

Bhattacharjee, Debjyoti, Abdullah Ash Saki, Mahabubul Alam, Anupam Chattopadhyay, and Swaroop Ghosh (2019). "MUQUT: Multi-constraint quantum circuit mapping on NISQ computers." In: *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, pp. 1–7.

Boiko, Daniil A, Robert MacKnight, Benjamin Kline, and Gabe Gomes (2023). "Autonomous chemical research with large language models." In: *Nature* 624.7992, pp. 570–578.

Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle

Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang (2021). "On the Opportunities and Risks of Foundation Models." In: *arXiv e-prints arXiv:2108.07258*.

Bondhugula, Uday, Albert Hartono, J. Ramanujam, and P. Sadayappan (2008). "A practical automatic polyhedral parallelizer and locality optimizer." In: *PLDI*, pp. 101–113.

Born, Max (1955). "Statistical interpretation of quantum mechanics." In: *Science* 122.3172, pp. 675–679.

Brooks, Rodney (2003). "A robust layered control system for a mobile robot." In: *IEEE Journal on Robotics and Automation* 2.1, pp. 14–23.

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei (2020). "Language models are few-shot learners." In: *Advances in neural information processing systems* 33, pp. 1877–1901.

Burke, Edmund, Tim Curtois, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Sanja Petrovic, José A Vázquez-Rodríguez, and Michel Gendreau (2010). "Iterated local search vs. hyper-heuristics: Towards general-purpose search algorithms." In: *IEEE congress on evolutionary computation*. IEEE, pp. 1–8.

Burke, Edmund, Graham Kendall, Jim Newall, Emma Hart, Peter Ross, and Sonia Schulenburg (2003). "Hyper-heuristics: An emerging direction in modern search technology." In: *Handbook of metaheuristics*. Springer, pp. 457–474.

Burke, Edmund K, Michel Gendreau, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Ender Özcan, and Rong Qu (2013). "Hyper-heuristics: A survey of the state of the art." In: *Journal of the Operational Research Society* 64.12, pp. 1695–1724.

Burke, Edmund K, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Ender Özcan, and John R Woodward (2010). "A classification of hyper-heuristic approaches." In: *Handbook of metaheuristics*. Springer, pp. 449–468.

Chakhlevitch, Konstantin and Peter Cowling (2008). "Hyperheuristics: recent developments." In: *Adaptive and multilevel metaheuristics*. Springer, pp. 3–29.

Chen, Mark, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke

Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba (2021). "Evaluating large language models trained on code." In: *arXiv e-prints arXiv:2107.03374*.

Chowdhery, Aakanksha, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel (2023). "Palm: Scaling language modeling with pathways." In: *Journal of machine learning research* 24.240, pp. 1–113.

Christiano, Paul F, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei (2017). "Deep reinforcement learning from human preferences." In: *Advances in neural information processing systems* 30.

Chung, Hyung Won, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei (2024). "Scaling instruction-finetuned language models." In: *Journal of Machine Learning Research* 25.70, pp. 1–53.

Clarke, John and Frank K Wilhelm (2008). "Superconducting quantum bits." In: *Nature* 453.7198, pp. 1031–1042.

Cross, Andrew W, Lev S Bishop, Sarah Sheldon, Paul D Nation, and Jay M Gambetta (2019). "Validating quantum computers using randomized model circuits." In: *Physical Review A* 100.3, p. 032328.

Cummins, Chris, Volker Seeker, Jordi Armengol-Estapé, Aram H Markosyan, Gabriel Synnaeve, and Hugh Leather (2024). "Don't Transform the Code, Code the Transforms: Towards Precise Code Rewriting using LLMs." In: *arXiv e-prints arXiv:2410.08806*.

Dawson, Christopher M and Michael A Nielsen (2006). "The Solovay-Kitaev algorithm." In: *Quantum Information & Computation* 6.1, pp. 81–95.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). "Bert: Pre-training of deep bidirectional transformers for language understanding." In: *Proceedings of the 2019*

conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pp. 4171–4186.

Dirac, Paul Adrien Maurice (1981). *The principles of quantum mechanics*. 27. Oxford university press.

Dong, Xiaofei, Xueqiang Zhang, Weixin Bu, Dan Zhang, and Feng Cao (2024). "A survey of llm-based agents: Theories, technologies, applications and suggestions." In: *2024 3rd International Conference on Artificial Intelligence, Internet of Things and Cloud Computing Technology (AIoTC)*, pp. 407–413.

Du, Yilun, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch (2024). "Improving factuality and reasoning in language models through multiagent debate." In: *Forty-first international conference on machine learning*.

Feautrier, P. (1988). "Array expansion." In: *Proceedings of the 2nd international conference on Supercomputing*. St. Malo, France: ACM, pp. 429–441.

Feautrier, Paul and Christian Lengauer (2011). "The Polyhedron Model." In: *Encyclopedia of Parallel Computing*. Ed. by David Padua. Springer, pp. 1581–1592.

Feigenbaum, E A and J Feldman (1963). "Computers and thought." In: *Computers and Thought*.

Feynman, Richard P (1986). "Quantum mechanical computers." In: *Found. Phys.* 16.6, pp. 507–532.

Ghallab, Malik, Dana Nau, and Paolo Traverso (2004). *Automated Planning: Theory and Practice*. Elsevier.

Grosser, Tobias, Albert Cohen, Justin Holewinski, P. Sadayappan, and Sven Verdoolaege (2014). "Hybrid Hexagonal/Classical Tiling for GPUs." In: *Proceedings of Annual IEEE/ACM International Symposium on Code Generation and Optimization*. CGO '14. Orlando, FL, USA: ACM, 66:66–66:75.

Grosser, Tobias, Armin Groslinger, and Christian Lengauer (2012). "Polly - Performing Polyhedral Optimizations on a Low-Level Intermediate Representation." In: *Parallel Processing Letters* 22.4.

Grover, Lov K (1996). "A fast quantum mechanical algorithm for database search." In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219.

Gushu, Li, Yufei Ding, and Yuan Xie (2019). "Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices." In: *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS '19. Providence, RI, USA: ACM, pp. 1001–1014.

Herbert, Steven (2020). "On the depth overhead incurred when running quantum algorithms on near-term quantum computers with limited qubit connectivity." In: *Quantum Information and Computation* 20.9-10.

Hoffmann, Jordan, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre (2022). "Training compute-optimal large language models." In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pp. 30016–30030.

Hong, Sirui, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. (2024). "MetaGPT: Meta programming for a multi-agent collaborative framework." In: *International Conference on Learning Representations*. Vol. 2024, pp. 23247–23275.

Horodecki, Ryszard, Paweł Horodecki, Michał Horodecki, and Karol Horodecki (2009). "Quantum entanglement." In: *Reviews of modern physics* 81.2, pp. 865–942.

Hu, Qinglong and Qingfu Zhang (2026). "Partition to evolve: Niching-enhanced evolution with llms for automated algorithm discovery." In: *Advances in Neural Information Processing Systems* 38, pp. 154771–154817.

Huang, Ching-Yao, Chi-Hsiang Lien, and Wai-Kei Mak (2022). "Reinforcement Learning and DEAR Framework for Solving the Qubit Mapping Problem." In: *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. ICCAD '22. San Diego, California: Association for Computing Machinery.

Irigoin, F. and R. Triolet (Jan. 1988). "Supernode Partitioning." In: *POPL'88*. San Diego, CA, pp. 319–328.

Jazaeri, Farzan, Arnout Beckers, Armin Tajalli, and Jean-Michel Sallese (2019). "A Review on Quantum Computing: Qubits, Cryogenic Electronics and Cryogenic MOSFET Physics." In: *arXiv e-prints arXiv:1908.02656*.

Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung (2023). "Survey of hallucination in natural language generation." In: *ACM computing surveys* 55.12, pp. 1–38.

Jimenez, Carlos E, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan (2024). "Swe-bench: Can language models resolve real-world github issues?" In: *International Conference on Learning Representations*. Vol. 2024, pp. 54107–54157.

Jurafsky, Daniel and James H. Martin (2009). *Speech and Language Processing*. Pearson.

Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei (2020). "Scaling Laws for Neural Language Models." In: *arXiv e-prints arXiv:2001.08361*.

Korfhage, Robert R (2003). "Algorithm." In: *Encyclopedia of Computer Science*, pp. 36–38.

Koza, John R (1994). *Genetic programming as a means for programming computers by natural selection*. Vol. 4. 2. Springer, pp. 87–112.

Kudo, Taku and John Richardson (2018). "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing." In: pp. 66–71.

Ladd, Thaddeus D, Fedor Jelezko, Raymond Laflamme, Yasunobu Nakamura, Christopher Monroe, and Jeremy Lloyd O'Brien (2010). "Quantum computers." In: *nature* 464.7285, pp. 45–53.

Lamouri, Djamel Rassem, Iheb Nassim Aouadj, Smail Kourta, and Riyadh Baghdadi (2025). "Pearl: Automatic Code Optimization Using Deep Reinforcement Learning." In: *Proceedings of the 39th ACM International Conference on Supercomputing*. ICS '25. Association for Computing Machinery, pp. 959–974.

Lefebvre, Vincent and Paul Feautrier (1998). "Automatic storage management for parallel programs." In: *Parallel Computing* 24, pp. 649–671.

Leibfried, D., R. Blatt, C. Monroe, and D. Wineland (Mar. 2003). "Quantum dynamics of single trapped ions." In: *Rev. Mod. Phys.* 75 (1), pp. 281–324.

Lenat, Douglas B (1982). "The nature of heuristics." In: *Artificial intelligence* 19.2, pp. 189–249.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela (2020). "Retrieval-augmented generation for knowledge-intensive nlp tasks." In: *Advances in Neural Information Processing Systems* 33, pp. 9459–9474.

Li, Ang, Samuel Stein, Sriram Krishnamoorthy, and James Ang (2023). "Qasmbench: A low-level quantum benchmark suite for nisq evaluation and simulation." In: *ACM Transactions on Quantum Computing* 4.2, pp. 1–26.

Li, Guohao, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem (2023). "Camel: Communicative agents for MIND exploration of large language model society." In: *Advances in neural information processing systems* 36, pp. 51991–52008.

Liu, F., Y. Yao, P. Guo, Z. Yang, X. Lin, Z. Zhao, and Q. Zhang (2026). "A Systematic Survey on Large Language Models for Algorithm Design." In: *ACM Computing Surveys* 58.8.

Liu, Fei, Tong Xialiang, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang (Aug. 2024a). "Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model." In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 32201–32223.

Liu, Fei, Yiming Yao, Ping Guo, Zhiyuan Yang, Zhe Zhao, Xi Lin, Xialiang Tong, Kun Mao, Zhichao Lu, Zhenkun Wang, Mingxuan Yuan, and Qingfu Zhang (2026). "A systematic survey on large language models for algorithm design." In: *ACM Computing Surveys* 58.8, pp. 1–32.

Liu, Fei, Rui Zhang, Zhuoliang Xie, Rui Sun, Kai Li, Qinglong Hu, Ping Guo, Xi Lin, Xialiang Tong, Mingxuan Yuan, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang (2024b). "Llm4ad: A platform for algorithm design with large language model." In: *arXiv e-prints arXiv:2412.17287*.

Liu, Nelson F, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang (2024). "Lost in the middle: How language models use long contexts." In: *Transactions of the association for computational linguistics* 12, pp. 157–173.

Luo, Junyu, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang (2025). "Large language model agent: A survey on methodology, applications and challenges." In: *arXiv e-prints arXiv:2503.21460*.

M. Bran, Andres, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller (2024). "Augmenting large language models with chemistry tools." In: *Nature machine intelligence* 6.5, pp. 525–535.

Merouani, Massinissa, Khaled Afif Boudaoud, Iheb Nassim Aouadj, Nassim Tchoulak, Islem Kara Bernou, Hamza Benyamina, Fatima Benbouzid-Si Tayeb, Karima Benatchba, Hugh Leather, and Riyadh Baghdadi (2024). "LOOPer: A Learned Automatic Code Optimizer For Polyhedral Compilers." In: *arXiv preprint arXiv:2403.11522*.

Mikolov, Tomas, Martin Karafiát, Lukas Burget, Jan Cernockỳ, and Sanjeev Khudanpur (2010). "Recurrent neural network based language model." In: *Interspeech*. Vol. 2. 3. Makuhari, pp. 1045–1048.

Murali, Prakash, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi (2019). "Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers." In: *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS '19. Providence, RI, USA: ACM, pp. 1015–1029.

Nielsen, Michael A and Isaac L Chuang (2010). *Quantum computation and quantum information*. Cambridge university press.

Nwana, Hyacinth S (1996). "Software agents: An overview." In: *The Knowledge Engineering Review* 11.3, pp. 205–244.

O'brien, Jeremy L, Akira Furusawa, and Jelena Vučković (2009). "Photonic quantum technologies." In: *Nature photonics* 3.12, pp. 687–695.

Ouyang, Long, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe (2022). "Training language models to follow instructions with human feedback." In: *Advances in Neural Information Processing Systems* 35, pp. 27730–27744.

Packer, Charles, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez (2023). "MemGPT: Towards LLMs as Operating Systems." In: *arXiv e-prints arXiv:2310.08560*.

Paler, Alexandru, Lucian Sasu, Adrian-Cătălin Florea, and Răzvan Andonie (2023). "Machine Learning Optimization of Quantum Circuit Layouts." In: *ACM Transactions on Quantum Computing* 4.2.

Park, Joon Sung, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein (2023). "Generative agents: Interactive simulacra of human behavior." In: *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22.

Pascoal, Gonçalo, João Paulo Fernandes, and Rui Abreu (2024). "Deep Reinforcement Learning Strategies for Noise-Adaptive Qubit Routing." In: *2024 IEEE International Conference on Quantum Software (QSW)*, pp. 146–156.

Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez (2023). "Gorilla: Large language model connected with massive apis." In: *Advances in Neural Information Processing Systems* 37.37.

Phan, Long, Alice Gatti, Nathaniel Li, Adam Khoja, Ryan Kim, Richard Ren, Jason Hausenloy, Oliver Zhang, and Mantas Mazeika (2025). "Humanity's last exam." In: *arXiv e-prints arXiv:2501.14249*.

Poli, Riccardo, William B Langdon, Nicholas F McPhee, and John R Koza (2008). *A Field Guide to Genetic Programming*. Springer, p. 250.

Pouchet, Louis-Noël, Uday Bondhugula, Cédric Bastoul, Albert Cohen, J. Ramanujam, P. Sadayappan, and Nicolas Vasilache (Jan. 2011). "Loop Transformations: Convexity, Pruning and Optimization." In: *38th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL'11)*. Austin, TX: ACM Press, pp. 549–562.

Preskill, John (2018). "Quantum computing in the NISQ era and beyond." In: *Quantum* 2, p. 79.

Qin, Yujia, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. (2024). "Toolllm: Facilitating large language models to master 16000+ real-world apis." In: *International Conference on Learning Representations*. Vol. 2024, pp. 9695–9717.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn (2023). "Direct preference optimization: Your language model is secretly a reward model." In: *Advances in neural information processing systems* 36, pp. 53728–53741.

Rattacaso, Davide, Daniel Jaschke, Marco Ballarin, Ilaria Siloi, and Simone Montangero (2025). "Quantum circuit compilation with quantum computers." In: *Physical Review Research* 7.3, p. 033268.

Romanycia, M H J and F J Pelletier (1985). "What is a heuristic?" In: *Computational Intelligence*.

Romera-Paredes, Bernardino, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi (2024). "Mathematical discoveries from program search with large language models." In: *Nature* 625.7995, pp. 468–475.

Russell, Stuart and Peter Norvig (2010). *Artificial Intelligence: A Modern Approach*. Pearson Education, Inc.

Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom (2023). "Toolformer: Language models can teach themselves to use tools." In: *Advances in neural information processing systems* 36, pp. 68539–68551.

Sennrich, Rico, Barry Haddow, and Alexandra Birch (2016). "Neural machine translation of rare words with subword units." In: *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pp. 1715–1725.

Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao (2023). "Reflexion: Language agents with verbal reinforcement learning." In: *Advances in neural information processing systems* 36, pp. 8634–8652.

Sinha, Animesh, Utkarsh Azad, and Harjinder Singh (2022). "Qubit Routing Using Graph Neural Network Aided Monte Carlo Tree Search." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 36.9, pp. 9935–9943.

Siraichi, Marcos Yukio, Vinícius Fernandes dos Santos, Sylvain Collange, and Fernando Magno Quintao Pereira (2018). "Qubit Allocation." In: *Proceedings of the 2018 International Symposium on Code Generation and Optimization*. CGO 2018. Vienna, Austria: ACM, pp. 113–125.

Sivarajah, Seyon, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan (Nov. 2020). "tket: a retargetable compiler for NISQ devices." In: *Quantum Science and Technology* 6.1, p. 014003.

Stein, Niki van and Thomas Bäck (Apr. 2025). "LLaMEA: A Large Language Model Evolutionary Algorithm for Automatically Generating Metaheuristics." In: *IEEE Transactions on Evolutionary Computation* 29.2, pp. 331–345.

Sutton, Richard S and Andrew Barto (1998). *Reinforcement learning: An introduction*. Vol. 1. 1. MIT press Cambridge.

Tan, Bochen and Jason Cong (2020). "Optimality Study of Existing Quantum Computing Layout Synthesis Tools." In: *IEEE Transactions on Computers*.

Touvron, Hugo, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample (2023). "LLaMA: Open and Efficient Foundation Language Models." In: *arXiv e-prints arXiv:2302.13971*.

Vasilache, Nicolas, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zach DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen (2018). "Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions." In: *CoRR* abs/1802.04730.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin (2017). "Attention is All You Need." In: vol. 30.

Wang, Guanzhi, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar (2024). "Voyager: An Open-Ended Embodied Agent with Large Language Models." In: *Transactions on Machine Learning Research*.

Wang, Kun, Guibin Zhang, Zhenhong Zhou, Jiahao Wu, Miao Yu, Shiqian Zhao, Chenlong Yin, Jinhu Fu, Yibo Yan, Hanjun Luo, Liang Lin, Zhihao Xu, Haolang Lu, Xinye Cao, Xinyun Zhou, Weifei Jin, Fanci Meng, Shicheng Xu, Junyuan Mao, Yu Wang, Hao Wu, Minghe Wang, Fan Zhang, Junfeng Fang, Wenjie Qu, Yue Liu, Chengwei Liu, Yifan Zhang, Qiankun Li, Chongye Guo, Yalan Qin, Zhaoxin Fan, Kai Wang, Yi Ding, Donghai Hong, Jiaming Ji, Yingxin Lai, Zitong Yu, Xinfeng Li, Yifan Jiang, Yanhui Li, Xinyu Deng, Junlin Wu, Dongxia Wang, Yihao Huang, Yufei Guo, Jen-tse Huang, Qiufeng Wang, Xiaolong Jin, Wenxuan Wang, Dongrui Liu, Yanwei Yue, Wenke Huang, Guancheng Wan, Heng Chang, Tianlin Li, Yi Yu, Chenghao Li, Jiawei Li, Lei Bai, Jie Zhang, Qing Guo, Jingyi Wang, Tianlong Chen, Joey Tianyi Zhou, Xiaojun Jia, Weisong Sun, Cong Wu, Jing Chen, Xuming Hu, Yiming Li, Xiao Wang, Ningyu Zhang, Luu Anh Tuan, Guowen Xu, Jiaheng Zhang, Tianwei Zhang, Xingjun Ma, Jindong Gu, Liang Pang, Xiang Wang, Bo An, Jun Sun, Mohit Bansal, Shirui Pan, Lingjuan Lyu, Yuval Elovici, Bhavya Kailkhura, Yaodong Yang, Hongwei Li, Wenyuan Xu, Yizhou Sun, Wei Wang, Qing Li, Ke Tang, Yu-Gang Jiang, Felix Juefei-Xu, Hui Xiong, Xiaofeng Wang, Dacheng Tao, Philip S. Yu, Qingsong Wen, and Yang Liu (2025). "A Comprehensive Survey in LLM (-Agent) Full Stack Safety: Data, Training and Deployment." In: *arXiv e-prints arXiv:2504.15585*.

Wang, Lei, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen (2024). "A survey on large language model based autonomous agents." In: *Frontiers of Computer Science* 18.6, p. 186345.

Wei, Jason, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus (2022a). "Emergent Abilities of Large Language Models." In: *Transactions on Machine Learning Research*.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou (2022b). "Chain-of-thought prompting elicits reasoning in large language models." In: *Advances in neural information processing systems* 35, pp. 24824–24837.

Wille, Robert and Lukas Burgholzer (2023). "MQT QMAP: Efficient quantum circuit mapping." In: *Proceedings of the 2023 International Symposium on Physical Design*, pp. 198–204.

Wille, Robert, Lukas Burgholzer, and Alwin Zulehner (2019). "Mapping quantum circuits to IBM QX architectures using the minimal number of SWAP and H operations." In: *2019 56th ACM/IEEE Design Automation Conference (DAC)*. IEEE, pp. 1–6.

Wolf, Michael E and Monica S Lam (1991). "A loop transformation theory and an algorithm to maximize parallelism." In: *IEEE transactions on parallel and distributed systems* 2.4, pp. 452–471.

Wolpert, David H and William G Macready (2002). "No free lunch theorems for optimization." In: *IEEE transactions on evolutionary computation* 1.1, pp. 67–82.

Wooldridge, Michael (1999). "Intelligent agents." In: *Multiagent systems: A modern approach to distributed artificial intelligence* 1, pp. 27–73.

Wooldridge, Michael and Nicholas R Jennings (1995). "Intelligent agents: Theory and practice." In: *The knowledge engineering review* 10.2, pp. 115–152.

Yang, Chengrun, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen (2024). "Large Language Models as Optimizers." In: *International Conference on Learning Representations*. Ed. by B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun. Vol. 2024, pp. 12028–12068.

Yao, Shunyu, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan (2023a). "Tree of thoughts: Deliberate problem solving with large language models." In: *Advances in neural information processing systems* 36, pp. 11809–11822.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao (2023b). "ReAct: Synergizing Reasoning and Acting in Language Models." In: *The Eleventh International Conference on Learning Representations*.

Ye, Haoran, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song (2024). "Reevo: Large language models as hyper-heuristics with reflective evolution." In: vol. 37, pp. 43571–43608.

Zhang, Chi, Yanhao Chen, Yuwei Jin, Wonsun Ahn, Youtao Zhang, and Eddy Z. Zhang (2020). "A depth-aware swap insertion scheme for the qubit mapping problem." In: *arXiv e-prints arXiv:2002.07289*.

Zhang, Shuoming, Jiacheng Zhao, Qiuchu Yu, Chunwei Xia, Zheng Wang, Xiaobing Feng, and Huimin Cui (2026). "The new compiler stack: a survey on the synergy of LLMs and compilers." In: *CCF Transactions on High Performance Computing*, pp. 1–32.

Zheng, Zhi, Zhuoliang Xie, Zhenkun Wang, and Bryan Hooi (2025). "Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design." In: *Forty-second International Conference on Machine Learning*.

Zhong, Ming, Fang Lv, Lulin Wang, Lei Qiu, Yingying Wang, Ying Liu, Huimin Cui, Xiaobing Feng, and Jingling Xue (2025). "Vega: Automatically generating compiler backends using a pre-trained transformer model." In: *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, pp. 90–106.

Zhu, Chenghong, Xian Wu, Zhaohui Yang, Jingbo Wang, Anbang Wu, Shenggen Zheng, and Xin Wang (2025). "Quantum Compiler Design for Qubit Mapping and Routing: A Cross-Architectural Survey of Superconducting, Trapped-Ion, and Neutral Atom Systems." In: *arXiv e-prints arXiv:2505.16891*.

Zou, Henry, Matthew Treinish, Kevin Hartman, Alexander Ivrii, and Jake Lishman (2024). "LightSABRE: A Lightweight and Enhanced SABRE Algorithm." In: *arXiv e-prints arXiv:2409.08368*.

Zulehner, Alwin, Alexandru Paler, and Robert Wille (2018). "An efficient methodology for mapping quantum circuits to the IBM QX architectures." In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38.7, pp. 1226–1236.

Zulehner, Alwin and Robert Wille (2019). "Compiling SU (4) quantum circuits to IBM QX architectures." In: *Proceedings of the 24th Asia and South Pacific design automation conference*, pp. 185–190.

Webography

Anthropic (2026a). *Introducing Claude Opus 4.7*. https://www.anthropic.com/news/claude-opus-4-7.

— (2026b). *Introducing Claude Sonnet 4.6*. https://www.anthropic.com/news/claude-sonnet-4-6.

Baghdadi, Riyadh, Albert Cohen, Cedric Bastoul, Louis-Noel Pouchet, and Lawrence Rauchwerger (2011). *The Potential of Synergistic Static, Dynamic and Speculative Loop Nest Optimizations for Automatic Parallelization*. arXiv: 1111.6756 [cs.DC].

DeepSeek-AI (2026). *DeepSeek V4 Preview Release*. https://api-docs.deepseek.com/news/news260424.

Google DeepMind (2025). *Gemini*. https://deepmind.google/technologies/gemini/.

IBM (2024). *Qiskit*. https://qiskit.org.

— (2025). *IBM Quantum Sherbrooke QPU*. https://quantum.ibm.com/services/resources?tab=systems&system=ibm_sherbrooke.

OpenAI (2025). *Introducing GPT-5.2*. https://openai.com/index/introducing-gpt-5-2/.

Rigetti (2025). *Rigetti QPUs*. https://qcs.rigetti.com/qpus.